

UESTC_SNOW_HALATION's TEMPLATE

Contents

1. 数学	1
1.1. exgcd	1
1.2. 组合数	1
1.3. 线性求逆元	1
1.4. 杜教筛 (廖)	2
1.5. Pollard rho (lyy)	3
1.6. Lucas	4
1.7. 数论分块 (廖)	5
1.8. 线性筛	6
1.9. Poly	7
1.10. Catalan	11
1.11. Min_25	11
1.12. 求原根	13
1.13. excrt	14
1.14. BSGS	15
2. 其他	16
2.1. 区间加等差数列	16
2.2. make	16
2.3. 子集 FWT	17
2.4. 取模优化	18
2.5. 二维 hash	18
2.6. 树上莫队	18
2.7. 子集卷积	19
2.8. 前缀线性基 PLB	20
2.9. 售货员	22
2.10. 线性基	22
2.11. 子集 dp	23
2.12. ksc	23
2.13. duipai	23
2.14. 异或分段	24
2.15. 火车头	24
3. 几何	25
3.1. cjj	25
3.2. lyy 几何	37
3.3. lyy 凸包	41
4. 图论	41
4.1. tarjan 求点双边数	41
4.2. Stoer-Wagner	43
4.3. 点分治	44
4.4. 差分约束	45
4.5. 圆方树	46
4.6. 并查集维护缩点	47
4.7. 二分图最小点覆盖 (dinic)	48
4.8. 欧拉回路	50
4.9. Seq	51
4.10. 匈牙利算法	52

4.11. 最大流	52
4.12. 虚树	54
4.13. 二分图最大权完美匹配 (KM)	55
4.14. 删边最短路	56
4.15. 树哈希	59
4.16. 二分图最小点覆盖	59
4.17. 带花树	60
4.18. 有源汇上下界最大流	62
4.19. tarjan 求桥和割点	64
4.20. K 短路	65
4.21. 2-sat	67
4.22. 矩阵树定理	68
4.23. 边分治缩点	69
4.24. 费用流	71
5. 字符串	72
5.1. runs (hash)	72
5.2. PAM	74
5.3. manacher	74
5.4. Z 函数	75
5.5. 广义 SAM (离线)	76
5.6. kmp	77
5.7. SA	78
5.8. SAM	78
5.9. 广义 SAM (在线)	80
5.10. AC 自动机	81
5.11. Lyndon 分解	82
5.12. SA+ST	82
6. 多项式	84
6.1. NTT	84
6.2. 多项式分治乘	85
6.3. 多项式 vector	86
6.4. 多项式求逆	90
6.5. 多项式快速幂	91
6.6. 多项式开根	93
6.7. 多项式快速幂 (双 log)	95
6.8. 01 序列生成函数 (小武)	97
6.9. 第一类斯特林数 (行)	98
6.10. 三模数 NTT	100
6.11. 多项式除法	102
6.12. 第二类斯特林数 (行)	104
6.13. FFT	105
6.14. 第二类斯特林数 (列)	106
7. 数据结构	108
7.1. 李超树	108
7.2. 点分树	110
7.3. LCT	112
7.4. 线段树合并 (雨天的尾巴)	113

7.5. 换根树剖	115
7.6. 可持久化 01tire	118
7.7. 树剖	119
7.8. LCA (ST 版)	120
7.9. 左偏树	121
7.10. 李超树 ($kx+b$)	122
7.11. FHQ_Treap_Segment	124
7.12. Splay(下标顺序)	126
7.13. Splay(大小顺序)	127
7.14. 线段树分治+可撤销并查集	130
7.15. MST	131
7.16. 主席树	131
7.17. 李超树可持久化	132
7.18. ST 表	134
7.19. DSU	134
7.20. FHQ_Treap	135

1. 数学

1.1. exgcd

```
int X,Y;

int _exgcd(int A,int B) {
    if(!B) { X = Y = 0; return A; }
    int res = _exgcd(B,A%B);
    X = Y; Y = (res - A*X) / B;
    return res;
}

int exgcd(int A,int B) {
    if(!B) { X=1; Y=0; return A; }
    int res = exgcd(B,A%B);
    int t = X; X = Y; Y = t-A/B*Y;
    return res;
}

int main() {
    int a,b;
    cin>>a>>b;
    int d = exgcd(a,b);
    cout<<a<<"*"<<X<<" + "<<b<<"*"<<Y<<" = "<<d<<"\n";
    return 0;
}
```

1.2. 组合数

```
ll f[N],ni[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

inline ll C(ll aa,ll bb) { return f[aa] * ni[bb] %p * ni[aa-bb] %p; }

void qiu() {
    f[0] = ni[0] = 1;
    for(ll i=1;i<=N-10;i++) f[i] = f[i-1] * i %p;
    ni[N-10] = ksm(f[N-10],p-2);
    for(ll i=N-11;i>=1;i--) ni[i] = ni[i+1] * (i+1) %p;
}
```

1.3. 线性求逆元

```
#include<cstdio>
#define ll long long
using namespace std;
const int maxn=3e6+5;
ll inv[maxn]={0,1};
int main(){
    int n,p;
```

```

scanf("%d%d",&n,&p);
printf("1\n");
for(int i=2;i<=n;i++)
    inv[i]=(ll)p-(p/i)*inv[p%i]%p,printf("%d\n",inv[i]);
return 0;
}

```

1.4. 杜教筛 (廖)

```

#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 2123456;
ll prime[maxn],mu[maxn],sum[maxn],vis[maxn],cnt;
map<ll,ll> mp;
void init()
{
    mu[1] = 1;
    for(int i=2;i<maxn;++i){
        if(!vis[i])prime[++cnt] = i,mu[i] = -1;
        for(int j=1;j<=cnt&&i*prime[j]<maxn;++j){
            vis[i*prime[j]] = 1;
            if(i%prime[j]) mu[i*prime[j]] = - mu[i];
            else{
                mu[i * prime[j]] = 0;
                break;
            }
        }
    }
    for(int i=1;i<maxn;++i)sum[i] = sum[i-1] + mu[i];
}
ll S_mu(ll x)
{
    if(x < maxn) return sum[x];
    if(mp.find(x) != mp.end()) return mp[x];
    ll res = 1ll;
    for(ll l=2,r;l<=x;l=r+1){
        r = x/(x/l);
        res -= S_mu(x/l) * (r - l + 1);
    }
    return mp[x] = res;
}
ll S_phi(ll x)
{
    ll res = 0;
    for(ll l=1,r;l<=x;l=r+1){
        r = x/(x/l);
        res += (S_mu(r) - S_mu(l-1)) * (x / l) * (x / l);
    }
    return (res - 1) / 2 + 1;
}
int main()
{
    ios::sync_with_stdio(false);
    init();
    ll T,x;
    cin>>T;
}

```

```

    while(T--){
        cin>>x;
        cout<<S_phi(x)<<" "<<S_mu(x)<<'\n';
    }
    return 0;
}

```

1.5. Pollard rho (lyy)

```

#define ll long long
#define lll __int128

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;

ll T;
ll n,m;
map <ll,ll> f;
ll zz[] = {2,3,5,7,11,13,17,19,23};

ll gcd(ll aa,ll bb) { return !bb ? aa : gcd(bb,aa%bb); }

inline ll gsc(ll aa,ll bb,ll p) {
    return (lll)aa*bb %p;
    // ll sum = 0;
    // while(bb) {
    //     if(bb&1) sum = (sum+aa) %p;
    //     bb >>= 1; aa = (aa+aa) %p;
    // }
    // return sum;
}

inline ll ksm(ll aa,ll bb,ll p) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = gsc(sum,aa,p);
        bb >>= 1; aa = gsc(aa,aa,p);
    }
    return sum;
}

inline bool miller_rabin(ll h,ll b) {
    ll k = h-1;
    while(k) {
        ll now = ksm(b,k,h);
        if(now!=1 && now!=h-1) return 0;
        if(k&1 || now==h-1) return 1;
        k >>= 1;
    }
    return 1;
}

inline bool prime(ll h) {
    if(h==1) return 0;
    if(h==2 || h==3 || h==5 || h==7 || h==11 || h==13 || h==17 || h==19 || h==23)

```

```

return 1;
    for(ll i=0;i<9;i++) if(!miller_rabin(h,zz[i])) return 0;
    return 1;
}

ll pollard_rho(ll h) { //h cannot be prime
    ll c = 1919810;
    if(h==4) return 2;
    while(1) {
        auto F = [=](ll x) { return (gsc(x,x,h) + c) % h; };
        c--;
        ll t=0, r=0, p=1, q;
        do {
            for(int i=0;i<128;i++) {
                t = F(t); r = F(F(r));
                if(t==r || (q = gsc(p,abs(t-r),h)) == 0) break;
                p = q;
            }
            ll d = gcd(p,h);
            if(d>1) return d;
        } while(t!=r);
    }
}

void divide(ll h) {
    if(h==1) return;
    if(prime(h)) { f[h]++; return ; }
    ll d = pollard_rho(h);
    // cout<<"h = "<<h<<" d = "<<d<<endl;
    // system("pause");
    divide(d); divide(h/d);
}

void chushihua() {
    f.clear();
}

int main() {
    srand(time(NULL));
    T = read();
    while(T--) {
        chushihua();
        n = read();
        if(prime(n)) { cout<<"Prime\n"; continue; }
        divide(n);
        // for(auto it=f.begin();it!=f.end();it++) cout<< it->first <<" ^ "<< it->second <<"\n";
        auto it = --f.end();
        cout<< it->first <<endl;
    }
    return 0;
}

```

1.6. Lucas

```

#include <bits/stdc++.h>
#define ll long long

```

```

using namespace std;
const ll N=301010;
const ll inf=0x3f3f3f3f;

ll T;
ll n,m,p;
ll f[N];

ll ksm(ll a,ll b) { ll sum = 1; while(b) { if(b&1) sum = (sum*a) % p; b >>= 1; a = (a*a) % p; } return sum; }

ll C(ll a,ll b) { if(b>a) return 0; return f[a] * ksm(f[b],p-2) % p * ksm(f[a-b],p-2) % p; }

ll Lucas(ll a,ll b) { if(!b) return 1; return C(a%p,b%p) * Lucas(a/p,b/p) % p; }

inline void qiu() { f[0] = 1; for(ll i=1;i<=n+m;i++) f[i] = f[i-1] * i % p; }

int main() {
    cin>>T;
    while(T-->0) {
        cin>>n>>m>>p;
        qiu();
        cout<<Lucas(n+m,m)<<"\n";
    }
    return 0;
}

```

1.7. 数论分块 (廖)

```

#include<bits/stdc++.h>

using namespace std;
typedef long long ll;

ll G(ll n,ll k)
{
    ll res = n * k;
    for(ll l=1,r;l<=n;l=r+1)
    {
        if(k/l>0)r=min(n,k/(k/l));
        else r = n;
        res -= (l + r) * (r - l + 1) / 2ll * (k / l);
    }
    return res;
}

int main()
{
    ios::sync_with_stdio(false);
    ll x,y;
    cin>>x>>y;
    cout<<G(x,y)<<endl;
    return 0;
}

```

1.8. 线性筛

```
// p[]
for(int i=2; i<=h; i++) {
    if(!vis[i]) p[++cnt] = i;
    for(int j=1; j<=cnt && i*p[j]<=h; j++) {
        vis[ i*p[j] ] = 1;
        if(i%p[j]==0) break;
    }
}

// phi[]
for(int i=2; i<=h; i++) {
    if(!vis[i]) {
        p[++cnt] = i;
        phi[i] = i-1;
    }
    for(int j=1; j<=cnt && i*p[j]<=h; j++) {
        vis[ i*p[j] ] = 1;
        if(i%p[j]==0) {
            phi[ i*p[j] ] = phi[i] * p[j];
            break;
        }
        phi[ i*p[j] ] = phi[i] * (p[j]-1);
    }
}

// d[]    number of factors
d[1] = 1;
for(int i=2; i<=n; i++) {
    if(!vis[i]) {
        p[++cnt] = i;
        d[i] = 2; g[i] = 1;
    }
    for(int j=1; j<=cnt && i*p[j]<=n; j++) {
        vis[ i*p[j] ] = 1;
        if(i%p[j]==0) {
            d[ i*p[j] ] = d[i] / (g[i]+1) * (g[i]+2);
            g[ i*p[j] ] = g[i] + 1;
            break;
        }
        d[ i*p[j] ] = d[i] * 2;
        g[ i*p[j] ] = 1;
    }
}

// f[]    sum of factors
f[1] = 1;
for(int i=2; i<=n; i++) {
    if(!vis[i]) {
        p[++cnt] = i;
        f[i] = i+1; g[i] = 1;
    }
    for(int j=1; j<=cnt && i*p[j]<=n; j++) {
        vis[ i*p[j] ] = 1;
        if(i%p[j]==0) {
            f[ i*p[j] ] = f[i] * p[j] + g[i];
        }
    }
}
```

```

        g[ i*p[j] ] = g[i];
        break;
    }
    f[ i*p[j] ] = f[i] * (p[j]+1);
    g[ i*p[j] ] = f[i];
}
}

// mu[]
mu[1] = 1;
for(int i=2; i<=n; i++) {
    if(!vis[i]) {
        p[++cnt] = i;
        mu[i] = -1;
    }
    for(int j=1; j<=cnt && i*p[j]<=n; j++) {
        vis[ i*p[j] ] = 1;
        if(i%p[j]==0) break;    //mu[i*p[j]]=0; 就没必要写了。
        mu[ i*p[j] ] = -mu[i];
    }
}

```

1.9. Poly

```

#include <iostream>
#include <algorithm>
#include <vector>
#include <cstdio>

using namespace std;

typedef long long ll;

const int N = 1 << 22;
const int M = 3e6 + 7;
const int Mod = 998244353;

int rk[N];

inline ll ModRead()
{
    ll x = 0, flag = 1;
    char c = getchar();
    while (c < '0' || c > '9')
    {
        if (c == '-')
            flag = 0;
        c = getchar();
    }
    while (c >= '0' && c <= '9')
    {
        x = ((x << 1) + (x << 3) + c - 48) % Mod;
        c = getchar();
    }
    return flag ? x : -x;
}

```

```

ll qpow(ll x, ll y = Mod - 2)
{
    ll ret = 1;
    while (y)
    {
        if (y & 1)
            ret = ret * x % Mod;
        x = x * x % Mod, y >>= 1;
    }
    return ret;
}

const ll G = 3ll;
const ll Gi = qpow(3ll);
const ll inv2 = qpow(2ll);

void NTT(const bool &op, const int &n, vector<ll> &F)
{
    for (int i = 0; i < n; i++)
        if (i < rk[i])
            swap(F[i], F[rk[i]]);
    for (int p = 2; p <= n; p <<= 1)
    {
        int len = p >> 1;
        ll w = qpow(op ? G : Gi, (Mod - 1) / p);
        for (int k = 0; k < n; k += p)
        {
            ll now = 1;
            for (int l = k; l < k + len; l++)
            {
                ll t = F[l + len] * now % Mod;
                F[l + len] = (F[l] - t + Mod) % Mod;
                F[l] = (F[l] + t) % Mod;
                now = now * w % Mod;
            }
        }
    }
}

inline void Rk(const int &n)
{
    for (int i = 0; i < n; i++)
        rk[i] = (rk[i >> 1] >> 1) | (i & 1 ? n >> 1 : 0);
}

void print(const vector<ll> &X)
{
    for (const ll &v : X)
        cout << v << " ";
    cout << "!!!\n";
}

void Mul(vector<ll> &X, vector<ll> &a, vector<ll> &b)
{
    static vector<ll> x, y;
    int n = a.size() - 1, m = b.size() - 1;

```

```

x = a, y = b;
for (m += n, n = 1; n <= m; n <= 1)
    ;
Rk(n);
x.resize(n), y.resize(n);
NTT(1, n, x), NTT(1, n, y);
for (int i = 0; i < n; i++)
    x[i] = x[i] * y[i] % Mod;
NTT(0, n, x);
ll inv = qpow(n);
X.resize(m + 1);
for (int i = 0; i <= m; i++)
    X[i] = x[i] * inv % Mod;
}

void Inv(int n, vector<ll> &a, vector<ll> &b)
{
    static vector<ll> x;
    if (n == 1)
    {
        b.resize(1);
        b[0] = qpow(a[0]);
        return;
    }
    Inv((n + 1) >> 1, a, b);
    const int m = n;
    for (n = 1; n < (m < 1); n <= 1)
        ;
    Rk(n);
    x = a;
    x.resize(n), b.resize(n);
    for (int i = m; i < n; i++)
        x[i] = 0;
    NTT(1, n, x), NTT(1, n, b);
    for (int i = 0; i < n; i++)
        b[i] = b[i] * (2ll - x[i] * b[i] % Mod + Mod) % Mod;
    NTT(0, n, b);
    ll inv = qpow(n);
    b.resize(m);
    for (int i = 0; i < m; i++)
        b[i] = b[i] * inv % Mod;
}

void Ln(vector<ll> &a, vector<ll> &b)
{
    static vector<ll> x;
    const int n = a.size();
    Inv(n, a, x);
    b.resize(n);
    for (int i = 0; i < n - 1; i++)
        b[i] = a[i + 1] * (i + 1) % Mod;
    b[n - 1] = 0;
    Mul(x, b, x);
    for (int i = 1; i < n; i++)
        b[i] = x[i - 1] * qpow(i) % Mod;
    b[0] = 0;
}

```

```

}

void Exp(int n, vector<ll> &a, vector<ll> &b)
{
    static vector<ll> x, y;
    if (n == 1)
    {
        b.resize(1);
        b[0] = 1;
        return;
    }
    Exp((n + 1) >> 1, a, b);
    int m = n;
    for (n = 1; n < (m << 1); n <= 1)
        ;
    y = a;
    y.resize(n);
    for (int i = m; i < n; i++)
        y[i] = 0;
    b.resize(m), Ln(b, x);
    b.resize(n), x.resize(n);
    NTT(1, n, x), NTT(1, n, y), NTT(1, n, b);
    for (int i = 0; i < n; i++)
        b[i] = (1ll - x[i] + y[i] + Mod) % Mod * b[i] % Mod;
    NTT(0, n, b);
    ll inv = qpow(n);
    b.resize(m);
    for (int i = 0; i < m; i++)
        b[i] = b[i] * inv % Mod;
}

void Sqrt(int n, vector<ll> &a, vector<ll> &b)
{
    static vector<ll> x, y;
    if (n == 1)
    {
        b.resize(1);
        b[0] = 1;
        return;
    }
    Sqrt((n + 1) >> 1, a, b);
    int m = n;
    for (n = 1; n < (m << 1); n <= 1)
        ;
    b.resize(m), Inv(m, b, x);
    y = a, y.resize(m);
    Mul(y, x, y);
    for (int i = 0; i < m; i++)
        b[i] = (b[i] + y[i]) % Mod * inv2 % Mod;
}

void Kpow(vector<ll> &a, const ll &k)
{
    static vector<ll> x;
    const int n = a.size();
    Ln(a, x);

```

```

    x.resize(n + 1);
    for (int i = 0; i <= n; i++)
        x[i] = x[i] * k % Mod;
    Exp(n + 1, x, a);
}

```

1.10. Catalan

```

#include <bits/stdc++.h>
#define ll long long
using namespace std;
const ll N=101010;
const ll p=998244353;

ll T;
ll n;
ll f[N],ni[N];

ll ksm(ll a,ll b) { ll sum = 1; while(b) { if(b&1) sum = (sum*a) % p; b >>= 1; a = (a*a) % p; } return sum; }

void qiu(ll h) {
    f[0] = ni[0] = 1;
    for(ll i=1;i<=h;i++) f[i] = f[i-1] * i %p;
    ni[h] = ksm(f[h],p-2);
    for(ll i=h-1;i;i--) ni[i] = ni[i+1] * (i+1) %p;
}

ll catalan(ll a) { return f[a<<1] * ni[a+1] %p * ni[a] %p; }
ll catalan(ll a) { return C(2*a,a) - C(2*a,a-1); }
ll catalan(ll a) { if(!a) return 1; ll res = 0; for(int i=0;i<=a-1;i++) res += catalan(i) * catalan(a-1-i); return res; }

int main() {
    qiu(N-10);
    cin>>T;
    while(T--) {
        cin>>n;
        cout<<catalan(n)<<endl;
    }
    return 0;
}

```

1.11. Min_25

```

#include <iostream>
#include <algorithm>
#include <cmath>

using namespace std;

typedef long long ll;
typedef double db;

const int N = 2e5 + 7;
const ll M = 1e10;
const int Mod = 1e9 + 7;

```

```

ll Lim, lim;
ll lis[N];
int mp[N][2], cnt = 0, lpf[N], pcnt = 0, pri[N];

ll G[N][2], Fprime[N];

ll qpow(ll x, ll y)
{
    ll ret = 1;
    while (y)
    {
        if (y & 1)
            ret = ret * x % Mod;
        x = x * x % Mod, y >>= 1;
    }
    return ret;
}

inline const int Id(const ll &x)
{
    return x <= lim ? mp[x][0] : mp[Lim / x][1];
}

inline const ll f(const ll &x)
{
    return x % Mod * ((x - 1ll) % Mod) % Mod;
}

const ll inv2 = qpow(2ll, Mod - 2);
const ll inv6 = qpow(6ll, Mod - 2);

ll F(const ll n, const int k)
{
    if (n < pri[k] || n <= 1)
        return 0;
    ll ans = (Fprime[Id(n)] - Fprime[Id(pri[k - 1])]) + Mod % Mod;
    for (int i = k; i <= pcnt && 1ll * pri[i] * pri[i] <= n; i++)
        for (ll pw = pri[i]; pw <= n / pri[i]; pw *= pri[i])
            ans = (ans + f(pw) * F(n / pw, i + 1) % Mod + f(pw * pri[i])) % Mod;
    return ans;
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    int T;
    cin >> T;
    while(T--)
    {
        cin >> Lim;
        pri[0] = 1;
        lim = sqrtl(Lim);
        cnt = pcnt = 0;
        for (int i = 2; i <= lim; i++)
            lpf[i] = 0;
    }
}

```

```

for (int i = 2; i <= lim; i++)
{
    if (!lpf[i])
    {
        lpf[i] = ++pcnt;
        pri[pcnt] = i;
    }
    for (int j = 1; j <= lpf[i] && 1ll * i * pri[j] <= lim; j++)
        lpf[i * pri[j]] = j;
}
for (ll l = 1, r = 0, v; l <= Lim; l = r + 1)
{
    r = Lim / (Lim / l);
    lis[++cnt] = v = Lim / l;
    (v <= lim ? mp[v][0] : mp[Lim / v][1]) = cnt;
    G[cnt][0] = (v + 2ll) % Mod * ((v - 1ll) % Mod) % Mod * inv2 % Mod;
    G[cnt][1] = v % Mod * ((v + 1) % Mod) % Mod * ((2ll * v + 1ll) % Mod) %
Mod * inv6 % Mod;
    G[cnt][1] = (G[cnt][1] + Mod - 1ll) % Mod;
}
ll v;
for (int k = 1; k <= pcnt; k++)
{
    const ll pw = (ll)pri[k] * pri[k];
    for (int i = 1; pw <= lis[i]; i++)
    {
        const int id1 = Id(lis[i] / pri[k]);
        const int id2 = Id(pri[k - 1]);
        G[i][0] = (G[i][0] - pri[k] * ((G[id1][0] - G[id2][0] + Mod) % Mod) %
Mod + Mod) % Mod;
        G[i][1] = (G[i][1] - 1ll * pri[k] * pri[k] % Mod * ((G[id1][1] -
G[id2][1] + Mod) % Mod) % Mod + Mod) % Mod;
    }
}
for (int i = 1; i <= cnt; i++)
    Fprime[i] = (G[i][1] - G[i][0] + Mod) % Mod;
cout << (F(Lim, 1) + 1ll) % Mod << "\n";
}
}

```

1.12. 求原根

```

const ll N=501010;
const ll qwq=3030303;
const ll inf=0x3f3f3f3f;

ll T;
ll n,m;
ll visp[qwq],pi[N],cntp;
ll st[N],cntl;

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %n;
        bb >>= 1; aa = aa * aa %n;
    }
}

```

```

    }
    return sum;
}

void shai(ll h) {
    for(ll i=2;i<=h;i++) {
        if(!visp[i]) pi[++cntp] = i;
        for(ll j=1;j<=cntp && i*pi[j]<=h;j++) {
            visp[ i*pi[j] ] = 1;
            if(i%pi[j]==0) break;
        }
    }
}

void chushihua() {
    cnt1 = 0;
}

int get_root(int P) {
    ll now = P-1;
    for(ll i=1;i<=cntp;i++) {
        if(now%pi[i]==0) st[++cnt1] = pi[i];
        while(now%pi[i]==0) now /= pi[i];
    }
    if(now!=1) st[++cnt1] = now;
    // for(ll i=1;i<=cnt1;i++) cout<<st[i]<<" ";
    for(ll i=2;;i++) {
        bool you = 0;
        for(ll j=1;j<=cnt1;j++) if(ksm(i,(P-1)/st[j])==1) { you = 1; break; }
        if(!you) return i;
    }
}

int main() {
    shai(qwq-100);
    while(1) {
        chushihua();
        cin>>n;
        cout<<get_root(n)<<"\n";
    }
    return 0;
}

// n has r when n=1,2,4,p^a,2p^a
// 7 -> 3      1,3,2,6,4,5

```

1.13. exCRT

```

ll T;
ll n,m;
ll a[N],b[N];
ll x,y;

ll gsc(ll a,ll b,ll p) {
    ll sum = 0;
    while(b) {

```



```

inline ll BSGS(ll a,ll b,ll p) { //  $a^{\text{res}} = b \pmod p$ 
    f.clear();
    ll now = b % p;
    ll sq = sqrt(p) + 1;
    ll at = ksm(a,sq,p);
    f[now] = 0;
    for(ll i=1;i<=sq;i++) {
        now = now * a %p;
        f[now] = i;
    }
    now = 1;
    for(ll i=1;i<=sq;i++) {
        now = now * at %p;
        if(f[now]) return (i * sq %p - f[now] + p) %p;
    }
    return -1;
}

```

2. 其他

2.1. 区间加等差数列

```

void add(int l,int r,db a0,db d) {
    if(l>r) return ;
    db mo = a0 + (r-l) * d;
    cha2[l] += a0;
    cha2[l+1] -= a0-d;
    cha2[r+1] -= mo+d;
    cha2[r+2] += mo;
}

```

2.2. make

```

#include <bits/stdc++.h>
#define ll long long

using namespace std;
const ll N=201010;

ll rand(ll L,ll R) {
    return (rand() * rand() + rand()) % (R-L+1) + L;
}

ll T = 10, n ;

int main() {
    freopen("data.in","w",stdout);
    srand(time(NULL)^getpid());
    mt19937_64 rng(random_device{}());
    // val[i] = rng();
    cout<<T<<"\n";
    while(T--) {

    }
    return 0;
}

```

2.3. 子集 FWT

```
#include <bits/stdc++.h>
#define ll long long

using namespace std;
const ll N=501010;
const ll inf=0x3f3f3f3f;
const ll p=998244353, inv2=499122177;

inline ll read() {
    ll sum = 0, ff = 1; char c = getchar();
    while(c<'0' || c>'9') { if(c=='-') ff = -1; c = getchar(); }
    while(c>='0'&&c<='9') { sum = (sum * 10 + c - '0') %p; c = getchar(); }
    return sum * ff;
}

ll K, da;
ll a[N],b[N],c[N];
ll A[N],B[N];

inline void OR(ll *F,ll cl) {
    for(ll o=2;o<=da;o<=1) {
        for(ll i=0,k=o>>1;i<da;i+=o) {
            for(ll j=0;j<k;j++) {
                F[i+j+k] = (F[i+j+k] + F[i+j]*cl + p) %p;
            }
        }
    }
}

inline void AND(ll *F,ll cl) {
    for(ll o=2;o<=da;o<=1) {
        for(ll i=0,k=o>>1;i<da;i+=o) {
            for(ll j=0;j<k;j++) {
                F[i+j] = (F[i+j] + F[i+j+k]*cl + p) %p;
            }
        }
    }
}

inline void XOR(ll *F,ll cl) {
    for(ll o=2;o<=da;o<=1) {
        for(ll i=0,k=o>>1;i<da;i+=o) {
            for(ll j=0;j<k;j++) {
                ll X = F[i+j], Y = F[i+j+k];
                F[i+j] = (X+Y) * (cl==1?1:inv2) %p;
                F[i+j+k] = (X-Y+p) * (cl==1?1:inv2) %p;
            }
        }
    }
}

void juan(ll *H,ll *F,ll *G) {
    for(int i=0;i<da;i++) A[i] = F[i], B[i] = G[i];
    OR(A, 1);
    OR(B, 1);
}
```

```

    for(int i=0;i<da;i++) A[i] = A[i] * B[i] %p;
    OR(A, -1);
    for(int i=0;i<da;i++) H[i] = A[i];
}

```

```

int main() {
    K = read(); da = 1<<K;
    for(ll i=0;i<da;i++) a[i] = read();
    for(ll i=0;i<da;i++) b[i] = read();
    juan(c, a, b);
    for(ll i=0;i<da;i++) cout<<c[i]<<" ";
    return 0;
}

```

2.4. 取模优化

```

double dp;

inline ll mod(ll A) {
    return A - (ll)(A / dp) * p;
}

```

2.5. 二维 hash

```

struct Hash {
    ll p=998244353, m1=2333, m2=13331;
    ll f1[N], f2[N];
    ll h[N][N], a[N][N];

    void init() {
        f1[0] = f2[0] = 1;
        for(int i=1;i<=n;i++) f1[i] = f1[i-1] * m1 %p;
        for(int i=1;i<=m;i++) f2[i] = f2[i-1] * m2 %p;
        for(int i=1;i<=n;i++) {
            for(int j=1;j<=m;j++) {
                h[i][j] = (h[i-1][j]*m1%p + h[i][j-1]*m2%p - h[i-1][j-1]*m1%p*m2%p +
a[i][j] + p) %p;
            }
        }
    }

    ll calc(ll i,ll j,ll I,ll J) {
        i--; j--;
        ll X = f1[I-i], Y = f2[J-j];
        return (h[I][J] - h[i][J]*X%p - h[I][j]*Y%p + h[i][j]*X%p*Y%p + 2*p) %p;
    }
};

```

2.6. 树上莫队

```

ll f[N][24];
ll dep[N],siz[N];
ll st[N],ed[N],ob[N],cnt;

struct Q{
    ll id,l,r,lca;
} q[N];

inline bool cmp(Q A,Q B) {

```

```

    if(A.l/K != B.l/K) return A.l < B.l;
    if((A.l/K)&1) return A.r < B.r;
    else return A.r > B.r;
}

void DFS(ll u) {
    dep[u] = dep[f[u][0]] + 1; siz[u] = 1;
    st[u] = ++cnt; ob[cnt] = u;
    for(ll v : e[u]) {
        DFS(v);
        siz[u] += siz[v];
    }
    ed[u] = ++cnt; ob[cnt] = u;
}

inline void add(ll u) {
    // cout<<"add(" <<u<<")\n";
    ll c = num[u];
    num[u] ^= 1;
    if(c) (res -= dp[u]-p) %= p;
    else (res += dp[u]) %= p;
}

int main() {
    n = read(); m = read();
    for(ll i=1;i<=m;i++) {
        x = read(); y = read();
        if(st[x]>st[y]) swap(x,y);
        ll lca = LCA(x,y);
        if(lca==x) q[i] = {i,st[x],st[y],0};
        else q[i] = {i,ed[x],st[y],lca};
    }
    sort(q+1,q+m+1,cmp);

    ll L = q[1].l, R = q[1].l;
    res = dp[ ob[L] ];
    num[ ob[L] ] = 1;
    for(ll i=1;i<=m;i++) {
        // cout<<q[i].l<<" "<<q[i].r<<" i = "<<i<<"\n";

        while(L>q[i].l) add(ob[--L]);
        while(R<q[i].r) add(ob[++R]);
        while(L<q[i].l) add(ob[L++]);
        while(R>q[i].r) add(ob[R--]);
        // cout<<"res = "<<res<<"\n";
        if(q[i].lca) add(q[i].lca);
        ans[ q[i].id ] = res;
        if(q[i].lca) add(q[i].lca);
    }
    return 0;
}

```

2.7. 子集卷积

```

#include <bits/stdc++.h>
#define ll long long

```

```

using namespace std;
const ll N=2020202;
const ll inf=0x3f3f3f3f;
const ll p=1000000009;

ll K, da;
ll a[N],b[N],c[N];
ll A[22][N], B[22][N], C[22][N], wei[N];

inline void OR(ll *F,ll cl) {
    for(ll o=2;o<=da;o<=1) {
        for(ll i=0,k=o>1;i<da;i+=o) {
            for(ll j=0;j<k;j++) {
                F[i+j+k] = (F[i+j+k] + F[i+j]*cl + p) %p;
            }
        }
    }
}

void juan(ll *H,ll *F,ll *G) {
    for(ll i=0;i<da;i++) A[wei[i]][i] = F[i];
    for(ll i=0;i<da;i++) B[wei[i]][i] = G[i];
    for(ll i=0;i<=K;i++) {
        OR(A[i], 1);
        OR(B[i], 1);
    }
    for(ll i=0;i<=K;i++) {
        for(ll j=0;j<=i;j++) {
            for(ll k=0;k<da;k++) {
                (C[i][k] += A[j][k]*B[i-j][k] %p) %= p;
            }
        }
    }
    for(ll i=0;i<=K;i++) OR(C[i], -1);
    for(ll i=0;i<da;i++) H[i] = C[wei[i]][i];
}

int main() {
    for(ll i=1;i<=N-10;i++) wei[i] = wei[i^(i&-i)] + 1;
    K = read(); da = 1<<K;
    for(ll i=0;i<da;i++) a[i] = read();
    for(ll i=0;i<da;i++) b[i] = read();
    juan(c, a, b);
    for(ll i=0;i<da;i++) cout<<c[i]<<" ";
    return 0;
}

```

2.8. 前缀线性基 PLB

```

using namespace std;
const ll N=505050;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;

int n,q;

struct PLB{

```

```

int d[22], pos[22];

void insert(int x,int wei) {
    for(int i=21;i>=0;i--) {
        if(!((x>>i)&1)) continue;
        if(!d[i]) {
            d[i] = x;
            pos[i] = wei;
            return ;
        }
        if(pos[i]<wei) {
            swap(pos[i],wei);
            swap(d[i],x);
        }
        x ^= d[i];
    }
}

int query_max(int l) {
    int ans = 0;
    for(int i=21;i>=0;i--)
        if(d[i] && pos[i]>=l)
            ans = max(ans,ans^d[i]);
    return ans;
}

bool query(int k,int l) {
    for(int i=21;i>=0;i--) {
        if(!(k>>i)) continue;
        if(!d[i] || pos[i]<l) return 0;
        k ^= d[i];
    }
    return 1;
}

}B[N];

int main() {
    int l,r;
    n = read();
    for(int i=1;i<=n;i++) {
        B[i] = B[i-1];
        B[i].insert(read(),i);
    }
    Q = read();
    while(Q--) {
        l = read(); r = read();
        cout<<B[r].query(l)<<"\n";
    }
    return 0;
}

```

2.9. 售货员

```
#define ls now<<1
#define rs now<<1|1
using namespace std;
const int N=1044*1044;
const int qwq=2003030;
const int inf=0x3f3f3f3f;

int n,tot;
int w[23][23];
int f[N][21];
int ans=inf;

int main() {
    n = read(); tot = (1<<n)-1;
    for(int i=1;i<=n;i++) for(int j=1;j<=n;j++) w[i][j] = read();

    memset(f,0x3f,sizeof(f));
    f[1][1] = 0;
    for(int i=3;i<=tot;i++) {
        for(int j=1;j<=n;j++) {
            if((i>>j-1)&1)
                for(int k=1;k<=n;k++)
                    if((i>>k-1)&1)
                        f[i][j] = min(f[i][j], f[i^(1<<j-1)][k]+w[j][k]);
        }
    }
    // cout<<f[tot][n]; 终点 n 结束
    // cout<<MIN{ f[tot][i] }; 任意点结束
    // cout<<MIN{ w[1][i]+f[tot][i] }; 返回 1 结束
    return 0;
}

/*

*/

*/
```

2.10. 线性基

```
ll n;
ll a[N];
ll d[N];
ll ans;

int main() {
    n = read();
    for(ll i=1;i<=n;i++) a[i] = read();
    for(ll i=1;i<=n;i++) {
        ll x = a[i];
        for(ll j=60;j>=0;j--) {
            if(!(x>>j)) continue;
            if(!d[j]) {
                d[j] = x;
                break;
            }
        }
    }
}
```

```

        }
        x ^= d[j];
    }
}
for(ll j=60;j>=0;j--) {
    if((ans^d[j])>ans) ans ^= d[j];
}
cout<<ans;

return 0;
}

```

2.11. 子集 dp

```
for(int i=u; i; i=(i-1)&u) f[u] += f[i];    // 枚举子集
```

// 高维前缀和:

```
for(int k=0;k<=19;k++)
    for(int i=0;i<=maxn;i++)
        if((i>>k)&1) g[i] += g[i^(1<<k)];    // i 是超集, 有第 k 位
```

// 高维后缀和:

```
for(int k=0;k<=19;k++)
    for(int i=0;i<=maxn;i++)
        if(!((i>>k)&1)) g[i] += g[i^(1<<k)];    // i 是子集, 无第 k 位
```

// 前缀差分:

```
for(int k=0;k<=19;k++)
    for(int i=0;i<=maxn;i++)
        if((i>>k)&1) f[i] -= f[i^(1<<k)];    // i 的前缀和减去 i 的子集, 就是 i 本身。
```

// 后缀差分:

```
for(int k=0;k<=19;k++)
    for(int i=0;i<=maxn;i++)
        if(!((i>>k)&1)) f[i] -= f[i^(1<<k)];    // i 的后缀和减去 i 的超集, 就是 i 本身。
```

2.12. ksc

```
IL int qmul(int x,int y,int mod)
{
    return (x*y-(long long)((long double)x/mod*y)*mod+mod)%mod;
}

```

2.13. duipai

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int Case = 1;
    while(1) {
        printf("The result of No. %d Case is: ", ++Case);
        system("make");    //windows 不加"./", linux 下需要在可执行文件的文件名前加"./"
        system("A");
        system("A2");
        if(system("fc A.out A2.out")) { //linux 环境用    system("diff *** ***");
            printf("qwq");
            return 0;
        }
    }
}

```

```

    }
    else printf("No different\n");
}
return 0;
}

```

2.14. 异或分段

```

ll T;
ll m,P;
struct D{
    ll l,r;
}d[N];
inline bool cmp(D A,D B) { return A.l < B.l; }
ll cnt;

void get_fen(ll R,ll X) { // [0, R] ^ X ---> [ d[i].l, d[i].r ]
    ll da = 1ll << 60ll;
    d[cnt=1] = {0,R};
    for(ll i=60;i>=0;i--,da>>=1) {
        if(d[1].l+da<=d[1].r) {
            if((X>>i)&1) {
                d[++cnt] = {d[1].l+da,d[1].l+2*da-1};
                d[1].r -= da;
            }
            else {
                d[++cnt] = {d[1].l,d[1].l+da-1};
                d[1].l += da;
            }
        }
        else if((X>>i)&1) d[1].l += da, d[1].r += da;
    }
    sort(d+1,d+cnt+1,cmp);
    // for(ll i=1;i<=cnt;i++) cout<<d[i].l<<" "<<d[i].r<<"\n";
}

```

2.15. 火车头

```

#pragma GCC optimize("Ofast")
#pragma GCC optimize("inline")
#pragma GCC optimize("-fgcse")
#pragma GCC optimize("-fgcse-lm")
#pragma GCC optimize("-fipa-sra")
#pragma GCC optimize("-ftree-pre")
#pragma GCC optimize("-ftree-vrp")
#pragma GCC optimize("-fpeephole2")
#pragma GCC optimize("-ffast-math")
#pragma GCC optimize("-fsched-spec")
#pragma GCC optimize("unroll-loops")
#pragma GCC optimize("-falign-jumps")
#pragma GCC optimize("-falign-loops")
#pragma GCC optimize("-falign-labels")
#pragma GCC optimize("-fdevirtualize")
#pragma GCC optimize("-fcaller-saves")
#pragma GCC optimize("-fcrossjumping")
#pragma GCC optimize("-fthread-jumps")
#pragma GCC optimize("-funroll-loops")
#pragma GCC optimize("-fwhole-program")

```

```

#pragma GCC optimize("-freorder-blocks")
#pragma GCC optimize("-fschedule-insns")
#pragma GCC optimize("inline-functions")
#pragma GCC optimize("-ftree-tail-merge")
#pragma GCC optimize("-fschedule-insns2")
#pragma GCC optimize("-fstrict-aliasing")
#pragma GCC optimize("-fstrict-overflow")
#pragma GCC optimize("-falign-functions")
#pragma GCC optimize("-fcse-skip-blocks")
#pragma GCC optimize("-fcse-follow-jumps")
#pragma GCC optimize("-fsched-interblock")
#pragma GCC optimize("-fpartial-inlining")
#pragma GCC optimize("no-stack-protector")
#pragma GCC optimize("-freorder-functions")
#pragma GCC optimize("-findirect-inlining")
#pragma GCC optimize("-frerun-cse-after-loop")
#pragma GCC optimize("inline-small-functions")
#pragma GCC optimize("-finline-small-functions")
#pragma GCC optimize("-ftree-switch-conversion")
#pragma GCC optimize("-foptimize-sibling-calls")
#pragma GCC optimize("-fexpensive-optimizations")
#pragma GCC optimize("-funsafe-loop-optimizations")
#pragma GCC optimize("inline-functions-called-once")
#pragma GCC optimize("-fdelete-null-pointer-checks")

```

3. 几何

3.1. cjj

```

#include <bits/stdc++.h>

using namespace std;

using point_t = long double; //全局数据类型,可修改为 long long 等

constexpr point_t eps = 1e-8;
constexpr long double PI = 3.14159265358979323841;

// 点与向量
template<typename T>
struct point {
    T x, y;

    point() = default;

    point(T x, T y) : x(x), y(y) {}

    bool operator==(const point &a) const { return (abs(x - a.x) <= eps && abs(y - a.y) <= eps); }

    bool operator<(const point &a) const {
        if (abs(x - a.x) <= eps) return y < a.y - eps;
        return x < a.x - eps;
    }

    bool operator>(const point &a) const { return !(*this < a || *this == a); }
}

```

```

point operator+(const point &a) const { return {x + a.x, y + a.y}; }

point operator-(const point &a) const { return {x - a.x, y - a.y}; }

point operator-() const { return {-x, -y}; }

point operator*(const T k) const { return {k * x, k * y}; }

point operator/(const T k) const { return {x / k, y / k}; }

T operator*(const point &a) const { return x * a.x + y * a.y; } // 点积
T operator^(const point &a) const { return x * a.y - y * a.x; } // 叉积, 注意优先级
int toleft(const point &a) const {
    const auto t = (*this) ^ a;
    return (t > eps) - (t < -eps);
} // to-left 测试
T len2() const { return (*this) * (*this); } // 向量长度的平方
T dis2(const point &a) const { return (a - (*this)).len2(); } // 两点距离的平方

// 涉及浮点数
long double len() const { return hypotl(x, y); } // 向量长度
long double dis(const point &a) const { return ((*this) - a).len(); } // 两点距离
long double ang(const point &a) const {
    return acosl(max(-1.0l, min(1.0l, ((*this) * a) / (len() * a.len()))));
} // 向量夹角
point rot(const long double rad) const {
    return {x * cos(rad) - y * sin(rad), x * sin(rad) + y * cos(rad)};
} // 逆时针旋转 (给定角度)
point rot(const long double cosr, const long double sinr) const {
    return {x * cosr - y * sinr, x * sinr + y * cosr};
} // 逆时针旋转 (给定角度的正弦与余弦)
};

using Point = point<point_t>;

// 极角排序
struct argcmp {
    bool operator()(const Point &a, const Point &b) const {
        const auto quad = [](const Point &a) {
            if (a.y < -eps) return 1;
            if (a.y > eps) return 4;
            if (a.x < -eps) return 5;
            if (a.x > eps) return 3;
            return 2;
        };
        const int qa = quad(a), qb = quad(b);
        if (qa != qb) return qa < qb;
        const auto t = a ^ b;
        if (abs(t) <= eps) return a.len2() < b.len2() - eps; // 不同长度的向量需要分开
        return t > eps;
    }
};

// 直线
template<typename T>
struct line {

```

```

point<T> p, v; // p 为直线上一点, v 为方向向量

bool operator==(const line &a) const { return v.toleft(a.v) == 0 && v.toleft(p -
a.p) == 0; }

int toleft(const point<T> &a) const { return v.toleft(a - p); } // to-left 测试
bool operator<(const line &a) const // 半平面交算法定义的排序
{
    if (abs(v ^ a.v) <= eps && v * a.v >= -eps) return toleft(a.p) == -1;
    return argcmp()(v, a.v);
}

// 涉及浮点数
point<T> inter(const line &a) const { return p + v * ((a.v ^ (p - a.p)) / (v ^
a.v)); } // 直线交点
long double dis(const point<T> &a) const { return abs(v ^ (a - p)) /
v.len(); } // 点到直线距离
point<T> proj(const point<T> &a) const { return p + v * ((v * (a - p)) / (v *
v)); } // 点在直线上的投影
};

using Line = line<point_t>;

// 线段
template<typename T>
struct segment {
    point<T> a, b;

    bool operator<(const segment &s) const { return make_pair(a, b) < make_pair(s.a,
s.b); }

    // 判定性函数建议在整数域使用

    // 判断点是否在线段上
    // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
    int is_on(const point<T> &p) const {
        if (p == a || p == b) return -1;
        return (p - a).toleft(p - b) == 0 && (p - a) * (p - b) < -eps;
    }

    // 判断线段直线是否相交
    // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
    int is_inter(const line<T> &l) const {
        if (l.toleft(a) == 0 || l.toleft(b) == 0) return -1;
        return l.toleft(a) != l.toleft(b);
    }

    // 判断两线段是否相交
    // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
    int is_inter(const segment<T> &s) const {
        if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
        const line<T> l{a, b - a}, ls{s.a, s.b - s.a};
        return l.toleft(s.a) * l.toleft(s.b) == -1 && ls.toleft(a) * ls.toleft(b) ==
-1;
    }
}

```

```

// 点到线段距离
long double dis(const point<T> &p) const {
    if ((p - a) * (b - a) < -eps || (p - b) * (a - b) < -eps) return
min(p.dis(a), p.dis(b));
    const line<T> l{a, b - a};
    return l.dis(p);
}

// 两线段间距离
long double dis(const segment<T> &s) const {
    if (is_inter(s)) return 0;
    return min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
}
};

using Segment = segment<point_t>;

// 多边形
template<typename T>
struct polygon {
    vector<point<T>> p; // 以逆时针顺序存储

    size_t nxt(const size_t i) const { return i == p.size() - 1 ? 0 : i + 1; }

    size_t pre(const size_t i) const { return i == 0 ? p.size() - 1 : i - 1; }

    // 回转数
    // 返回值第一项表示点是否在多边形边上
    // 对于狭义多边形, 回转数为 0 表示点在多边形外, 否则点在多边形内
    pair<bool, int> winding(const point<T> &a) const {
        int cnt = 0;
        for (size_t i = 0; i < p.size(); i++) {
            const point<T> u = p[i], v = p[nxt(i)];
            if (abs((a - u) ^ (a - v)) <= eps && (a - u) * (a - v) <= eps) return
{true, 0};
            if (abs(u.y - v.y) <= eps) continue;
            const Line uv = {u, v - u};
            if (u.y < v.y - eps && uv.toleft(a) <= 0) continue;
            if (u.y > v.y + eps && uv.toleft(a) >= 0) continue;
            if (u.y < a.y - eps && v.y >= a.y - eps) cnt++;
            if (u.y >= a.y - eps && v.y < a.y - eps) cnt--;
        }
        return {false, cnt};
    }

    // 多边形面积的两倍
    // 可用于判断点的存储顺序是顺时针或逆时针
    T area() const {
        T sum = 0;
        for (size_t i = 0; i < p.size(); i++) sum += p[i] ^ p[nxt(i)];
        return sum;
    }

    // 多边形的周长
    long double circ() const {
        long double sum = 0;

```

```

        for (size_t i = 0; i < p.size(); i++) sum += p[i].dis(p[nxt(i)]);
        return sum;
    }
};

using Polygon = polygon<point_t>;

//凸多边形
template<typename T>
struct convex : polygon<T> {
    // 闵可夫斯基和
    convex operator+(const convex &c) const {
        const auto &p = this->p;
        vector<Segment> e1(p.size()), e2(c.p.size()), edge(p.size() + c.p.size());
        vector<point<T>> res;
        res.reserve(p.size() + c.p.size());
        const auto cmp = [](const Segment &u, const Segment &v) { return argcmp()(u.b
- u.a, v.b - v.a); };
        for (size_t i = 0; i < p.size(); i++) e1[i] = {p[i], p[this->nxt(i)]};
        for (size_t i = 0; i < c.p.size(); i++) e2[i] = {c.p[i], c.p[c.nxt(i)]};
        rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
        rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
        merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
        const auto check = [](const vector<point<T>> &res, const point<T> &u) {
            const auto back1 = res.back(), back2 = *prev(res.end(), 2);
            return (back1 - back2).toleft(u - back1) == 0 && (back1 - back2) * (u -
back1) >= -eps;
        };
        auto u = e1[0].a + e2[0].a;
        for (const auto &v: edge) {
            while (res.size() > 1 && check(res, u)) res.pop_back();
            res.push_back(u);
            u = u + v.b - v.a;
        }
        if (res.size() > 1 && check(res, res[0])) res.pop_back();
        return {res};
    }

    // 旋转卡壳
    // func 为更新答案的函数，可以根据题目调整位置
    template<typename F>
    void rotcaliper(const F &func) const {
        const auto &p = this->p;
        const auto area = [](const point<T> &u, const point<T> &v, const point<T> &w)
{ return (w - u) ^ (w - v); };
        for (size_t i = 0, j = 1; i < p.size(); i++) {
            const auto nexti = this->nxt(i);
            func(p[i], p[nexti], p[j]);
            while (area(p[this->nxt(j)], p[i], p[nexti]) >= area(p[j], p[i], p[nexti]))
{
                j = this->nxt(j);
                func(p[i], p[nexti], p[j]);
            }
        }
    }
};

```

```

// 凸多边形的直径的平方
T diameter2() const {
    const auto &p = this->p;
    if (p.size() == 1) return 0;
    if (p.size() == 2) return p[0].dis2(p[1]);
    T ans = 0;
    auto func = [&](const point<T> &u, const point<T> &v, const point<T> &w) {
        ans = max({ans, w.dis2(u), w.dis2(v)});
    };
    rotcaliper(func);
    return ans;
}

// 面积前缀和
vector<T> sum;

void get_sum() {
    const auto &p = this->p;
    vector<T> a(p.size());
    for (size_t i = 0; i < p.size(); i++) a[i] = p[this->pre(i)] ^ p[i];
    sum.resize(p.size());
    partial_sum(a.begin(), a.end(), sum.begin());
}

T query_sum(const size_t l, const size_t r) const {
    const auto &p = this->p;
    if (l <= r) return sum[r] - sum[l] + (p[r] ^ p[l]);
    return sum[p.size() - 1] - sum[l] + sum[r] + (p[r] ^ p[l]);
}

T query_sum() const { return sum.back(); }

// 判断点是否在凸多边形内
// 复杂度 O(logn)
// -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
int is_in(const point<T> &a) const {
    const auto &p = this->p;
    if (p.size() == 1) return a == p[0] ? -1 : 0;
    if (p.size() == 2) return segment<T>{p[0], p[1]}.is_on(a) ? -1 : 0;
    if (a == p[0]) return -1;
    if ((p[1] - p[0]).toleft(a - p[0]) == -1 || (p.back() - p[0]).toleft(a -
p[0]) == 1) return 0;
    const auto cmp = [&](const Point &u, const Point &v) { return (u -
p[0]).toleft(v - p[0]) == 1; };
    const size_t i = lower_bound(p.begin() + 1, p.end(), a, cmp) - p.begin();
    if (i == 1) return segment<T>{p[0], p[i]}.is_on(a) ? -1 : 0;
    if (i == p.size() - 1 && segment<T>{p[0], p[i]}.is_on(a)) return -1;
    if (segment<T>{p[i - 1], p[i]}.is_on(a)) return -1;
    return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
}

// 凸多边形关于某一方向的极点
// 复杂度 O(logn)
// 参考资料: https://codeforces.com/blog/entry/48868
template<typename F>
size_t extreme(const F &dir) const {

```

```

        const auto &p = this->p;
        const auto check = [&](const size_t i) { return dir(p[i]).toleft(p[this-
>next(i)] - p[i]) >= 0; };
        const auto dir0 = dir(p[0]);
        const auto check0 = check(0);
        if (!check0 && check(p.size() - 1)) return 0;
        const auto cmp = [&](const Point &v) {
            const size_t vi = &v - p.data();
            if (vi == 0) return 1;
            const auto checkv = check(vi);
            const auto t = dir0.toleft(v - p[0]);
            if (vi == 1 && checkv == check0 && t == 0) return 1;
            return checkv ^ (checkv == check0 && t <= 0);
        };
        return partition_point(p.begin(), p.end(), cmp) - p.begin();
    }

    // 过凸多边形外一点求凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    // 必须保证点在多边形外
    pair<size_t, size_t> tangent(const point<T> &a) const {
        const size_t i = extreme([&](const point<T> &u) { return u - a; });
        const size_t j = extreme([&](const point<T> &u) { return a - u; });
        return {i, j};
    }

    // 求平行于给定直线的凸多边形的切线，返回切点下标
    // 复杂度 O(logn)
    pair<size_t, size_t> tangent(const line<T> &a) const {
        const size_t i = extreme([&](...) { return a.v; });
        const size_t j = extreme([&](...) { return -a.v; });
        return {i, j};
    }
};

using Convex = convex<point_t>;

// 点集的凸包
// Andrew 算法，复杂度 O(nlogn)
Convex convexhull(vector<Point> p) {
    vector<Point> st;
    if (p.empty()) return Convex{st};
    sort(p.begin(), p.end());
    const auto check = [&](const vector<Point> &st, const Point &u) {
        const auto back1 = st.back(), back2 = *prev(st.end(), 2);
        return (back1 - back2).toleft(u - back1) <= 0;
    };
    for (const Point &u: p) {
        while (st.size() > 1 && check(st, u)) st.pop_back();
        st.push_back(u);
    }
    size_t k = st.size();
    p.pop_back();
    reverse(p.begin(), p.end());
    for (const Point &u: p) {
        while (st.size() > k && check(st, u)) st.pop_back();

```

```

        st.push_back(u);
    }
    st.pop_back();
    return Convex{st};
}

// 圆
struct Circle {
    Point c;
    long double r;

    bool operator==(const Circle &a) const { return c == a.c && abs(r - a.r) <=
eps; }

    long double circ() const { return 2 * PI * r; } // 周长
    long double area() const { return PI * r * r; } // 面积

    // 点与圆的关系
    // -1 圆上 | 0 圆外 | 1 圆内
    int is_in(const Point &p) const {
        const long double d = p.dis(c);
        return abs(d - r) <= eps ? -1 : d < r - eps;
    }

    // 直线与圆关系
    // 0 相离 | 1 相切 | 2 相交
    int relation(const Line &l) const {
        const long double d = l.dis(c);
        if (d > r + eps) return 0;
        if (abs(d - r) <= eps) return 1;
        return 2;
    }

    // 圆与圆关系
    // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
    int relation(const Circle &a) const {
        if (*this == a) return -1;
        const long double d = c.dis(a.c);
        if (d > r + a.r + eps) return 0;
        if (abs(d - r - a.r) <= eps) return 1;
        if (abs(d - abs(r - a.r)) <= eps) return 3;
        if (d < abs(r - a.r) - eps) return 4;
        return 2;
    }

    // 直线与圆的交点
    vector<Point> inter(const Line &l) const {
        const long double d = l.dis(c);
        const Point p = l.proj(c);
        const int t = relation(l);
        if (t == 0) return vector<Point>();
        if (t == 1) return vector<Point>{p};
        const long double k = sqrt(r * r - d * d);
        return vector<Point>{p - (l.v / l.v.len()) * k, p + (l.v / l.v.len()) * k};
    }
}

```

```

// 圆与圆交点
vector<Point> inter(const Circle &a) const {
    const long double d = c.dis(a.c);
    const int t = relation(a);
    if (t == -1 || t == 0 || t == 4) return vector<Point>();
    Point e = a.c - c;
    e = e / e.len() * r;
    if (t == 1 || t == 3) {
        if (r * r + d * d - a.r * a.r >= -eps) return vector<Point>{c + e};
        return vector<Point>{c - e};
    }
    const long double costh = (r * r + d * d - a.r * a.r) / (2 * r * d), sinth =
sqrt(1 - costh * costh);
    return vector<Point>{c + e.rot(costh, -sinth), c + e.rot(costh, sinth)};
}

// 圆与圆交面积
long double inter_area(const Circle &a) const {
    const long double d = c.dis(a.c);
    const int t = relation(a);
    if (t == -1) return area();
    if (t < 2) return 0;
    if (t > 2) return min(area(), a.area());
    const long double costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d), costh2
=
        (a.r * a.r + d * d - r * r) / (2 * a.r * d);
    const long double sinth1 = sqrt(1 - costh1 * costh1), sinth2 = sqrt(1 -
costh2 * costh2);
    const long double th1 = acos(costh1), th2 = acos(costh2);
    return r * r * (th1 - costh1 * sinth1) + a.r * a.r * (th2 - costh2 * sinth2);
}

// 过圆外一点圆的切线
vector<Line> tangent(const Point &a) const {
    const int t = is_in(a);
    if (t == 1) return vector<Line>();
    if (t == -1) {
        const Point v = {-(a - c).y, (a - c).x};
        return vector<Line>{{a, v}};
    }
    Point e = a - c;
    e = e / e.len() * r;
    const long double costh = r / c.dis(a), sinth = sqrt(1 - costh * costh);
    const Point t1 = c + e.rot(costh, -sinth), t2 = c + e.rot(costh, sinth);
    return vector<Line>{{a, t1 - a},
                        {a, t2 - a}};
}

// 两圆的公切线
vector<Line> tangent(const Circle &a) const {
    const int t = relation(a);
    vector<Line> lines;
    if (t == -1 || t == 4) return lines;
    if (t == 1 || t == 3) {
        const Point p = inter(a)[0], v = {-(a.c - c).y, (a.c - c).x};
        lines.push_back({p, v});
    }
}

```

```

    }
    const long double d = c.dis(a.c);
    const Point e = (a.c - c) / (a.c - c).len();
    if (t <= 2) {
        const long double costh = (r - a.r) / d, sinth = sqrt(1 - costh * costh);
        const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
        const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c + d1 * a.r, v2 =
a.c + d2 * a.r;
        lines.push_back({u1, v1 - u1});
        lines.push_back({u2, v2 - u2});
    }
    if (t == 0) {
        const long double costh = (r + a.r) / d, sinth = sqrt(1 - costh * costh);
        const Point d1 = e.rot(costh, -sinth), d2 = e.rot(costh, sinth);
        const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c - d1 * a.r, v2 =
a.c - d2 * a.r;
        lines.push_back({u1, v1 - u1});
        lines.push_back({u2, v2 - u2});
    }
    return lines;
}

// 圆的反演
tuple<int, Circle, Line> inverse(const Line &l) const {
    const Circle null_c = {{0.0, 0.0}, 0.0};
    const Line null_l = {{0.0, 0.0},
                        {0.0, 0.0}};
    if (l.toleft(c) == 0) return {2, null_c, l};
    const Point v = l.toleft(c) == 1 ? Point{l.v.y, -l.v.x} : Point{-l.v.y,
l.v.x};
    const long double d = r * r / l.dis(c);
    const Point p = c + v / v.len() * d;
    return {1, {(c + p) / 2, d / 2}, null_l};
}

tuple<int, Circle, Line> inverse(const Circle &a) const {
    const Circle null_c = {{0.0, 0.0}, 0.0};
    const Line null_l = {{0.0, 0.0},
                        {0.0, 0.0}};
    const Point v = a.c - c;
    if (a.is_in(c) == -1) {
        const long double d = r * r / (a.r + a.r);
        const Point p = c + v / v.len() * d;
        return {2, null_c, {p, {-v.y, v.x}}};
    }
    if (c == a.c) return {1, {c, r * r / a.r}, null_l};
    const long double d1 = r * r / (c.dis(a.c) - a.r), d2 = r * r / (c.dis(a.c) +
a.r);
    const Point p = c + v / v.len() * d1, q = c + v / v.len() * d2;
    return {1, {(p + q) / 2, p.dis(q) / 2}, null_l};
}
};

// 圆与任意多边形面积交
long double area_inter(const Circle &circ, const Polygon &poly) {
    const auto cal = [](const Circle &circ, const Point &a, const Point &b) {

```

```

    if ((a - circ.c).toleft(b - circ.c) == 0) return 0.01;
    const auto ina = circ.is_in(a), inb = circ.is_in(b);
    const Line ab = {a, b - a};
    if (ina && inb) return ((a - circ.c) ^ (b - circ.c)) / 2;
    if (ina && !inb) {
        const auto t = circ.inter(ab);
        const Point p = t.size() == 1 ? t[0] : t[1];
        const long double ans = ((a - circ.c) ^ (p - circ.c)) / 2;
        const long double th = (p - circ.c).ang(b - circ.c);
        const long double d = circ.r * circ.r * th / 2;
        if ((a - circ.c).toleft(b - circ.c) == 1) return ans + d;
        return ans - d;
    }
    if (!ina && inb) {
        const Point p = circ.inter(ab)[0];
        const long double ans = ((p - circ.c) ^ (b - circ.c)) / 2;
        const long double th = (a - circ.c).ang(p - circ.c);
        const long double d = circ.r * circ.r * th / 2;
        if ((a - circ.c).toleft(b - circ.c) == 1) return ans + d;
        return ans - d;
    }
    const auto p = circ.inter(ab);
    if (p.size() == 2 && Segment{a, b}.dis(circ.c) <= circ.r + eps) {
        const long double ans = ((p[0] - circ.c) ^ (p[1] - circ.c)) / 2;
        const long double th1 = (a - circ.c).ang(p[0] - circ.c), th2 = (b -
circ.c).ang(p[1] - circ.c);
        const long double d1 = circ.r * circ.r * th1 / 2, d2 = circ.r * circ.r *
th2 / 2;
        if ((a - circ.c).toleft(b - circ.c) == 1) return ans + d1 + d2;
        return ans - d1 - d2;
    }
    const long double th = (a - circ.c).ang(b - circ.c);
    if ((a - circ.c).toleft(b - circ.c) == 1) return circ.r * circ.r * th / 2;
    return -circ.r * circ.r * th / 2;
};

long double ans = 0;
for (size_t i = 0; i < poly.p.size(); i++) {
    const Point a = poly.p[i], b = poly.p[poly.nxt(i)];
    ans += cal(circ, a, b);
}
return ans;
}

// 半平面交
vector<Line> _halfinter(vector<Line> l) {
    constexpr double LIM = 1e9;
    const auto check = [](const Line &a, const Line &b, const Line &c) { return
a.toleft(b.inter(c)) < 0; };
    // const auto check= [](const Line &a,const Line &b,const Line &c)
    // {
    //     const Point p=a.v*(b.v^c.v),q=b.p*(b.v^c.v)+b.v*(c.v^(b.p-c.p))-
a.p*(b.v^c.v);
    //     return p.toleft(q)<0;
    // };
    l.push_back({{-LIM, 0},

```

```

        {0, -1}));
l.push_back({{0, -LIM},
            {1, 0}});
l.push_back({{LIM, 0},
            {0, 1}});
l.push_back({{0, LIM},
            {-1, 0}});
sort(l.begin(), l.end());
deque<Line> q;
for (size_t i = 0; i < l.size(); i++) {
    if (i > 0 && l[i - 1].v.toleft(l[i].v) == 0 && l[i - 1].v * l[i].v > eps)
continue;
    while (q.size() > 1 && check(l[i], q.back(), q[q.size() - 2])) q.pop_back();
    while (q.size() > 1 && check(l[i], q[0], q[1])) q.pop_front();
    q.push_back(l[i]);
}
while (q.size() > 1 && check(q[0], q.back(), q[q.size() - 2])) q.pop_back();
while (q.size() > 1 && check(q.back(), q[0], q[1])) q.pop_front();
if (q.size() <= 2) return vector<Line>();
return vector<Line>(q.begin(), q.end());
}

Convex halfinter(const vector<Line> &l) {
    const auto lines = _halfinter(l);
    Convex poly;
    poly.p.resize(lines.size());
    if (lines.empty()) return poly;
    for (size_t i = 0; i < lines.size(); i++) {
        const size_t j = (i == lines.size() - 1 ? 0 : i + 1);
        poly.p[i] = lines[i].inter(lines[j]);
    }
    poly.p.erase(unique(poly.p.begin(), poly.p.end()), poly.p.end());
    if (poly.p.front() == poly.p.back()) poly.p.pop_back();
    return poly;
}

pair<point_t, point_t> minmax_triangle(const vector<Point> &vec) {
    if (vec.size() <= 2) return {0, 0};
    vector<pair<int, int>> evt;
    evt.reserve(vec.size() * vec.size());
    point_t maxans = 0, minans = numeric_limits<point_t>::max();
    for (size_t i = 0; i < vec.size(); i++) {
        for (size_t j = 0; j < vec.size(); j++) {
            if (i == j) continue;
            if (vec[i] == vec[j]) minans = 0;
            else evt.push_back({i, j});
        }
    }
    sort(evt.begin(), evt.end(), [&](const pair<int, int> &u, const pair<int, int>
&v) {
        const Point du = vec[u.second] - vec[u.first], dv = vec[v.second] -
vec[v.first];
        return argcmp()({du.y, -du.x}, {dv.y, -dv.x});
    });
    vector<size_t> vx(vec.size(), pos(vec.size()));
    for (size_t i = 0; i < vec.size(); i++) vx[i] = i;
}

```

```

sort(vx.begin(), vx.end(), [&](int x, int y) { return vec[x] < vec[y]; });
for (size_t i = 0; i < vx.size(); i++) pos[vx[i]] = i;
for (auto [u, v]: evt) {
    const size_t i = pos[u], j = pos[v];
    const size_t _i = min(i, j), _j = max(i, j);
    const Point vecu = vec[u], vecv = vec[v];
    if (_i > 0) minans = min(minans, abs((vec[vx[_i - 1]] - vecu) ^ (vec[vx[_i - 1]] - vecv)));
    if (_j < vx.size() - 1) minans = min(minans, abs((vec[vx[_j + 1]] - vecu) ^ (vec[vx[_j + 1]] - vecv)));
    maxans = max({maxans, abs((vec[vx[0]] - vecu) ^ (vec[vx[0]] - vecv)),
                  abs((vec[vx.back()] - vecu) ^ (vec[vx.back()] - vecv))});
    if (i < j) swap(vx[i], vx[j]), pos[u] = j, pos[v] = i;
}
return {minans, maxans};
}

```

3.2. lyy 几何

// 点 直线 多边形 圆类型

// 极角排序

// 重载 + - * / ^ sign ==

// 两点距离 两线夹角 距离平方 距离 三角形面积 点到直线投影 点绕定点旋转 向量定长 两直线交点 直线向点方向移动固定距离

// 点与圆位置关系 0 1 2

// 直线与圆的位置关系, P1, P2 为交点

// 三角形与圆面积交 (三角形一点为圆心)

// 多边形与圆面积交

// 两圆面积交 (半径都非负)

// 多边形向内缩小 r

// 凸多边形内最大三角形 n^2

#include <bits/stdc++.h>

using namespace std;

const int N=10101;

typedef double db;

const db pi = acos(-1.0);

const db eps = 1e-8;

const db inf = 1e20;

const db gen2 = sqrt(2.0);

struct D{ db x,y; D(){ } D(db _x,db _y){x=_x;y=_y; } };

struct line{ D s,t; line(){ } line(D _s,D _t){s=_s;t=_t; } };

struct pol{ int n; D d[N]; line l[N]; };

struct cir{ db x,y; db r; cir(){ } cir(D P,db _r){x=P.x;y=P.y;r=_r; } };

D O = {0,0}, P = {-1,0};

inline bool cmp_point(D A,D B) { //极角排序 (三象限最小, 逆时针)

if(atan2(A.y,A.x)*atan2(B.y,B.x)<0.0) return atan2(A.y,A.x) < atan2(B.y,B.x);

return ((A-0) ^ (B-0)) > 0;

}

```

inline bool cmp_vec(D A,D B) { //极角排序 (OP 向量逆时针)
    if(((P-O)^(A-O))==0 && (P.x-O.x)*(A.x-O.x)>0) return 1;
    if(((P-O)^(B-O))==0 && (P.x-O.x)*(B.x-O.x)>0) return 0;
    if((((P-O)^(A-O))>0) != (((P-O)^(B-O))>0)) return ((P-O)^(A-O)) > ((P-O)^(B-O));
    return ((A-O) ^ (B-O)) > 0;
}

int sgn(db x) { if(fabs(x) < eps) return 0; if(x<0) return -1; return 1; }
bool operator == (D A,D B) { return sgn(A.x-B.x)==0 && sgn(A.y-B.y)==0; }
bool operator < (D A,D B) { return sgn(A.x-B.x)==0 ? sgn(A.y-B.y)<0 : A.x<B.x; }
D operator - (D A,D B) { return {A.x-B.x,A.y-B.y}; }
D operator + (D A,D B) { return {A.x+B.x,A.y+B.y}; }
db operator ^ (D A,D B) { return A.x*B.y - B.x*A.y; }
db operator * (D A,D B) { return A.x*B.x + A.y*B.y; }
D operator * (D A,db k) { return {A.x*k,A.y*k}; }
D operator / (D A,db k) { return {A.x/k,A.y/k}; }

db dist(D A,D B) { return hypot(A.x-B.x,A.y-B.y); } //两点距离
db rad(D P,D A,D B) { return fabs( atan2( fabs((A-P)^(B-P)), (A-P)*(B-P) ) ); }
db len2(D P) { return P.x*P.x + P.y*P.y; }
db len1(D P) { return sqrt(len2(P)); }
db SS(D A,D B,D C) { return fabs((A-B)^(A-C)) / 2.0; } //三角形面积
D lineprog(D P,line L) { //投影
    D M = L.t-L.s;
    return L.s + ( M * (M*(P-L.s) ) ) / len2(M);
}
D rotate(D A,D P,db ang) { // A 绕 P 逆时针转 ang 角度
    A = A-P; db c = cos(ang), s = sin(ang);
    return {P.x + A.x*c-A.y*s, P.y + A.x*s+A.y*c};
}
D trunc(D P,db k) { //向量定长
    db l = len1(P);
    if(!sgn(l)) return P;
    return (P/l) * k;
}
D cross(line A,line B){ //两直线交点
    db a1 = (A.t-A.s)^(B.s-A.s);
    db a2 = (A.t-A.s)^(B.t-A.s);
    return D((B.s.x*a2-B.t.x*a1)/(a2-a1),(B.s.y*a2-B.t.y*a1)/(a2-a1));
}
line move(line L,D P,db r) { //L 向 P 方向移动距离 r
    D H = lineprog(P,L);
    D vec = trunc(P-H,r);
    return {L.s+vec,L.t+vec};
}

int relat_D(D A,cir C) { //点与圆位置关系
    db d = dist(A,{C.x,C.y});
    if(sgn(d-C.r) < 0) return 2;
    if(sgn(d-C.r) > 0) return 0; // 0 在外
    return 1;
}

int relat_L(line L,cir C,D &p1,D &p2) { //直线与圆的位置关系 , P1,P2 为交点
    D P = {C.x,C.y};

```

```

    D A = lineprog(P,L);
    db d = dist(A,P);
    if(sgn(d-C.r)>0) return 0;
    if(sgn(d-C.r)==0) { p1=A; p2=A; return 1; }
    d = sqrt(C.r*C.r-d*d);
    p1 = A + trunc(L.t-L.s,d);
    p2 = A - trunc(L.t-L.s,d);
    return 2;
}

db TCArea(D A,D B,cir C) { //三角形与圆面积交 (三角形一点为圆心)
    D P = {C.x,C.y};
    if(sgn((P-A)^(P-B))==0) return 0.0;
    line L(A,B);
    D q[5];
    int cnt = 0;
    q[cnt++] = A;
    if(relat_L(L,C,q[1],q[2])==2) {
        if(sgn((A-q[1])*(B-q[1])) < 0) q[cnt++] = q[1];
        if(sgn((A-q[2])*(B-q[2])) < 0) q[cnt++] = q[2];
    }
    q[cnt++] = B;
    if(cnt==4 && sgn((q[0]-q[1])*(q[2]-q[1])) > 0) swap(q[1],q[2]);
    db ans = 0;
    for(int i=0; i<cnt-1; i++) {
        if(relat_D(q[i],C)==0 || relat_D(q[i+1],C)==0)
            ans += C.r*C.r*rad(P,q[i],q[i+1])/2.0;
        else
            ans += fabs((q[i]-P)^(q[i+1]-P))/2.0;
    }
    return ans;
}

db PCArea(pol P,cir C) { //多边形与圆面积交
    db ans = 0; D O = {C.x,C.y};
    for(int i=0;i<P.n;i++) {
        int j = (i+1) % P.n;
        db S = TCArea(P.d[i], P.d[j], C);
        if(sgn((P.d[i]-O)^(P.d[j]-O)) >= 0) ans += S;
        else ans -= S;
    }
    return ans;
}

db CCArea(cir c1,cir c2) { //两圆面积交 (半径都非负)
    db d = sqrt((c1.x-c2.x)*(c1.x-c2.x) + (c1.y-c2.y)*(c1.y-c2.y));
    if(d>=c1.r+c2.r) return 0;
    if(c1.r-c2.r>=d) return pi * c2.r * c2.r;
    if(c2.r-c1.r>=d) return pi * c1.r * c1.r;
    db ang1 = acos((d*d + c1.r*c1.r - c2.r*c2.r) / (2*c1.r*d));
    db ang2 = acos((d*d + c2.r*c2.r - c1.r*c1.r) / (2*c2.r*d));
    db s1 = ang1 * c1.r*c1.r;
    db s2 = ang2 * c2.r*c2.r;
    db s3 = d * c1.r * sin(ang1);
    return s1+s2-s3;
}

```

```

bool check(line A,line B,line C,db r) { //检查缩小后的边
    D p1 = B.t, p2 = C.s, p3 = A.s, p4 = A.t;
    line nB = move(B,p3,r);
    line nC = move(C,p4,r);
    line nA = move(A,p1,r);
    D p5 = cross(nA,nC);
    D p6 = cross(nA,nB);
    return sgn( (p6-p5)*(A.t-A.s) ) > 0;
}

pol shrink_pol(pol P,db r) { //多边形向内缩小 r
    pol Q; Q.n = 0;
    line A,B,C;
    for(int i=0;i<P.n;i++) {
        A = P.l[i];
        if(i!=P.n-1) B = P.l[i+1];
        else B = Q.l[0];
        if(Q.n) C = Q.l[Q.n-1];
        else C = P.l[P.n-1];
        if(check(A,B,C,r))
            Q.l[Q.n++] = A;
    }
    pol SQ; SQ.n = Q.n;
    for(int i=0;i<Q.n;i++) Q.d[i] = Q.l[i].s;
    for(int i=0;i<Q.n;i++) SQ.l[i] = move(Q.l[i],Q.d[(i+2)%Q.n],r);
    for(int i=0;i<Q.n;i++) SQ.d[i] = cross(SQ.l[i],SQ.l[(i-1+Q.n)%Q.n]);
    return SQ;
}

db max_T(pol P) { //凸多边形内最大三角形 n^2
    db ans = 0;
    for(int i=0;i<P.n;i++) {
        D S1 = P.d[i];
        int j = (i+1)%P.n;
        int k1 = (i+1)%P.n;
        int k2 = (i+2)%P.n;
        while(j!=i) {
            D S2 = P.d[j];
            if(j!=(i+1)%P.n) {
                while((k1+1)%P.n!=j && SS(S1,S2,P.d[k1]) < SS(S1,S2,P.d[(k1+1)%P.n]))
                    k1 = (k1+1) % P.n;
                ans = max(ans,SS(S1,S2,P.d[k1]));
            }
            if((j+1)%P.n!=i) {
                while((k2+1)%P.n!=i && SS(S1,S2,P.d[k2]) < SS(S1,S2,P.d[(k2+1)%P.n]))
                    k2 = (k2+1) % P.n;
                ans = max(ans,SS(S1,S2,P.d[k2]));
            }
            j = (j+1) % P.n;
        }
    }
    return ans;
}

void outing(db as) { cout<<fixed<<setprecision(6)<<as<<endl; }

```

3.3. lyy 凸包

```
const int N=101010;
const int qwq=303030;
const int inf=0x3f3f3f3f;
typedef double db;

int n,cnt;
struct D{
    double x,y;
    D(double xx=0,double yy=0) { x=xx, y=yy; }
} d[N],st[N];
D operator - (D A,D B) { return {A.x-B.x,A.y-B.y}; }
db operator ^ (D A,D B) { return A.x*B.y - B.x*A.y; }
inline bool cmp(D aa,D bb) { return (aa.x==bb.x) ? (aa.y<bb.y) : (aa.x<bb.x); }
double ans;

double ju(D aa,D bb) { return sqrt( (aa.x-bb.x)*(aa.x-bb.x) + (aa.y-bb.y)*(aa.y-bb.y) ); }

void TUBAO() {
    st[1] = d[1]; st[2] = d[2]; cnt = 2;
    for(int i=3;i<=n;i++) {
        while(cnt>1 && ((st[cnt]-st[cnt-1])^(d[i]-st[cnt-1]))<=0) cnt--;
        st[++cnt] = d[i];
    }
    st[++cnt] = d[n-1];
    for(int i=n-2;i>=1;i--) {
        while(cnt>1 && ((st[cnt]-st[cnt-1])^(d[i]-st[cnt-1]))<=0) cnt--;
        st[++cnt] = d[i];
    }
}

int main() {
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
        scanf("%lf%lf",&d[i].x,&d[i].y);
    sort(d+1,d+n+1,cmp);
    if(n==1) { cout<<0; return 0; }
    TUBAO();
    for(int i=2;i<=cnt;i++) ans += ju(st[i],st[i-1]);
    printf("%.2lf",ans);
    return 0;
}
```

4. 图论

4.1. tarjan 求点双边数

```
int n,m;
int qiao,tot,dian,da,shuang;
int bian;
int dfn[N],low[N],tim;
int st[N],cnt;
bool in[N];
vector<int> e[N],d[N];
```

```

void tarjan(int u,int zu) {
    dfn[u] = low[u] = ++tim;
    st[++cnt] = u;
    int son = 0;
    FOR() {
        int v = e[u][i];
        if(!dfn[v]) {
            son++;
            tarjan(v,zu);
            low[u] = min(low[u],low[v]);
            if(low[v]==dfn[u]) {
                ++tot;
                while(1) {
                    int now = st[cnt--];
                    d[tot].push_back(now);
                    d[now].push_back(tot);
                    if(now==v) break;
                }
                d[tot].push_back(u);
                d[u].push_back(tot);
            }
        }
        else low[u] = min(low[u],dfn[v]);
    }
    if(u==zu) cnt = 0;
}

int query(int fang) {
    int res = 0;
    for(int i=0;i<d[fang].size();i++) in[ d[fang][i] ] = 1;
    for(int i=0;i<d[fang].size();i++) {
        int u = d[fang][i];
        for(int j=0;j<e[u].size();j++) {
            int v = e[u][j];
            if(in[v]) res++;
        }
    }
    for(int i=0;i<d[fang].size();i++) in[ d[fang][i] ] = 0;
    return res/2;
}

int main() {
    int x,y;
    n = read(); m = read(); tot = n;
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i,i);

    for(int i=1;i<=n;i++) if(d[i].size()>=2) dian++;

    for(int i=n+1;i<=tot;i++) {
        if(d[i].size()==2) bian++;
        if(d[i].size()>=2) shuang++;
    }
}

```

```

    }

    for(int i=n+1;i<=tot;i++)
        da = max(da,query(i));
    cout<<dian<<" "<<bian<<" "<<shuang<<" "<<da;
    return 0;
}

```

4.2. Stoer-Wagner

```

const int N=1010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int T;
int n,m;
int ans = inf, a[N];
int dis[N][N];
int belong[N],vis[N],w[N];

void SW() {
    for(int h=n;h>=2;h--) {
        memset(vis,0,sizeof(vis));
        memset(w,0,sizeof(w));
        int t = 0, s = 0;
        for(int i=1;i<=h;i++) {
            s = t; t = 0;
            for(int j=1;j<=n;j++) {
                if(!belong[j] && !vis[j] && w[j]>=w[t]) t = j;
            }
            vis[t] = 1;
            for(int j=1;j<=n;j++) {
                if(!belong[j] && !vis[j]) w[j] += dis[t][j];
            }
        }
        belong[t] = s;
        ans = min(ans,w[t]);
        for(int j=1;j<=n;j++) {
            dis[s][j] += dis[t][j];
            dis[j][s] += dis[j][t];
        }
    }
}

int main() {
    int x,y,z;
    n = read(); m = read();
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        dis[x][y] = dis[y][x] = 1;
    }
    SW();
    cout<<ans;
    return 0;
}

```

4.3. 点分治

```
int n,Q;
struct E{ int to,we; };
vector <E> e[N];
int q[N],ans[N];
int big[N],sss,rt;
int siz[N],vis[N],dis[N];
int st[N],cnt;
int st2[N],cnt2;
bool f[qwq];

void getid(int u,int fa) {
    siz[u] = 1; big[u] = 0;
    FOR() {
        int v = e[u][i].to;
        if(v==fa || vis[v]) continue;
        getid(v,u);
        siz[u] += siz[v];
        big[u] = max(big[u],siz[v]);
    }
    big[u] = max(big[u],sss-siz[u]);
    if(big[u] < big[rt]) rt = u;
}

void DFS(int u,int fa) {
    if(dis[u]>10000000) return ;
    st[++cnt] = dis[u];
    FOR() {
        int v = e[u][i].to;
        if(vis[v] || v==fa) continue;
        dis[v] = dis[u] + e[u][i].we;
        DFS(v,u);
    }
}

void calc(int u) {
    FOR() {
        int v = e[u][i].to;
        if(vis[v]) continue;
        dis[v] = e[u][i].we;
        DFS(v,u);
        for(int j=1;j<=cnt;j++) {
            int d = st[j];
            for(int l=1;l<=Q;l++)
                if(q[l]>=d) ans[l] |= f[q[l]-d];
        }
        while(cnt) {
            f[ st[cnt] ] = 1;
            st2[++cnt2] = st[cnt--];
        }
    }
    while(cnt2) f[st2[cnt2--]] = 0;
}

void TREE(int u) {
    vis[u] = f[0] = 1;    // only TREE->u is visited.
```

```

    calc(u);
    FOR() {
        int v = e[u][i].to;
        if(vis[v]) continue;
        big[rt=0] = sss = siz[v];
        getid(v,0);
        TREE(rt);
    }
}

int main() {
    int x,y,z;
    n = read(); Q = read();
    for(int i=1;i<n;i++) {
        x = read(); y = read(); z = read();
        e[x].push_back(E){y,z};
        e[y].push_back(E){x,z};
    }
    for(int i=1;i<=Q;i++) q[i] = read();
    big[rt=0] = sss = n;
    getid(1,0);
    TREE(rt);
    for(int i=1;i<=Q;i++) {
        if(ans[i]) printf("AYE\n");
        else      printf("NAY\n");
    }
    return 0;
}

```

4.4. 差分约束

```

#include <bits/stdc++.h>
#define FOR() int le=e[u].size();for(int i=0;i<le;i++)
using namespace std;
const int N=101010;
const int inf=0x3f3f3f3f;

int n,m;
int dis[N],dep[N],in[N];
struct E{ int to,we; };
vector<E> e[N];
queue<int> q;

bool SPFA() {
    memset(dis,0x3f,sizeof(dis)); // min dis : the max solution
    // memset(dis,-0x3f,sizeof(dis)); // max dis : the min solution
    dis[0] = 0;
    q.push(0);
    while(!q.empty()) {
        int u = q.front(); q.pop(); in[u] = 0;
        FOR() {
            int v = e[u][i].to, w = e[u][i].we;
            if(dis[v] > dis[u] + w) { // max solution
                // if(dis[v] < dis[u] + w) { // min solution
                dis[v] = dis[u] + w;
                dep[v] = dep[u] + 1;
                if(dep[v]>n) return 0;
            }
        }
    }
}

```

```

        if(!in[v]) q.push(v);
        in[v] = 1;
    }
}
return 1;
}

int main() {
    int x,y,z;
    n = read(); m = read();
    while(m--) {
        x = read(); y = read(); z = read(); // x-y <= z
        e[y].push_back( (E){x,z} ); // x <= y+z    the max solution
        // e[x].push_back( (E){y,-z} ); // y >= x-z    the min solution
    }
    for(int i=1;i<=n;i++) e[0].push_back( (E){i,0} );
    if(!SPFA()) { cout<<"NO"; return 0; }
    for(int i=1;i<=n;i++) cout<<dis[i]<<" ";
    return 0;
}

```

4.5. 圆方树

```

const ll N=1010101;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;

int n,m;
int qiao,tot,dian,da,shuang;
int bian;
int dfn[N],low[N],tim;
bool cut[N];
int st[N],cnt;
bool in[N];
vector<int> e[N],d[N];

void tarjan(int u,int zu) {
    dfn[u] = low[u] = ++tim;
    st[++cnt] = u;
    int son = 0;
    FOR() {
        int v = e[u][i];
        if(!dfn[v]) {
            son++;
            tarjan(v,zu);
            low[u] = min(low[u],low[v]);
            if(low[v]==dfn[u]) {
                ++tot;
                while(1) {
                    int now = st[cnt--];
                    d[tot].push_back(now);
                    d[now].push_back(tot);
                    if(now==v) break;
                }
                d[tot].push_back(u);
            }
        }
    }
}

```

```

        d[u].push_back(tot);
    }
}
else low[u] = min(low[u],dfn[v]);

if(u==zu && son>=2) cut[u] = 1;
else if(low[v]>=dfn[u]) cut[u] = 1;
}
if(u==zu) cnt = 0;
}

int query(int fang) {
    int res = 0;
    for(int i=0;i<d[fang].size();i++) in[ d[fang][i] ] = 1;
    for(int i=0;i<d[fang].size();i++) {
        int u = d[fang][i];
        for(int j=0;j<e[u].size();j++) {
            int v = e[u][j];
            if(in[v]) res++;
        }
    }
    for(int i=0;i<d[fang].size();i++) in[ d[fang][i] ] = 0;
    return res/2;
}

int main() {
    int x,y;
    n = read(); m = read(); tot = n;
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i,i);

    for(int i=1;i<=n;i++) if(cut[i]) dian++;

    for(int i=n+1;i<=tot;i++) {
        if(d[i].size()==2) bian++;
        if(d[i].size()>=2) shuang++;
    }

    for(int i=n+1;i<=tot;i++)
        da = max(da,query(i));
    cout<<dian<<" "<<bian<<" "<<shuang<<" "<<da;
    return 0;
}

```

4.6. 并查集维护缩点

```

#include <bits/stdc++.h>
#define ll long long

using namespace std;
const int N=501010;
const int inf=0x3f3f3f3f;

```

```

int n,m;
int fa[N],dep[N];
vector<int> e[N];

int find(int x) { return fa[x]==x ? x : fa[x]=find(fa[x]); }

void merge(int x,int y) {
    if(x==y) return ;
    if(dep[x]<dep[y]) swap(x,y);
    fa[x] = y;
}

void tarjan(int u) {
    for(int v : e[u]) {
        if(!dep[v]) {
            dep[v] = dep[u] + 1;
            tarjan(v);
        }
        int vv = find(v), uu = find(u);
        if(dep[vv] > 0) merge(uu,vv);
    }
    dep[u] = -1;
}

int main() {
    int x,y;
    n = read(); m = read();
    for(int i=1;i<=n;i++) fa[i] = i;
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        e[x].push_back(y);
    }
    for(int i=1;i<=n;i++) {
        if(!dep[i]) tarjan(i);
    }

    for(int i=1;i<=n;i++) cout<<"belong["<<i<<" ] = "<<find(i)<<"\n";
    return 0;
}

```

4.7. 二分图最小点覆盖 (dinic)

```

#include<bits/stdc++.h>
#define ll long long

using namespace std;
const int N=101010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n1,n2,m,s,t;
int ans;
struct E{
    int to,nxt,cap;
}e[qwq];
int cnt = 1;

```

```

int head[N],cur[N];
int dep[N],vis[N];
queue <int> q;
int tag[N],belong[N];

inline void add(int u,int v,int w) {
    // cout<<u<<" -> "<<v<<" "<<w<<endl;
    e[++cnt] = (E){ v,head[u],w }; head[u] = cnt;
    e[++cnt] = (E){ u,head[v],0 }; head[v] = cnt;
}

inline bool SPFA() {
    for(int i=s;i<=t;i++) dep[i] = inf, vis[i] = 0, cur[i] = head[i];
    q.push(s); dep[s] = 0;
    while(!q.empty()) {
        int u = q.front(); q.pop();
        vis[u] = 0;
        for(int i=head[u]; i; i=e[i].nxt) {
            int v = e[i].to;
            if(dep[v] > dep[u] + 1 && e[i].cap) {
                dep[v] = dep[u] + 1;
                if(vis[v]) continue;
                q.push(v);
                vis[v] = 1;
            }
        }
    }
    return dep[t]!=inf;
}

int DFS(int u,int flow) {
    int res = 0, f;
    if(u==t || !flow) return flow;
    for(int i=cur[u]; i; i=e[i].nxt) {
        cur[u] = i;
        int v = e[i].to;
        if(e[i].cap && (dep[v] == dep[u]+1)) {
            f = DFS(v,min(flow-res,e[i].cap));
            if(f) {
                res += f;
                e[i].cap -= f;
                e[i^1].cap += f;
                if(res==flow) break;
            }
        }
    }
    return res;
}

void mark(int u) {
    if(tag[u]) return ;
    tag[u] = 1;
    for(int i=head[u]; i; i=e[i].nxt) {
        int v = e[i].to;
        if(v==s) continue;
        if(!tag[v] && belong[v]) {

```

```

        tag[v] = 1;
        mark(belong[v]);
    }
}

int main() {
    int x,y;
    n1 = read(); n2 = read(); m = read(); s = 0; t = n1+n2+1;
    for(int i=1;i<=n1;i++) add(s,i,1);
    for(int i=1;i<=n2;i++) add(i+n1,t,1);
    while(m--) {
        x = read(); y = read();
        add(x,y+n1,1);
    }
    while(SPFA()) ans += DFS(s,inf);

    for(int u=n1+1;u<t;u++) {
        for(int i=head[u]; i; i=e[i].nxt) {
            int v = e[i].to;
            if(e[i].cap && v!=t) vis[v] = 1, belong[u] = v;
        }
    }
    for(int i=1;i<=n1;i++) if(!vis[i]) mark(i);

    for(int i=1;i<=n1;i++) if(!tag[i]) cout<<i<<" ";
    cout<<endl;
    for(int i=n1+1;i<t;i++) if(tag[i]) cout<<i<<" ";
    cout<<endl;
    return 0;
}

/*

4 4 7
1 1
2 2
1 3
2 3
2 4
3 2
4 2

*/

```

4.8. 欧拉回路

```

const int N=1010101;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n,m;
struct E{
    int to,id;
};
vector <E> e[N];

```

```

int vis[N],cur[N];
int st[N],cnt;

void EULER(int u) {
    for(int i=cur[u];i<e[u].size();i=cur[u]) {
        cur[u]=i+1;
        int v = e[u][i].to, id = e[u][i].id;
        if(vis[id]) continue;
        vis[id] = 1;
        EULER(v);
        st[++cnt] = u;
    }
}

int main() {
    int x,y;
    n = read(); m = read();
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        e[x].push_back({y,i});
        e[y].push_back({x,i});
    }

    EULER(1);
    for(int i=cnt;i>=1;i--) cout<<st[i]<<" ";
    cout<<1<<"\n";
    return 0;
}

```

```

/*

```

```

7 8
1 2
2 3
3 4
4 5
5 1
3 6
6 7
7 3

```

```

*/

```

4.9. Seq

```

/*

```

```

S starts at 0
go range from 1 to n
n <- length of string S
*/

```

```

void build()
{
    for (int i = 0; i < 26; i++)
        last[i] = -1;
    for (int i = n; i; i--)

```

```

{
    for (int j = 0; j < 26; j++)
        go[i][j] = last[j];
    last[S[i - 1] - 'a'] = i;
}
int now1 = last[T[0] - 'a'];
for (int i = 1; i < m; i++)
    if (~now1)
        now1 = go[now1][T[i] - 'a'];
    else
        break;
}

```

4.10. 匈牙利算法

```

const int N=101010;
const int qwq=303030;
const int inf=0x3f3f3f3f;
int n,m,k;
int ans;
int vis[N],belong[N]; // index is the right_part of G(V,E)
int mp[1234][1234];

bool DFS(int u) {
    for(int i=1;i<=m;i++) {
        if(!mp[u][i] || vis[i]) continue; vis[i] = 1;
        if(!belong[i] || DFS(belong[i])) { belong[i] = u; return 1; }
    }
    return 0;
}

int main() {
    int x,y;
    scanf("%d%d%d",&n,&m,&k);
    while(k--) { scanf("%d%d",&x,&y); mp[x][y] = 1; }
    for(int i=1;i<=n;i++) {
        memset(vis,0,sizeof(vis));
        if(DFS(i)) ans++;
    }
    cout<<ans;
    return 0;
}

```

4.11. 最大流

```

const int N=101010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n,m,s,t;
struct E{
    int to,nxt,cap;
}e[qwq];
int cnt = 1;
int head[N],cur[N];
int dep[N],vis[N];
queue <int> q;

```

```

inline void add(int u,int v,int w) {
    e[++cnt] = (E){ v,head[u],w }; head[u] = cnt;
    e[++cnt] = (E){ u,head[v],0 }; head[v] = cnt;
}

inline bool SPFA() {
    for(int i=s;i<=t;i++) dep[i] = inf, vis[i] = 0, cur[i] = head[i];
    q.push(s); dep[s] = 0;
    while(!q.empty()) {
        int u = q.front(); q.pop();
        vis[u] = 0;
        for(int i=head[u]; i; i=e[i].nxt) {
            int v = e[i].to;
            if(dep[v] > dep[u] + 1 && e[i].cap) {
                dep[v] = dep[u] + 1;
                if(vis[v]) continue;
                q.push(v);
                vis[v] = 1;
            }
        }
    }
    return dep[t]!=inf;
}

int DFS(int u,int flow) {
    int res = 0, f;
    if(u==t || !flow) return flow;
    for(int i=cur[u]; i; i=e[i].nxt) {
        cur[u] = i;
        int v = e[i].to;
        if(e[i].cap && (dep[v] == dep[u]+1)) {
            f = DFS(v,min(flow-res,e[i].cap));
            if(f) {
                res += f;
                e[i].cap -= f;
                e[i^1].cap += f;
                if(res==flow) break;
            }
        }
    }
    return res;
}

int main() {
    int x,y,z;
    n = read(); m = read(); s = read(); t = read();
    while(m--) {
        x = read(); y = read(); z = read();
        add(x,y,z);
    }
    int max_flow = 0;
    while(SPFA()) max_flow += DFS(s,inf);
    cout<<max_flow;
    return 0;
}

```

4.12. 虚树

```
const int N=501010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int T;
int n,a[N],da;
vector<int> e[N];
vector<int> g[N];
int dfn[N],tot,dep[N];
int f[N][22],rec[N][22];
int ob[N];
int cnt,st[N];
vector<int> d[N];

inline bool cmp(int x,int y) { return dfn[x] < dfn[y]; }

void DFS(int u,int fa) {
    dep[u] = dep[fa] + 1;
    f[++tot][0] = dep[u]; rec[tot][0] = u;
    dfn[u] = tot;
    FOR() {
        int v = e[u][i];
        if(v==fa) continue;
        DFS(v,u);
        f[++tot][0] = dep[u]; rec[tot][0] = u;
    }
}

inline int LCA(int x,int y) {
    if(dfn[x]>dfn[y]) swap(x,y);
    int l = dfn[x], r = dfn[y], k = ob[r-l+1];
    if(f[l][k] < f[r-(1<<k)+1][k]) return rec[l][k];
    else return rec[r-(1<<k)+1][k];
}

void built(int h) {
    cout<<"\nh = "<<h<<endl;
    cnt = 0;
    st[++cnt] = 1;
    for(int v : g[h]) st[++cnt] = v;
    sort(st+1, st+cnt+1, cmp);
    int now = cnt;
    for(int i=2;i<=now;i++) st[++cnt] = LCA(st[i-1],st[i]);
    sort(st+1, st+cnt+1, cmp);
    cnt = unique(st+1, st+cnt+1) - st - 1;
    for(int i=2;i<=cnt;i++) {
        d[ LCA(st[i-1],st[i]) ].push_back( st[i] );
        cout<<LCA(st[i-1],st[i])<<" -> "<<st[i]<<endl;
    }
    // TREE(1);
    for(int i=1;i<=cnt;i++) d[st[i]].clear();
}

int main() {
```

```

int x,y;
ob[0] = -1; for(int i=1;i<=N-10;i++) ob[i] = ob[i>>1] + 1;
n = read(); da = read();
for(int i=1;i<=n;i++) a[i] = read(), g[a[i]].push_back(i);
for(int i=1;i<=n;i++) {
    x = read(); y = read();
    e[x].push_back(y);
    e[y].push_back(x);
}
DFS(1,1);
for(int k=1;k<=20;k++) {
    for(int i=1;i+(1<<k-1)<=tot;i++) {
        if(f[i][k-1] < f[i+(1<<k-1)][k-1])
            f[i][k] = f[i][k-1], rec[i][k] = rec[i][k-1];
        else
            f[i][k] = f[i+(1<<k-1)][k-1], rec[i][k] = rec[i+(1<<k-1)][k-1];
    }
}
for(int i=1;i<=da;i++) {
    built(i);
}
return 0;
}

/*

11 4
1 4 2 1 3 4 1 1 4 2 1
1 2
1 3
2 4
4 5
4 6
3 7
7 8
7 9
1 10
2 11

*/

```

4.13. 二分图最大权完美匹配 (KM)

```

typedef long double db;
const ll N=2010;
const ll qwq=2030303;
const ll inf=0x3f3f3f3f3f3f3f3f;

ll n,m,ans;
ll mpx[N],mpy[N],visx[N],visy[N],lx[N],ly[N];
ll li[N][N],slack[N],pre[N];

void BFS(ll u) {
    ll x,y=0,yy=0,d;
    memset(pre,0,sizeof(pre));
    memset(slack,0x3f,sizeof(slack));
}

```

```

    mpy[y] = u;
    while(1) {
        x = mpy[y]; d = inf; visy[y] = 1;
        for(int i=1;i<=n;i++) {
            if(visy[i]) continue;
            if(slack[i] > lx[x]+ly[i]-li[x][i]) {
                slack[i] = lx[x]+ly[i]-li[x][i];
                pre[i] = y;
            }
            if(slack[i]<d) {
                d = slack[i];
                yy = i;
            }
        }
        for(int i=0;i<=n;i++) {
            if(visy[i]) lx[mpy[i]] -= d, ly[i] += d;
            else slack[i] -= d;
        }
        y = yy;
        if(mpy[y]==-1) break;
    }
    while(y) {
        mpy[y] = mpy[pre[y]];
        y = pre[y];
    }
}

int main() {
    ll x,y,z;
    n = read(); m = read();
    memset(li,-0x3f,sizeof(li));
    memset(mpy,-1,sizeof(mpy));
    for(ll i=1;i<=m;i++) {
        x = read(); y = read(); z = read();
        li[x][y] = max(li[x][y], z);
    }

    for(ll i=1;i<=n;i++) {
        memset(visy,0,sizeof(visy));
        BFS(i);
    }

    ll ans = 0;
    for(ll i=1;i<=n;i++) ans += li[mpy[i]][i];
    cout<<ans<<"\n";
    for(ll i=1;i<=n;i++) cout<<mpy[i]<<" ";
    return 0;
}

```

4.14. 删边最短路

```

#include <bits/stdc++.h>
#define ll long long
#define ls now<<1
#define rs now<<1|1

using namespace std;

```

```

const ll N=202020;
const ll inf=0x3f3f3f3f3f3f3f3f;

ll n,m,Q;
ll X[N],Y[N],Z[N];
struct E{
    ll to,we,id;
};
vector<E> e[N];
ll dis1[N],dis2[N],L[N],R[N],pre[N];
ll vis[N],in[N],da;
struct D {
    ll id,di;
};
inline bool operator < (D A,D B) { return A.di > B.di; }
priority_queue<D> q;

void DIJ(ll s,ll *dis,ll cl) {
    for(ll i=1;i<=n;i++) dis[i] = inf;
    dis[s] = 0;
    q.push({s,0});
    while(!q.empty()) {
        D now = q.top(); q.pop();
        ll u = now.id;
        if(dis[u]!=now.di) continue;
        for(E vv : e[u]) {
            ll v = vv.to;
            if(dis[v] > dis[u]+vv.we) {
                dis[v] = dis[u]+vv.we;
                pre[v] = vv.id;
                if(cl==1 && !vis[v]) L[v] = L[u];
                if(cl==2 && !vis[v]) R[v] = R[u];
                q.push({v,dis[v]});
            }
        }
    }
}

void access() {
    ll u = 1;
    vis[u] = 1; L[u] = R[u] = 0;
    while(u!=n) {
        ll id = pre[u];
        in[id] = ++da;
        u ^= X[id] ^ Y[id];
        vis[u] = 1;
        L[u] = R[u] = da;
    }
}

ll t[N<<2];
void built(ll now,ll l,ll r) {
    t[now] = inf;
    if(l==r) return ;
    ll mid = l+r >> 1;
    built(ls, l, mid);

```

```

    built(rs, mid+1, r);
}

void insert(ll now,ll l,ll r,ll x,ll y,ll g) {
    if(x<=l && r<=y) { t[now] = min(t[now],g); return; }
    ll mid = l+r >> 1;
    if(x<=mid) insert(ls, l, mid, x, y, g);
    if(y>mid) insert(rs, mid+1, r, x, y, g);
}

ll query(ll now,ll l,ll r,ll x) {
    if(l==r) return t[now];
    ll mid = l+r >> 1, res = t[now];
    if(x<=mid) res = min(res, query(ls, l, mid, x));
    else res = min(res, query(rs, mid+1, r, x));
    return res;
}

int main() {
    n = read(); m = read(); Q = read();
    for(ll i=1;i<=m;i++) {
        X[i] = read(); Y[i] = read(); Z[i] = read();
        e[X[i]].push_back({Y[i],Z[i],i});
        e[Y[i]].push_back({X[i],Z[i],i});
    }
    DIJ(n, dis2, 0);
    access();
    DIJ(1, dis1, 1);
    DIJ(n, dis2, 2);
    built(1, 1, da);
    for(ll i=1;i<=m;i++) {
        if(!in[i]) {
            ll u = X[i], v = Y[i];
            if(L[u]<R[v]) insert(1, 1, da, L[u]+1, R[v], dis1[u]+dis2[v]+Z[i]);
            if(L[v]<R[u]) insert(1, 1, da, L[v]+1, R[u], dis1[v]+dis2[u]+Z[i]);
        }
    }
    ll id,x;
    while(Q--) {
        id = read(); x = read();
        ll ans = dis1[n];
        if(!in[id]) {
            if(x<Z[id]) {
                ans = min(ans, dis1[X[id]]+dis2[Y[id]]+x);
                ans = min(ans, dis1[Y[id]]+dis2[X[id]]+x);
            }
        }
        else {
            ans = ans-Z[id]+x;
            if(x>Z[id]) {
                ans = min(ans, query(1, 1, da, in[id]));
            }
        }
        cout<<ans<<"\n";
    }
}

```

```

    return 0;
}

```

4.15. 树哈希

```

// solution 1
ll base = 13333;
bool cmp(ll A,ll B) { return ha[A] < ha[B]; }

void TREE(ll u) {
    for(ll i=0;i<e[u].size();i++) TREE(e[u][i],cl);
    sort(e[u].begin(),e[u].end(),cmp);
    ha[u] = a[u];
    ll now = 114514;
    for(ll v : e[u]) {
        (ha1[u] += now * ha1[u] %p) %= p;
        (now *= base) %= p;
    }
}

```

```

// solution 2
void dfs3(int u){
    dp1[u]=827,dp2[u]=827;
    for(int i=0;i<g2[u].size();i++){
        int v=g2[u][i];
        dfs3(v);
        dp1[u]+=(u11)(siz[v]+dp1[v]-(siz[v]^dp1[v])+siz[u]);
        dp2[u]+=(u11)(siz[v]+dp2[v]+(siz[v]^dp2[v])+siz[u]);
    }
    dp1[u]*=829;
    dp2[u]*=829;
}

```

4.16. 二分图最小点覆盖

```

#include <bits/stdc++.h>
#define ll long long

using namespace std;
const int N=1010;

int n1,n2,m;
vector<int> e[N];
int vis[N],belong[N]; // index is the right_part of G(V,E)
bool tagx[N],tagy[N];

int DFS(int u) {
    for(int v : e[u]) {
        if(vis[v]) continue;
        vis[v] = 1;
        if(!belong[v] || DFS(belong[v])) { belong[v] = u; return 1; }
    }
    return 0;
}

void mark(int u) {
    if(tagx[u]) return ;
}

```

```

    tagx[u] = 1;
    for(int v : e[u]) {
        if(!tagy[v] && belong[v]) {
            tagy[v] = 1;
            mark(belong[v]);
        }
    }
}

int main() {
    int x,y;
    n1 = read(); n2 = read(); m = read();
    while(m--) {
        x = read(); y = read();
        e[x].push_back(y);
    }
    int ans = 0;
    for(int i=1;i<=n1;i++) {
        memset(vis, 0, sizeof(vis));
        ans += DFS(i);
    }
    cout<<"ans = "<<ans<<endl;
    memset(vis, 0, sizeof(vis));
    for(int i=1;i<=n2;i++) vis[ belong[i] ] = 1;
    for(int i=1;i<=n1;i++) if(!vis[i]) mark(i);
    for(int i=1;i<=n1;i++) if(!tagx[i]) cout<<i<<" ";
    cout<<endl;
    for(int i=1;i<=n2;i++) if(tagy[i]) cout<<i<<" ";
    cout<<endl;
    return 0;
}

```

/*

```

4 4 7
1 1
2 2
1 3
2 3
2 4
3 2
4 2

```

*/

4.17. 带花树

```

int n,m,ans;
int fa[N],cl[N],mp[N],pre[N];
int cnt,tim[N];
vector<int> e[N];
queue<int> q;

int find(int x) { return (fa[x]==x) ? x : fa[x]=find(fa[x]); }

inline void link(int x,int y) { mp[x] = y; mp[y] = x; }

```

```

void rev(int x) { if(x) { rev(mp[pre[x]]), link(x,pre[x]); } }

inline int lca(int x,int y) {
    ++cnt;
    x = find(x); y = find(y);
    while(tim[x]!=cnt) {
        tim[x] = cnt;
        x = find(pre[ mp[x] ]);
        if(y) swap(x,y);
    }
    return x;
}

inline void blossom(int x,int y,int ff) {
    for(; find(x)!=ff; x=pre[y]) {
        pre[x] = y;
        y = mp[x];
        fa[x] = fa[y] = ff;
        if(cl[y]==2) cl[y] = 1, q.push(y);
    }
}

int BFS(int s) {
    for(int i=1;i<=n;i++) pre[i] = cl[i] = 0, fa[i] = i;
    while(!q.empty()) q.pop();
    cl[s] = 1; q.push(s);
    while(!q.empty()) {
        int u = q.front(); q.pop();
        FOR() {
            int v = e[u][i];
            if(cl[v]==1) {
                int ff = lca(u,v);
                blossom(u,v,ff);
                blossom(v,u,ff);
            }
            else if(!cl[v]) {
                pre[v] = u; cl[v] = 2;
                if(!mp[v]) { rev(v); return 1; }
                else { cl[ mp[v] ] = 1; q.push(mp[v]); }
            }
        }
    }
    return 0;
}

int main() {
    int x,y;
    n = read(); m = read();
    for(int i=1;i<=n;i++) fa[i] = i;
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    for(int i=1;i<=n;i++) {

```

```

        if(!mp[i]) ans += BFS(i);
    }
    cout<<ans<<"\n";
    return 0;
}

```

4.18. 有源汇上下界最大流

```

#include <bits/stdc++.h>
#define ll long long

using namespace std;
const ll N=101010;
const ll inf=0x3f3f3f3f;

ll n,m,s,t,ss,tt;
struct EE{ ll u,v,l,r; } a[N]; ll cnta;
ll du[N];
struct E{
    ll to,nxt,cap;
}e[N];
ll cnt = 1;
ll head[N],cur[N];
ll dep[N],vis[N];
queue <ll> q;

inline void add(ll u,ll v,ll w) {
    // cout<<u<<" -> "<<v<<" "<<w<<endl;
    e[++cnt] = (E){ v,head[u],w }; head[u] = cnt;
    e[++cnt] = (E){ u,head[v],0 }; head[v] = cnt;
}

inline bool SPFA(ll S,ll T) {
    for(ll i=0;i<=tt;i++) dep[i] = inf, vis[i] = 0, cur[i] = head[i];
    q.push(S); dep[S] = 0;
    while(!q.empty()) {
        ll u = q.front(); q.pop();
        vis[u] = 0;
        for(ll i=head[u]; i; i=e[i].nxt) {
            ll v = e[i].to;
            if(dep[v] > dep[u] + 1 && e[i].cap) {
                dep[v] = dep[u] + 1;
                if(vis[v]) continue;
                q.push(v);
                vis[v] = 1;
            }
        }
    }
    return dep[T]!=inf;
}

ll DFS(ll u,ll goal,ll flow) {
    ll res = 0, f;
    if(u==goal || !flow) return flow;
    for(ll i=cur[u]; i; i=e[i].nxt) {
        cur[u] = i;
        ll v = e[i].to;

```

```

        if(e[i].cap && (dep[v] == dep[u]+1)) {
            f = DFS(v,goal,min(flow-res,e[i].cap));
            if(f) {
                res += f;
                e[i].cap -= f;
                e[i^1].cap += f;
                if(res==flow) break;
            }
        }
    }
    return res;
}

ll solve() {
    ll ans1 = 0, ans2 = 0, cha = 0;
    for(ll i=1;i<=cnta;i++) {
        ll u = a[i].u, v = a[i].v;
        du[u] += a[i].l;
        du[v] -= a[i].l;
        add(u, v, a[i].r-a[i].l);
    }
    ll lim = cnt;
    for(ll u=s;u<=t;u++) {
        if(du[u]>0) add(u,tt,du[u]), cha += du[u];
        if(du[u]<0) add(ss,u,-du[u]);
    }
    add(t,s,inf);
    while(SPFA(ss,tt)) cha -= DFS(ss,tt,inf);
    if(cha) return -1;

    ans1 = e[cnt].cap;

    for(ll i=lim+1;i<=cnt;i++) e[i].cap = 0;
    while(SPFA(s,t)) ans2 += DFS(s,t,inf);

    return ans1+ans2;
}

void chushihua() {
    cnt = 1;
    cnta = 0;
    for(ll i=0;i<=tt;i++) head[i] = du[i] = 0;
}

int main() {
    ll x,y,l,r;
    while(scanf("%lld%lld",&n,&m)!=EOF) {
        chushihua();

        s = 0; t = n+m+1; ss = n+m+2; tt = n+m+3;

        for(int i=1;i<=m;i++) {
            x = read(); y = read(); l = read(); r = read();
            a[++cnta] = {x,y,l,r};
        }
    }
}

```

```

        cout<<solve()<<"\n\n";
    }
    return 0;
}

```

4.19. tarjan 求桥和割点

```

#include <bits/stdc++.h>
#include <vector>

using namespace std;

const int N=101010;
const int qwq = 303030;

int n,m;
int X[N],Y[N],Z[N];
int head[N],to[qwq],cnt=1,we[qwq],nxt[qwq];
int dfn[N],low[N],tim;
bool cut[N],br[qwq];

void add(int u,int v,int z) {
    to[++cnt] = v;
    we[cnt] = z;
    nxt[cnt] = head[u];
    head[u] = cnt;

    to[++cnt] = u;
    we[cnt] = z;
    nxt[cnt] = head[v];
    head[v] = cnt;
}

void tarjan(int u,int fa) {
    dfn[u] = low[u] = ++tim;
    int son = 0;
    for(int i=head[u]; i; i=nxt[i]) {
        int v = to[i];
        if(v==fa) continue;
        if(!dfn[v]) {
            if(!fa) son++;
            tarjan(v,u);
            low[u] = min(low[u],low[v]);
            if(fa && low[v]>=dfn[u]) cut[u] = 1;
            if(low[v]>dfn[u]) br[i] = 1, br[i^1] = 1;
        }
        else low[u] = min(low[u],dfn[v]);
    }
    if(son>=2) cut[u] = 1;
}

int main() {
    n = read(); m = read();
    for(int i=1;i<=m;i++) {
        X[i] = read(); Y[i] = read(); Z[i] = read();
        add(X[i],Y[i],Z[i]);
    }
}

```

```

    }
    for(int i=1;i<=n;i++)
        if(!dfn[i]) tarjan(i,0);
    for(int i=1;i<=n;i++) {
        cout<<"cut["<<i<<"] = "<<cut[i]<<endl;
    }
    for(int i=2;i<=cnt;i++) {
        cout<<"to = "<<to[i]<<"  br = "<<br[i]<<"\n";
    }
    return 0;
}

```

4.20. K 短路

```

const ll N=101010;
const ll qwq=503030;
const ll inf=0x3f3f3f3f3f3f3f;

ll n,m,k;
ll dis[N],fa[N],head1[N],head2[N];
bool vis[N];
struct E{
    ll to,we,nxt,on;
} e[qwq],g[qwq]; ll cnt;
struct D{
    ll id,di;
};
bool operator < (D A,D B) { return A.di > B.di; }
priority_queue <D> q;

ll tot=0, rt[N<<4], ls[N<<4], rs[N<<4], dep[N<<4];
D tree[N<<4];

void add(ll u,ll v,ll z) {
    cnt++;
    e[cnt] = (E){v,z,head1[u],0}; head1[u] = cnt;
    g[cnt] = (E){u,z,head2[v],0}; head2[v] = cnt;
}

void DIJ() {
    memset(dis,0x3f,sizeof(dis));
    q.push({n,0}); dis[n] = 0;
    while(!q.empty()) {
        D now = q.top(); q.pop();
        ll u = now.id;
        if(dis[u]!=now.di) continue;
        for(ll i=head2[u]; i; i=g[i].nxt) {
            ll v = g[i].to, w = g[i].we;
            if(dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                q.push({v,dis[v]});
            }
        }
    }
}

```

```

ll addnew(ll u,ll di) { tree[++tot] = {u,di}; return tot; }

ll merge(ll x,ll y) {
    if(!x || !y) return x | y;
    if(tree[x] < tree[y]) swap(x,y);
    ll now = ++tot;
    tree[now] = tree[x];
    ls[now] = ls[x];
    rs[now] = merge(rs[x],y);
    if(dep[ls[now]] < dep[rs[now]]) swap(ls[now],rs[now]);
    dep[now] = dep[rs[now]] + 1;
    return now;
}

void DFS(ll u) {
    // cout<<"u = "<<u<<"\n";
    vis[u] = 1;
    for(ll i=head2[u]; i; i=g[i].nxt) {
        ll v = g[i].to;
        if(vis[v]) continue;
        if(dis[v] == dis[u]+g[i].we) {
            fa[v] = u;
            e[i].on = 1;
            DFS(v);
        }
    }
}

void DFS2(ll u) {
    // cout<<" u = "<<u<<"\n";
    vis[u] = 1;
    if(fa[u]) rt[u] = merge(rt[u], rt[ fa[u] ]);
    for(ll i=head2[u]; i; i=g[i].nxt) {
        ll v = g[i].to;
        if(fa[v]==u && !vis[v]) DFS2(v);
    }
}

void built() {
    for(ll u=1;u<=n;u++) {
        if(dis[u]==inf) continue;
        for(ll i=head1[u]; i; i=e[i].nxt) {
            ll v = e[i].to;
            if(e[i].on || dis[v]==inf) continue;
            rt[u] = merge(rt[u],addnew(v,dis[v]-dis[u]+e[i].we));
        }
    }
}

int main() {
    ll x,y,z;
    n = read(); m = read(); k = read();
    for(ll i=1;i<=m;i++) {
        x = read(); y = read(); z = read();
        add(x,y,z);
    }
}

```

```

DIJ(); DFS(n);
if(k==1) { cout<<dis[1]; return 0; }
memset(vis,0,sizeof(vis));
built();
DFS2(n);

q.push( {rt[1], tree[rt[1]].di} );
while(!q.empty()) {
    D now = q.top(); q.pop();
    ll u = now.id, w = now.di;
    k--; if(k==1) { cout<<dis[1]+w; return 0; }
    if(ls[u]) q.push( {ls[u], w-tree[u].di+tree[ls[u]].di} );
    if(rs[u]) q.push( {rs[u], w-tree[u].di+tree[rs[u]].di} );
    ll v = rt[ tree[u].id ];
    if(v) q.push( {v, w+tree[v].di} );
}
return 0;
}

```

4.21. 2-sat

```

#include <bits/stdc++.h>
#define FOR() int le=e[u].size();for(int i=0;i<le;i++)
#define QWQ cout<<"QwQ"<<endl;
#define ll long long
#include <vector>
#include <map>

using namespace std;
const int N=2010101;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n,m;
int belong[N],tot=0,low[N],dfn[N],tim=0;
int in[N],cnt,st[N];
vector <int> e[N];

void tarjan(int u) {
    dfn[u] = low[u] = ++tim;
    st[++cnt] = u; in[u] = 1;
    FOR() {
        int v = e[u][i];
        if(!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u],low[v]);
        }
        else if(in[v]) low[u] = min(low[u],dfn[v]);
    }
    if(low[u]==dfn[u]) {
        tot++;
        while(1) {
            int v = st[cnt--]; in[v] = 0;
            belong[v] = tot;
            if(v==u) break;
        }
    }
}

```

```

}

int main() {
    int x,lx,y,ly;
    n = read(); m = read();
    while(m--) {
        x = read(); lx = read(); y = read(); ly = read();
        if(!lx) x += n;    // x is false
        if(!ly) y += n;
        if(x>n) e[x-n].push_back(y); // if !x : y
        else    e[x+n].push_back(y);
        if(y>n) e[y-n].push_back(x);
        else    e[y+n].push_back(x);
    }
    for(int i=1;i<=2*n;i++)
        if(!dfn[i]) tarjan(i);
    for(int i=1;i<=n;i++) if(belong[i]==belong[i+n]) { printf("IMPOSSIBLE"); return
0; }
    printf("POSSIBLE\n");
    for(int i=1;i<=n;i++) {
        if(belong[i] > belong[i+n]) cout<<"0 ";    // (i) -> (i+n)    so i is false
        else    cout<<"1 ";
    }
    return 0;
}

```

4.22. 矩阵树定理

```

const int p=998244353;

int n,m;
int du[345];
int a[345][345];

int ksm(int aa,int bb) {
    int sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

int det() {
    int res = 1, cl = 1;
    for(int i=1;i<=n;i++) for(int j=1;j<=n;j++) a[i][j] = (a[i][j] + p) %p;
    for(int i=1;i<=n;i++) {
        if(!a[i][i])
            for(int j=i+1;j<=n;j++)
                if(a[j][i]) { cl ^= 1; for(int k=i;k<=n;k++) swap(a[j][k],a[i][k]);
break; }
        if(!a[i][i]) return 0;
        (res *= a[i][i]) %= p;
        int ni = ksm(a[i][i],p-2); for(int j=i;j<=n;j++) (a[i][j] *= ni) %= p;
        for(int j=1;j<=n;j++) {
            if(i!=j) {

```

```

        int bei = a[j][i];
        for(int k=i;k<=n;k++)
            a[j][k] = (a[j][k] - a[i][k] * bei %p +p) %p;
    }
}
return cl ? res : -res;
}

int main() {
    int x,y;
    n = read(); m = read();
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        du[x]++; du[y]++;
        a[x][y]--; a[y][x]--;
    }
    for(int i=1;i<=n;i++) a[i][i] = du[i];
    // for(int i=1;i<=n;i++) {
    //     for(int j=1;j<=n;j++) cout<<a[i][j]<<" ";
    //     cout<<endl;
    // }

    // 无向图:  $L(n) = D(n) - A(n)$ ;
    // 有向图根向:  $L(n) = D_{out}(n) - A(n)$ ; //k 为根去掉第 k 行
    // 有向图叶向:  $L(n) = D_{in}(n) - A(n)$ ;
    n--; //去一行
    cout<<det();
    return 0;
}

```

4.23. 边分治缩点

```

#define ls now<<1
#define rs now<<1|1

using namespace std;
const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;

ll T;
ll n,m;
int ans[N], _ans;
int fa[N];
struct E{
    int x,y,ti;
};
vector<E> d[N<<2];
vector<int> e[N];
int tim,dfn[N],low[N],belong[N];
int in[N],st[N],cnt;

int find(int x) { return fa[x]==x ? x : fa[x]=find(fa[x]); }

inline void merge(int x,int y) {
    x = find(x); y = find(y);
}

```

```

    if(x==y) return ;
    fa[x] = y;
    _ans--;
}

void tarjan(int u) {
    dfn[u] = low[u] = ++tim;
    st[++cnt] = u; in[u] = 1;
    for(int v : e[u]) {
        if(!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        }
        else if(in[v]) low[u] = min(low[u], dfn[v]);
    }
    if(low[u]==dfn[u]) {
        while(1) {
            int v = st[cnt--]; in[v] = 0;
            belong[v] = u;
            if(v==u) break;
        }
    }
}

void solve(ll now, ll l, ll r) {
    // cout<<"solve("<<l<<","<<r<<") : ";
    // for(E vv : d[now]) cout<<vv.x<<"->"<<vv.y<<" ";
    // cout<<endl;
    if(l==r) {
        for(E vv : d[now]) merge(vv.x, vv.y);
        vector<E>().swap(d[now]);
        ans[l] = _ans;
        return ;
    }
    ll mid = l+r >> 1;
    for(E &vv : d[now]) {
        e[vv.x = find(vv.x)].clear();
        e[vv.y = find(vv.y)].clear();
        dfn[vv.x] = dfn[vv.y] = 0;
    }
    for(E vv : d[now]) if(vv.ti<=mid) e[vv.x].push_back(vv.y);
    for(E vv : d[now]) {
        if(vv.ti<=mid) {
            if(!dfn[vv.x]) tarjan(vv.x);
            if(!dfn[vv.y]) tarjan(vv.y);
            if(belong[vv.x]==belong[vv.y]) d[ls].push_back(vv);
            else d[rs].push_back(vv);
        }
        else {
            d[rs].push_back(vv);
        }
    }
    vector<E>().swap(d[now]);
    solve(ls, l, mid);
    solve(rs, mid+1, r);
}

```

```

void chushihua() {
    _ans = 0;
    tim = 0;
}

int main() {
    int x,y;
    T = read();
    while(T--) {
        chushihua();
        n = read(); m = read();
        _ans = n;
        for(int i=1;i<=n;i++) fa[i] = i;
        for(int i=1;i<=m;i++) {
            x = read(); y = read();
            d[1].push_back({x,y,i});
        }
        solve(1, 1, m+1);
        for(int i=1;i<=m;i++) cout<<ans[i]<<"\n";
    }
    return 0;
}

```

4.24. 费用流

```

const int N=101010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n,m,s,t,cnt=1,maxflow,mincost;
struct E{ int to,nxt,cap,we; } e[N];
int head[N],dep[N],vis[N],cur[N];
queue <int> q;

inline void add(int u,int v,int w,int z) {
    e[++cnt] = (E){ v,head[u],w,z }; head[u] = cnt;
    e[++cnt] = (E){ u,head[v],0,-z }; head[v] = cnt;
}

bool SPFA() {
    for(int i=0;i<=t;i++) cur[i] = head[i], vis[i] = 0, dep[i] = inf;
    dep[s] = 0; q.push(s);
    while(!q.empty()) {
        int u = q.front(); q.pop(); vis[u] = 0;
        for(int i=head[u]; i; i=e[i].nxt) {
            int v = e[i].to, w = e[i].we;
            if(e[i].cap && dep[v] > dep[u]+w) {
                dep[v] = dep[u]+w;
                if(vis[v]) continue;
                q.push(v); vis[v] = 1;
            }
        }
    }
    return dep[t] != inf;
}

```

```

int DFS(int u,int flow) {
    int res = 0, f;
    if(u==t || !flow) return flow;
    vis[u] = 1;
    for(int i=cur[u]; i; i=e[i].nxt) {
        cur[u] = i;
        int v = e[i].to, w = e[i].we;
        if(vis[v]) continue;
        if(e[i].cap && dep[v]==dep[u]+w) {
            f = DFS( v, min(e[i].cap,flow-res) );
            if(f) {
                res += f;
                e[i].cap -= f;
                e[i^1].cap += f;
                mincost += f * w;
                if(res==flow) break;
            }
        }
    }
    return res;
}

int main() {
    int x,y,z,w;
    n = read(); m = read(); s = read(); t = read();
    while(m--) {
        x = read(); y = read(); z = read(); w = read();
        add(x,y,z,w);
    }
    while( SPFA() ) maxflow += DFS(s,inf);
    cout<<maxflow<<" "<<mincost;
    return 0;
}

```

5. 字符串

5.1. runs (hash)

```

#include <bits/stdc++.h>
#define ll long long
#define QWQ cout<<"QwQ\n";

using namespace std;
const ll N=1010101;
const ll inf=0x3f3f3f3f;
const ll p=998244353;

ll n;
char s[N];
ll a[N];
ll ha[N],fi[N],base = 2333;

ll ask(ll l,ll r) {
    return (ha[r] - ha[l-1] * fi[r-l+1] %p + p) %p;
}

```

```

inline ll lcp(ll x,ll y) {
    ll l=1, r=n-max(x,y)+1, res = 0;
    while(l<=r) {
        ll mid = (l+r) >> 1;
        if(ask(x,x+mid-1) == ask(y,y+mid-1)) l = mid+1, res = mid;
        else r = mid-1;
    }
    return res;
}

inline ll lcs(ll x,ll y) {
    ll l=1, r=min(x,y), res=0;
    while(l<=r) {
        ll mid = (l+r) >> 1;
        if(ask(x-mid+1,x) == ask(y-mid+1,y)) l = mid+1, res = mid;
        else r = mid-1;
    }
    return res;
}

bool cmp(ll l1,ll r1,ll l2,ll r2) {
    ll l = lcp(l1,l2);
    if(l>min(r1-l1,r2-l2)) return r1-l1<r2-l2;
    return a[l1+l]<a[l2+l];
}

struct runs{
    ll i,j,p;
    runs(ll i=0,ll j=0,ll p=0) : i(i), j(j), p(p) {}
    bool operator == (const runs A) const { return i==A.i && j==A.j && p==A.p; }
    bool operator < (const runs A) const { return i==A.i ? j<A.j : i<A.i; }
};

vector <runs> ans;
ll st[N],run[N];
void lyndon() {
    ll r = 0;
    for(ll i=n;i;i--) {
        st[++r] = i;
        for(;r>1 && cmp(i,st[r],st[r]+1,st[r-1]);r--);
        run[i] = st[r];
    }
}

void get_runs()
{
    for(int i=1;i<=n;i++) {
        int l1=i, r1=run[i], l2=l1-lcs(l1-1,r1), r2=r1+lcp(l1,r1+1);
        if(r2-l2+1>=(r1-l1+1)*2) ans.push_back(runs(l2,r2,r1-l1+1));
    }
}

int main() {
    fi[0] = 1;
    for(ll i=1;i<=N-10;i++) fi[i] = fi[i-1] * base %p;

    scanf("%s",s+1); n = strlen(s+1);
}

```

```

for(ll i=1;i<=n;i++) a[i] = s[i]-'a'+1, ha[i] = (ha[i-1] * base %p + a[i]) %p;

// int x,y;      // ask the lcp() and lcs();
// while(1) {
//   cin>>x>>y;
//   cout<<lcp(x,y)<<" "<<lcs(x,y)<<"\n";
// }

lyndon();
// for(int i=1;i<=n;i++) cout<<run[i]<<" ";
get_runs();

for(ll i=1;i<=n;i++) a[i] = 27-a[i];
lyndon();
get_runs();

sort(ans.begin(), ans.end());
ans.erase(unique(ans.begin(), ans.end()), ans.end());
cout<<ans.size()<<"\n";
for(runs vv : ans) cout<<vv.i<<" "<<vv.j<<" "<<vv.p<<endl;
return 0;
}

```

5.2. PAM

```

struct PAM {
    int a[N];
    int cnt=1, lst=0;
    int ch[N][26], fail[N], len[N], dep[N];

    int getfail(int i, int x) {
        while(a[i-len[x]-1] != a[i]) x = fail[x];
        return x;
    }

    void built() {
        len[1] = -1; fail[0] = 1;
        a[0] = -2333;
        for(int i=1;i<=n;i++) {
            int u = getfail(i, lst), c = a[i];
            if(!ch[u][c]) {
                int w = getfail(i, fail[u]);
                len[++cnt] = len[u] + 2;
                fail[cnt] = ch[w][c];
                dep[cnt] = dep[ fail[cnt] ] + 1;
                ch[u][c] = cnt;
            }
            lst = ch[u][c];
        }
    }
} P;

```

5.3. manacher

```

int n,m;
char z[N],s[N<<1];
int p[N<<1];
int R,id;

```

```

int ans;

void SAKI() {
    for(int i=1;i<=n;i++) {
        if(i<=R) p[i] = min(p[id*2-i],R-i+1);
        else p[i] = 1;
        while(s[i+p[i]]==s[i-p[i]]) p[i]++;
        if(i+p[i]-1 > R) R = i+p[i]-1, id = i;
    }
}

int main() {
    scanf("%s",z+1); m = strlen(z+1);
    for(int i=1;i<=m;i++) s[(i<<1)-1] = '#', s[i<<1] = z[i];
    n = m*2 + 1; s[n] = '#'; s[n+1] = '@';
    SAKI();
    cout<<s+1<<"\n";
    for(int i=1;i<=n;i++) cout<<p[i]<<" ";
    return 0;
}

```

5.4. Z 函数

```

#include <bits/stdc++.h>
using namespace std;
const int N=401010;
const int qwq=20003030;
const int inf=0x3f3f3f3f;

int n;
char s[N];
int nxt[N];

void qiu() {
    nxt[0] = n;
    for(int i=1,j=0;i<=n;i++) {
        if(j+nxt[j]>i) nxt[i] = min(nxt[i-j], j+nxt[j]-i);
        while(i+nxt[i]<n && s[nxt[i]]==s[i+nxt[i]]) nxt[i]++;
        if(i+nxt[i]>j+nxt[j]) j = i;
    }
    for(int i=0;i<n;i++) cout<<nxt[i]<<" ";
}

int main() {
    scanf("%s",s); n = strlen(s);
    qiu();
    return 0;
}

/*

abaaaba
7 0 1 1 3 0 1

*/

```

5.5. 广义 SAM (离线)

```
int n,num,cnt=1;
ll ans;
char s[N];
int to[qwq][26],ch[qwq][26];
int fa[qwq],len[qwq];
struct E{
    int id;
    int lst;
    int c;
};

inline void outing() {
    for(int u=1;u<=cnt;u++) {
        cout<<u<<" : fa="<<fa[u]<<" len="<<len[u]-len[fa[u]]<<"\n    ";
        for(int i=0;i<26;i++) {
            if(ch[u][i]) cout<<(char)('a'+i)<<"="<<ch[u][i]<<" ";
        }
        cout<<"\n";
    }
}

inline int insert(int u,int c) {
    int cur = ++cnt; len[cur] = len[u] + 1;
    for(; !ch[u][c] && u; u=fa[u]) ch[u][c] = cur;
    if(!u) fa[cur] = 1;
    else {
        int v = ch[u][c];
        if(len[v]==len[u]+1) fa[cur] = v;
        else {
            int w = ++cnt; len[w] = len[u] + 1;
            memcpy(ch[w], ch[v], sizeof ch[w]);
            fa[w] = fa[v]; fa[v] = fa[cur] = w;
            for(; ch[u][c]==v && u; u=fa[u]) ch[u][c] = w;
        }
    }
    return cur;
}

int main() {
    scanf("%d",&n);
    for(int u=0,k=1;k<=n;k++) {
        scanf("%s",s); int len = strlen(s);
        u = 0;
        for(int i=0;i<len;i++) {
            int c = s[i]-'a';
            if(!to[u][c]) to[u][c] = ++num;
            u = to[u][c];
        }
    }
    queue <E> q;
    for(int i=0;i<26;i++) if(to[0][i]) q.push({to[0][i],1,i});
    while(!q.empty()) {
        E now = q.front(); q.pop();
        int u = now.id;
        int tmp = insert(now.lst,now.c);
    }
}
```

```

        for(int i=0;i<26;i++) if(to[u][i]) q.push({to[u][i],tmp,i});
    }

    outing();
    for(int i=2;i<=cnt;i++) ans += len[i] - len[fa[i]];
    cout<<ans<<endl;
    cout<<cnt;
    return 0;
}

/*

4
aa
ab
bac
caa

*/

```

5.6. kmp

```

#include <bits/stdc++.h>
using namespace std;
const int N=2001010;
const int qwq=1003030;
const int inf=0x3f3f3f3f;

int n;
char s[N];
int nxt[N];

void kmp() {
    for(int i=2,j=0;i<=n;i++) {
        while(j && s[i]!=s[j+1]) {
            // if(j+1<i && nxt[j+1]*2>(j+1) )           // log
            // j=(j-1)%(j-nxt[j])+1;
            // else
            j = nxt[j];
        }
        if(s[i]==s[j+1]) j++;
        nxt[i] = j;

        // zu[i] = nxt[i];
        // if(nxt[i] && (i-nxt[i])==(nxt[i]-nxt[nxt[i]])) zu[i] = zu[nxt[i]];
    }
}

int main() {
    scanf("%s",s+1); n = strlen(s+1);
    kmp();
    for(int i=1;i<=n;i++) cout<<nxt[i]<<" ";
    return 0;
}

/*

```

```
abaaabab
0 0 1 1 1 2 3 2
```

```
*/
```

5.7. SA

```
// by Flandre_495 ---2023.06.26
```

```
const int N=1001010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n,m;
char s[N];
int sa[N],rk[N],tp[N],c[N],h[N];

inline void jipai() {
    for(int i=1;i<=n;i++) c[ rk[i] ]++;
    for(int i=1;i<=m;i++) c[i] += c[i-1];
    for(int i=n;i>=1;i--) sa[ c[rk[tp[i]]]-- ] = tp[i];
    for(int i=1;i<=m;i++) c[i] = 0;
}

void SA() {
    m = 75;
    for(int i=1;i<=n;i++) rk[i] = s[i] - '0' + 1, tp[i] = i;
    jipai();
    for(int k=1; ;k<=1) {
        int p = 0;
        for(int i=1;i<=k;i++) tp[++p] = n-k+i;
        for(int i=1;i<=n;i++) if(sa[i]>k) tp[++p] = sa[i]-k;
        jipai();
        swap(tp,rk); //多测改为 for
        rk[sa[1]] = p = 1;
        for(int i=2;i<=n;i++)
            if(tp[sa[i-1]]==tp[sa[i]] && tp[sa[i-1]+k]==tp[sa[i]+k])
                rk[ sa[i] ] = p;
            else
                rk[ sa[i] ] = ++p;
        m = p;
        if(p==n) break;
    }
    int o;
    for(int i=1,l=0; i<=n; h[rk[i++]] = l)
        for(l=(l?l-1:0),o=sa[rk[i]-1]; s[i+l]==s[o+l]; ++l) ;
}

int main() {
    scanf("%s",s+1); n = strlen(s+1);
    SA();
    for(int i=1;i<=n;i++) cout<<rk[i]<<" ";
    return 0;
}
```

5.8. SAM

```
// by Flandre_495 --2023.06.29
```

```

const int N=1001010;
const int qwq=2003030;
const int inf=0x3f3f3f3f;

int n,cnt=1,lst=1;
char s[N];
int fa[qwq],ch[qwq][26],len[qwq],siz[qwq];
vector<int> e[qwq];

inline int insert(int c) {
    int u = lst, cur = ++cnt; len[cur] = len[u] + 1;
    for(; !ch[u][c] && u; u=fa[u]) ch[u][c] = cur;
    if(!u) fa[cur] = 1;
    else {
        int v = ch[u][c];
        if(len[v]==len[u]+1) fa[cur] = v;
        else {
            int w = ++cnt; len[w] = len[u] + 1;
            memcpy(ch[w],ch[v],sizeof ch[w]);
            fa[w] = fa[v]; fa[v] = fa[cur] = w;
            for(; ch[u][c]==v && u; u=fa[u]) ch[u][c] = w;
        }
    }
    return cur;
}

void DFS(int u) {
    FOR() {
        int v = e[u][i];
        DFS(v);
        siz[u] += siz[v];
    }
}

int main() {
    scanf("%s",s+1); n = strlen(s+1);
    for(int i=1;i<=n;i++) lst = insert(s[i]-'a'), siz[lst]=1;

    for(int i=2;i<=cnt;i++) e[ fa[i] ].push_back(i);
    DFS(1);

    ll ans = 0;
    for(int i=2;i<=cnt;i++) if(siz[i]>1) ans = max( ans, (ll)siz[i]*len[i] );
    cout<<ans;
    return 0;
}

/*
aababaaba
*/

```

5.9. 广义 SAM (在线)

```
const int N=2010101;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int nn,n,m,lst=1,cnt=1;
int cl[N];
char s[N],z[N];
int fa[N],ch[N][26],len[N];
ll ans[N];
vector<int> g[N];

inline void outing() {
    for(int u=1;u<=cnt;u++) {
        cout<<u<<" : fa="<<fa[u]<<" len="
            <<len[u]-len[fa[u]]<<" siz = "<<g[u].size()<<"\n    ";
        for(int i=0;i<26;i++) {
            if(ch[u][i]) cout<<(char)('a'+i)<<"="<<ch[u][i]<<" ";
        }
        cout<<"\n";
    }
}

inline int insert(int c) {
    int u = lst;
    if(ch[u][c]) {
        int v = ch[u][c];
        if(len[v]==len[u]+1) return v;
        int w = ++cnt; len[w] = len[u] + 1;
        memcpy(ch[w],ch[v],sizeof ch[v]);
        fa[w] = fa[v]; fa[v] = w;
        for(; ch[u][c]==v && u; u=fa[u]) ch[u][c] = w;
        return w;
    }
    int cur = ++cnt; len[cur] = len[u] + 1;
    for(; !ch[u][c] && u; u=fa[u]) ch[u][c] = cur;
    if(!u) fa[cur] = 1;
    else {
        int v = ch[u][c];
        if(len[v]==len[u]+1) fa[cur] = v;
        else {
            int w = ++cnt; len[w] = len[u] + 1;
            memcpy(ch[w],ch[v],sizeof ch[v]);
            fa[w] = fa[v]; fa[v] = fa[cur] = w;
            for(; ch[u][c]==v && u; u=fa[u]) ch[u][c] = w;
        }
    }
    return cur;
}

int main() {
    scanf("%d",&n);
    for(int i=1;i<=n;i++) {
        scanf("%s",s); int le = strlen(s); lst = 1;
        for(int j=0;j<le;j++) lst = insert(s[j]-'a');
    }
}
```

```

    outing();
    for(int i=2;i<=cnt;i++) ans += len[i] - len[fa[i]];
    cout<<ans<<endl<<cnt;
    return 0;
}

```

5.10. AC 自动机

```

const int N=101010;
const int qwq=1003030;
const int inf=0x3f3f3f3f;

int n,m;
int cnt;
char s[qwq];
struct T{
    int to[26];
    int fail;
}f[qwq];
vector<int> e[qwq];
vector<int> g[qwq];
bool vis[qwq];
int ans;

void chushihua() {
    for(int i=0;i<=cnt;i++) e[i].clear(), g[i].clear(); //记得从零开始
    cnt = 0;
    memset( f,0,sizeof(f) );
}

inline void AC() {
    queue<int> q; q.push(0);
    while(!q.empty()) {
        int u = q.front(); q.pop();
        for(int i=0;i<26;i++) {
            int v = f[u].to[i];
            if(v) {
                if(u) f[v].fail = f[ f[u].fail ].to[i];
                q.push(v);
            }
            else f[u].to[i] = f[ f[u].fail ].to[i];
        }
    }
    for(int i=1;i<=cnt;i++) e[ f[i].fail ].push_back(i);
    return ;
}

int main() {
    n = read();
    for(int k=1;k<=n;k++) {
        scanf("%s",s); int len = strlen(s);
        int u = 0;
        for(int i=0;i<len;i++) {
            int v = s[i]-'a';
            if(!f[u].to[v]) f[u].to[v] = ++cnt;
            u = f[u].to[v];
        }
    }
}

```

```

    }
    g[u].push_back(k);
}
AC();
for(int i=1;i<=cnt;i++) cout<<"i = "<<i<<" fail = "
    <<f[i].fail<<" a = "<<f[i].to[0]<<" b = "<<f[i].to[1]<<"\n";

    return 0;
}

```

5.11. Lyndon 分解

```

#include <bits/stdc++.h>
#define ll long long

using namespace std;
const int N=5050505;

char s[N];
int n;
int ans;

int main() {
    scanf("%s",s+1); n = strlen(s+1);
    for(int i=1;i<=n;) {
        int j=i, k=i+1;
        while(k<=n && s[j]<=s[k]) {
            if(s[j]<s[k]) j = i;
            else j++;
            k++;
        }
        while(i<=j) {
            ans ^= i+k-j-1;
            i += k-j;
        }
    }
    cout<<ans;
    return 0;
}

/*
bbababaabaaabaaaab

*/

```

5.12. SA+ST

```

const ll N=2010101;
const ll qwq=N*3;
const ll inf=0x3f3f3f3f3f3f3f3f;

int n,Q;
int ob[N];

struct SAST{
    int m;

```

```

char s[N];
int sa[N],rk[N],tp[N],c[N],h[N];
int f[N][22];

void Qsort() {
    memset(c,0,sizeof(c));
    for(int i=1;i<=n;i++) c[rk[i]]++;
    for(int i=1;i<=m;i++) c[i] += c[i-1];
    for(int i=n;i>=1;i--) sa[ c[rk[tp[i]]]-- ] = tp[i];
}

void SA() {
    m = 75;
    for(int i=1;i<=n;i++) rk[i] = s[i] - 'a' + 1, tp[i] = i;
    Qsort();
    for(int k = 1; ;k <= 1) {
        int p = 0;
        for(int i=1;i<=k;i++) tp[++p] = n - k + i;
        for(int i=1;i<=n;i++) if(sa[i] > k) tp[++p] = sa[i] - k;
        Qsort();
        swap(tp,rk);
        rk[sa[1]] = p = 1;
        for(int i=2;i<=n;i++) {
            if(tp[sa[i-1]]==tp[sa[i]] && tp[sa[i-1]+k]==tp[sa[i]+k])
                rk[sa[i]] = p;
            else
                rk[sa[i]] = ++p;
        }
        m = p;
        if(p==n) break;
    }
    int b;
    for(int i=1,l=0;i<=n;h[rk[i++]]+=l)
        for(l=(l?l-1:0),b=sa[rk[i]-1];s[i+l]==s[b+l];++l);

    for(int i=1;i<=n;i++) f[i][0] = h[i];
    for(int k=1;k<=18;k++)
        for(int i=1;i+(1<<k)-1<=n;i++)
            f[i][k] = min(f[i][k-1],f[ i+(1<<(k-1)) ][k-1]);
}

inline int LCP(int i,int j) {
    if(i<1 || j<1 || i>n || j>n) return 0;
    if(i==j) return n-i+1;
    i = rk[i]; j = rk[j];
    if(i>j) swap(i,j);
    i++;
    int k = ob[j-i+1];
    return min(f[i][k],f[j-(1<<k)+1][k]);
}

}zheng,dao;

int main() {
    for(int i=2;i<=N-1000;i++) ob[i] = ob[i>>1] + 1;

```

```

n = read(); Q = read();
scanf("%s", zheng.s+1);
zheng.SA();

int x,y;
while(Q--) {
    x = read(); y = read();
    cout<<zheng.LCP(x,y)<<"\n";
}
return 0;
}

```

```
/*
```

```
*/
```

6. 多项式

6.1. NTT

```

const ll N=3001010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118;

ll n,m;
ll r[N];
ll F[N],G[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<=1));
        for(ll j=0;j<len;j+=(k<=1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++,gk=gk*g1%p) {
                ll x = A[i+j], y = gk*A[i+j+k] %p;
                A[i+j] = (x+y) %p; A[i+j+k] = (x-y+p) %p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(int i=0;i<len;i++) A[i] = A[i] * inv %p;
}

```

```

}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    ll len = init(n+m+1);
    NTT(F,len,1);
    NTT(G,len,1);
    for(ll i=0;i<len;i++) F[i] = F[i] * G[i] %p;
    NTT(F,len,-1);
}

int main() {
    n = read(); m = read();
    for(ll i=0;i<=n;i++) F[i] = read();
    for(ll i=0;i<=m;i++) G[i] = read();
    PMUL(F, G, n, m);
    for(ll i=0;i<=n+m;i++) cout<<F[i]<<" ";
    return 0;
}

```

6.2. 多项式分治乘

```

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118;

ll n,m;
ll rev[N],siz[N];
ll a[N],b[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<rev[i]) swap(A[i],A[rev[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++,gk=gk*g1%p) {
                ll x = A[i+j], y = gk*A[i+j+k] %p;
                A[i+j] = (x+y) %p; A[i+j+k] = (x-y+p) %p;
            }
        }
    }
}

```

```

    }
}
if(cl==1) return ;
ll inv = ksm(len,p-2);
for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

void solve(ll now,ll l,ll r,ll *F,ll *G) {
    siz[now] = r-l+1;
    if(l==r) {
        G[0] = F[l]; G[1] = 1;    // a[i] + x
        return ;
    }
    ll len = 1, L = 0; while(len<=siz[now]) len<=1, L++;
    for(int i=0;i<len;i++) G[i] = 0;
    ll A[len], B[len], mid = l+r >> 1;
    solve(ls, l, mid, F, A);
    solve(rs, mid+1, r, F, B);
    if(r-l<32) {
        for(ll i=0;i<=siz[ls];i++)
            for(ll j=0;j<=siz[rs];j++)
                G[i+j] = (G[i+j] + A[i] * B[j] %p) %p;
        return ;
    }
    for(int i=siz[ls]+1;i<len;i++) A[i] = 0;
    for(int i=siz[rs]+1;i<len;i++) B[i] = 0;
    for(ll i=0;i<len;i++) rev[i] = (rev[i>>1]>>1) | ((i&1)<<(L-1));
    NTT(A, len, 1);
    NTT(B, len, 1);
    for(ll i=0;i<len;i++) G[i] = A[i] * B[i] %p;
    NTT(G, len, -1);
}

int main() {
    n = read();
    for(ll i=1;i<=n;i++) a[i] = read();
    solve(1, 1, n, a, b);
    for(ll i=0;i<=n;i++) cout<<b[i]<<" ";
    return 0;
}

/*
10
10 9 8 7 6 5 4 3 2 1

3628800 10628640 12753576 8409500 3416930 902055 157773 18150 1320 55 1

*/

```

6.3. 多项式 vector

```

#include <bits/stdc++.h>
#define ll long long
#define poly vector<ll>

```

```

using namespace std;
const ll N=301010;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118, inv2=499122177;

ll r[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

inline int init(int wo) {
    ll len = 1; while(len<wo) len<<=1;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)?len>>1:0);
    return len;
}

void NTT(poly &A, ll len, ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++, (gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(int i=0;i<len;i++) A[i] = A[i] * inv %p;
}

poly PMUL(poly A,poly B,ll mod=-1) {
    int deg = A.size() + B.size() - 1;
    if(A.size()<=32 || B.size()<=32) {
        poly C(deg, 0);
        for(int i=0;i<A.size();i++)
            for(int j=0;j<B.size();j++)
                (C[i+j] += A[i] * B[j] %p) %= p;
        return C;
    }
    ll len = init(deg);
    A.resize(len); B.resize(len);
    NTT(A, len, 1);
    NTT(B, len, 1);
    for(int i=0;i<len;i++) A[i] = A[i] * B[i] %p;
    NTT(A, len, -1);
}

```

```

    if(mod==1) A.resize(deg);
    else      A.resize(mod);
    return A;
}

poly PI(poly A,int deg=-1) {
    if(deg==1) deg = A.size();
    poly B(1, ksm(A[0],p-2)), C;
    for(int k=2; (k>>1)<deg; k<=1) {
        C.resize(k);
        ll len = init(k<=1);
        for(int i=0;i<k;i++) C[i] = i<(int)A.size() ? A[i] : 0;
        C.resize(len);
        B.resize(len);
        NTT(C, len, 1);
        NTT(B, len, 1);
        for(int i=0;i<len;i++) B[i] = (2ll - C[i]*B[i] %p + p) %p * B[i] %p;
        NTT(B, len, -1);
        B.resize(k);
    }
    B.resize(deg);
    return B;
}

poly Sqrt(poly A,int deg=-1) { // a[0] = 1
    if(deg==1) deg = A.size();
    poly B(1,1), C, D;
    ll len = 0;
    for(int k=4; (k>>2)<deg; k<=1) {
        C = A;
        C.resize(k>>1);
        len = init(k);
        D = PI(B,k>>1);
        C.resize(len);
        D.resize(len);
        NTT(C, len, 1);
        NTT(D, len, 1);
        for(int i=0;i<len;i++) C[i] = C[i] * D[i] %p;
        NTT(C, len, -1);
        B.resize(k>>1);
        for(int i=0;i<(k>>1);i++) B[i] = (B[i] + C[i]) %p * inv2 %p;
    }
    B.resize(len);
    return B;
}

poly PDIV(poly A,poly B) {
    int deg = A.size() - B.size() + 1;
    reverse(A.begin(), A.end());
    reverse(B.begin(), B.end());
    A.resize(deg);
    A = PMUL(A, PI(B,deg));
    A.resize(deg);
    reverse(A.begin(), A.end());
    return A;
}

```

```

poly PMOD(poly A,poly B) {
    if(A.size() < B.size()) return A;
    poly C = PMUL(B, PDIV(A, B));
    for(int i=0;i<A.size();i++) A[i] = (A[i] - C[i] + p) %p;
    A.resize(B.size()-1);
    return A;
}

poly Pksm2(poly A,ll K,ll mod) { // log^2
    poly B(1,1);
    while(K) {
        if(K&1) B = PMUL(B, A, mod);
        K >>= 1; A = PMUL(A, A, mod);
    }
    return B;
}

poly Pdao(poly A) { for(int i=0;i+1<A.size();i++) A[i]=A[i+1]*(i+1)%p; A.pop_back();
return A; }
poly Pji(poly A) { for(int i=A.size()-1;i;i--) A[i]=A[i-1]*ksm(i,p-2)%p; A[0]=0;
return A; }
poly Pln(poly A,ll mod=-1) {
    if(mod==-1) mod = A.size();
    A = Pji( PMUL( Pdao(A), PI(A,mod) ) );
    A.resize(mod);
    return A;
}
poly Pexp(poly A,ll mod=-1) {
    if(mod==-1) mod = A.size();
    poly B(1,1), C;
    for(int k=2;(k>>1)<mod;k<=1) {
        C = Pln(B,k);
        for(int i=0;i<k;i++) C[i] = ((i<(int)A.size()?A[i]:0) - C[i] + p) %p;
        C[0]++;
        B = PMUL(B,C);
        B.resize(k);
    }
    B.resize(mod);
    return B;
}

poly Pksm(poly A,ll K,ll mod=-1) {
    if(mod==-1) mod = A.size();
    poly B = Pln(A);
    for(int i=0;i<mod;i++) B[i] = B[i] * K %p;
    A = Pexp(B);
    A.resize(mod);
    return A;
}

ll n,m,k;
poly a, b, e[N];

poly calc(int L,int R) {
    if(L==R) return e[L];

```

```

    int mid = L+R >> 1;
    poly A = calc(L,mid), B = calc(mid+1,R);
    return PMUL(A,B);
}

```

```

int main() {

    return 0;
}

```

6.4. 多项式求逆

```

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118;

ll n,m;
ll r[N];
ll a[N],b[N],c1[N],c2[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++, (gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

void PI(ll *F,ll *G,ll n) { // F*G = 1 (mod x^n)
    G[0] = ksm(F[0],p-2);
    ll *A=c1, *B=c2, k=1;
    for(ll len,L=1;k<(n+n);k<=1,L++) {
        len = k<<1;
        for(ll i=0;i<k;i++) A[i] = F[i], B[i] = G[i];
        for(ll i=0;i<len;i++) r[i] = (r[i]>>1)>>1 | ((i&1)<<(L-1));
        NTT(A, len, 1);
        NTT(B, len, 1);
        for(ll i=0;i<len;i++) G[i] = (2ll - A[i] * B[i] %p + p) %p * B[i] %p;
    }
}

```

```

        NTT(G, len, -1);
        for(ll i=k;i<len;i++) G[i] = 0;
    }
    for(ll i=0;i<k;i++) A[i] = B[i] = 0;
    for(ll i=n;i<k;i++) G[i] = 0;
}

int main() {
    n = read();
    for(ll i=0;i<n;i++) a[i] = read();
    PI(a, b, n);
    for(ll i=0;i<n;i++) cout<<b[i]<<" ";
    return 0;
}

/*

5
1 6 3 4 9

1 998244347 33 998244169 1020

*/

```

6.5. 多项式快速幂

```

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118;

ll n,m,K;
ll r[N];
ll a[N],b[N];
ll c1[N],c2[N],c3[N],c4[N],c5[N],c6[N],c7[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++, (gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
}

```

```

    }
}
if(cl==1) return ;
ll inv = ksm(len,p-2);
for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    ll len = init(n+m+1);
    NTT(F,len,1);
    NTT(G,len,1);
    for(ll i=0;i<len;i++) F[i] = F[i] * G[i] %p;
    NTT(F,len,-1);
}

void PI(ll *F,ll *G,ll n) { // F*G = 1 (mod x^n)
    G[0] = ksm(F[0],p-2);
    ll *A=c1, *B=c2, k=1;
    for(ll len,L=1;k<(n+n);k<=1,L++) {
        len = k<<1;
        for(ll i=0;i<k;i++) A[i] = F[i], B[i] = G[i];
        for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
        NTT(A, len, 1);
        NTT(B, len, 1);
        for(ll i=0;i<len;i++) G[i] = (2ll - A[i] * B[i] %p + p) %p * B[i] %p;
        NTT(G, len, -1);
        for(ll i=k;i<len;i++) G[i] = 0;
    }
    for(ll i=0;i<k;i++) A[i] = B[i] = 0;
    for(ll i=n;i<k;i++) G[i] = 0;
}

void Pdao(ll *F,ll *G,ll n) {
    for(ll i=1;i<n;i++) G[i-1] = i*F[i] %p; G[n-1] = 0;
}

void Pji(ll *F,ll *G,ll n) {
    for(ll i=1;i<n;i++) G[i] = ksm(i,p-2)*F[i-1] %p; G[0] = 0;
}

void Pln(ll *F,ll *G,ll n) { // G = ln(F) (mod x^n)
    ll *A=c3, *B=c4;
    Pdao(F, A, n);
    PI(F, B, n);
    PMUL(A, B, n, n);
    Pji(A, G, n);
}

void Pexp(ll *F,ll *G,ll n) { // G = exp(F) (mod x^n)
    if(n==1) { G[0] = 1; return ; }

```

```

Pexp(F, G, (n+1)>>1);
ll *A=c5, *B=c6;
for(ll i=0;i<=(n<<1);i++) A[i] = B[i] = 0;
Pln(G, A, n);
ll len = init(n+n);
for(ll i=0;i<n;i++) B[i] = F[i];
NTT(G, len, 1);
NTT(A, len, 1);
NTT(B, len, 1);
for(ll i=0;i<len;i++) G[i] = (1ll-A[i]+B[i]+p) %p *G[i] %p;
NTT(G, len, -1);
for(ll i=n;i<len;i++) G[i] = 0;
}

void Pksm(ll *F,ll *G,ll K,ll n) { // G = F^K (mod x^n)
    int wei = 0, a0 = 0;
    for(int i=0;i<=n;i++) {
        if(i==n) return ;
        if(F[i]) { wei = i; a0 = F[i]; break; }
    }
    int inv = ksm(a0,p-2);
    for(int i=0;i<n-wei;i++) F[i] = F[i+wei] * inv %p;
    /*----- a0 = 1 -----*/
    ll *A=c7;
    Pln(F, A, n);
    memset(c3,0,sizeof(c3));
    memset(c4,0,sizeof(c4));
    for(int i=0;i<n;i++) A[i] = A[i] * K %p;
    Pexp(A, G, n);
    /*-----*/
    int mi = ksm(a0,K);
    for(int i=n-1;i>=K*wei;i--) G[i] = G[i-K*wei] * mi %p;
    for(int i=0;i<K*wei;i++) G[i] = 0;
}

int main() {
    n = read(); K = read();
    for(ll i=0;i<n;i++) a[i] = read();
    Pksm(a, b, K, n);
    for(ll i=0;i<n;i++) cout<<b[i]<<" ";
    return 0;
}

/*
9 18948465
1 2 3 4 5 6 7 8 9

1 37896930 597086012 720637306 161940419 360472177 560327751 446560856 524295016

*/

```

6.6. 多项式开根

```

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;

```

```

const ll p=998244353, g=3, gi=332748118, inv2=499122177;

ll n,m;
ll r[N];
ll a[N],b[N],c1[N],c2[N],c3[N],c4[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++, (gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

void PI(ll *F,ll *G,ll n) { // F*G = 1 (mod x^n)
    G[0] = ksm(F[0],p-2);
    ll *A=c1, *B=c2, k=1;
    for(ll len,L=1;k<(n+n);k<=1,L++) {
        len = k<<1;
        for(ll i=0;i<k;i++) A[i] = F[i], B[i] = G[i];
        for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
        NTT(A, len, 1);
        NTT(B, len, 1);
        for(ll i=0;i<len;i++) G[i] = (2ll - A[i] * B[i] %p + p) %p * B[i] %p;
        NTT(G, len, -1);
        for(ll i=k;i<len;i++) G[i] = 0;
    }
    for(ll i=0;i<k;i++) A[i] = B[i] = 0;
    for(ll i=n;i<k;i++) G[i] = 0;
}

void Sqrt(ll *F,ll *G,ll n) { // G*G = F (mod x^n)
    G[0] = 1;
    ll *A=c3, *B=c4, k=1;
    for(ll len,L=1;k<(n+n);k<=1,L++) {
        len = k<<1;
        for(ll i=0;i<k;i++) A[i] = F[i];
    }

```

```

    PI(G, B, k);
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    NTT(A, len, 1);
    NTT(B, len, 1);
    for(ll i=0;i<len;i++) A[i] = A[i] * B[i] %p;
    NTT(A, len, -1);
    for(ll i=0;i<k;i++) G[i] = (G[i] + A[i]) %p * inv2 %p;
    for(ll i=k;i<len;i++) G[i] = 0;
}
for(ll i=0;i<k;i++) A[i] = B[i] = 0;
for(ll i=n;i<k;i++) G[i] = 0;
}

int main() {
    n = read();
    for(ll i=0;i<n;i++) a[i] = read();
    Sqrt(a, b, n);
    for(ll i=0;i<n;i++) cout<<b[i]<<" ";
    return 0;
}

/*

7
1 8596489 489489 4894 1564 489 35789489

1 503420421 924499237 13354513 217017417 707895465 411020414

*/

```

6.7. 多项式快速幂 (双 log)

```

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118;

inline ll read() {
    ll sum = 0, ff = 1; char c = getchar();
    while(c<'0' || c>'9') { if(c=='-') ff = -1; c = getchar(); }
    while(c>='0'&&c<='9') { sum = (sum * 10 + c - '0') %p; c = getchar(); }
    return sum * ff;
}

ll n,m,K;
ll r[N];
ll a[N],b[N];
ll c1[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

```

```

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<=1));
        for(ll j=0;j<len;j+=(k<=1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++, (gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

void Pksm(ll *F,ll *G,ll K,ll n) { // G = F^K (mod x^n)
    ll len = 1, L = 0; while(len-1<n+n+2) len<=1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    G[0] = 1;
    while(K) {
        NTT(F, len, 1);
        if(K&1) {
            NTT(G, len, 1);
            for(int i=0;i<len;i++) G[i] = G[i] * F[i] %p;
            NTT(G, len, -1);
            for(int i=n;i<len;i++) G[i] = 0;
        }
        K >>= 1;
        for(int i=0;i<len;i++) F[i] = F[i] * F[i] %p;
        NTT(F, len, -1);
        for(int i=n;i<len;i++) F[i] = 0;
    }
}

int main() {
    n = read(); K = read();
    for(ll i=0;i<n;i++) a[i] = read();
    Pksm(a, b, K, n);
    for(ll i=0;i<n;i++) cout<<b[i]<<" ";
    return 0;
}

/*
9 18948465
1 2 3 4 5 6 7 8 9

1 37896930 597086012 720637306 161940419 360472177 560327751 446560856 524295016

*/

```

6.8. 01 序列生成函数 (小武)

```
#include<stdio.h>
#define rint register int

typedef long long ll;

const int N=(1<<22)+7;
const int Mod=998244353;
const int G=3;

ll qpow(ll x,int y=Mod-2){
    ll ret=1;
    while(y){
        if(y&1) ret=ret*x%Mod;
        x=x*x%Mod,y>>=1;
    }
    return ret;
}

const int Gi=qpow(G);

int rk[N];
inline void swap(ll &x,ll &y){x^=y,y^=x,x^=y;}
void NTT(bool op,int n,ll *F){
    for(rint i=0;i<n;i++)
        if(i<rk[i]) swap(F[i],F[rk[i]]);
    for(rint p=2;p<=n;p<=1){
        rint len=p>>1;
        ll w=qpow(op? G:Gi,(Mod-1)/p);
        for(rint k=0;k<n;k+=p){
            ll now=1;
            for(rint l=k;l<k+len;l++){
                ll t=F[l+len]*now%Mod;
                F[l+len]=(F[l]-t+Mod)%Mod;
                F[l]=(F[l]+t)%Mod;
                now=now*w%Mod;
            }
        }
    }
}

inline void Cop(int n,ll *a,ll *b){for(int i=0;i<n;i++)a[i]=b[i];}
inline void Clear(int n,ll *F){for(int i=0;i<n;i++)F[i]=0;}
inline void Rk(int n){for(int i=0;i<n;i++)rk[i]=(rk[i>>1]>>1)|(i&1? n>>1:0);}

void Mul(ll *X,int n,int m,ll *a,ll *b){
    static ll x[N],y[N];
    Cop(n+1,x,a),Cop(m+1,y,b);
    for(m+=n,n=1;n<=m;n<=1); Rk(n);
    NTT(1,n,x),NTT(1,n,y);
    for(rint i=0;i<n;i++) x[i]=x[i]*y[i]%Mod;
    NTT(0,n,x); ll inv=qpow(n);
    for(rint i=0;i<=m;i++) X[i]=x[i]*inv%Mod;
    Clear(n,x),Clear(n,y);
}
```

```

void Inv(int n,ll *a,ll *b){
    static ll x[N];
    if(n==1){b[0]=qpow(a[0]);return;}
    Inv((n+1)>>1,a,b); int m=n;
    for(n=1;n<(m<<1);n<=1); Rk(n);
    Clear(n,x),Cop(m,x,a);
    NTT(1,n,x),NTT(1,n,b);
    for(rint i=0;i<n;i++)
        b[i]=b[i]*(2-x[i]*b[i]%Mod+Mod)%Mod;
    NTT(0,n,b); ll inv=qpow(n);
    for(rint i=0;i<m;i++) b[i]=b[i]*inv%Mod;
    for(rint i=m;i<n;i++) b[i]=0;
}

ll a[N],b[N],c[N];

ll calc(int n,int K){
    Clear(n+3,a),Clear(n+3,b);
    a[0]=1,a[1]=Mod-2,a[K+2]=1;
    Inv(n+3,a,b);
    a[0]=1,a[1]=Mod-1,a[K+2]=0;
    Mul(c,n+3,n+3,a,b);
    // printf("[%d %d]\n",n,K);
    // for(int i=1;i<=n+1;i++) printf("%lld ",c[i]);
    // printf("\n");
    return c[n+1];
}

int main(){
    int n=read(),K=read();
    // printf("%lld\n",calc(n,K));
    printf("%lld", (calc(n,K)-calc(n,K-1)+Mod)%Mod);
}

```

6.9. 第一类斯特林数（行）

```

#include <algorithm>
#include <iostream>
#include <cstring>
#include <cstdio>
#include <cmath>
#define FOR() ll le=e[u].size();for(ll i=0;i<le;i++)
#define QWQ cout<<"QwQ\n";
#define ll long long
#include <vector>
#include <queue>
#include <map>

using namespace std;
const ll N=801010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=167772161, g=3, gi=55924054;

inline ll read() {
    ll sum = 0, ff = 1; char c = getchar();
    while(c<'0' || c>'9') { if(c=='-') ff = -1; c = getchar(); }
}

```

```

    while(c>='0'&&c<='9') { sum = sum * 10 + c - '0'; c = getchar(); }
    return sum * ff;
}

ll f[N],ni[N];
ll rev[N];
ll a[N];
ll c1[N],c2[N],c3[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void qiu() {
    f[0] = ni[0] = 1;
    for(ll i=1;i<=N-10;i++) f[i] = f[i-1] * i %p;
    ni[N-10] = ksm(f[N-10],p-2);
    for(ll i=N-11;i>=1;i--) ni[i] = ni[i+1] * (i+1) %p;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<rev[i]) swap(A[i],A[rev[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++,gk=gk*g1%p) {
                ll x = A[i+j], y = gk*A[i+j+k] %p;
                A[i+j] = (x+y) %p; A[i+j+k] = (x-y+p) %p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(int i=0;i<len;i++) A[i] = A[i] * inv %p;
}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=1, L++;
    for(ll i=0;i<len;i++) rev[i] = (rev[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    ll len = init(n+m+1);
    NTT(F,len,1);
    NTT(G,len,1);
    for(ll i=0;i<len;i++) F[i] = F[i] * G[i] %p;
    NTT(F,len,-1);
}

```

```

void solve(ll *F, ll m) {
    if(m==1) { F[1] = 1; return ; }
    if(m&1) {
        solve(F, m-1);
        for(ll i=m;i>=1;i--) F[i] = (F[i-1] + F[i] * (m-1) %p) %p;
        F[0] = F[0] * (m-1) %p;
    }
    else {
        ll n = m/2;
        ll res = 1, *A = c1, *B = c2, *G = c3;
        solve(F, n);
        for(ll i=0;i<=n;i++) {
            A[i] = F[i] * f[i] %p;
            B[i] = res * ni[i] %p;
            res = res * n %p;
        }
        reverse(A, A+n+1);
        PMUL(A, B, n, n);
        for(ll i=0;i<=n;i++) G[i] = ni[i] * A[n-i] %p;
        PMUL(F, G, n, n);
        ll len = 1; while(len < (n+1)<<1) len <<= 1;
        for(ll i=n+1;i<len;i++) A[i] = B[i] = G[i] = 0;
        for(ll i=m+1;i<len;i++) F[i] = 0;
    }
}

int main() {
    qiu();
    ll n;
    n = read();
    solve(a, n);
    for(ll i=0;i<=n;i++) cout<<(a[i]%p+p)%p<<" ";
    return 0;
}

```

6.10. 三模数 NTT

```

const ll N=301010;
const ll inf=0x3f3f3f3f;
const ll p1=469762049, p2=998244353, p3=1004535809, g=3;
const ll M = p1 * p2;
typedef long double db;

ll n,m,P;
ll r[N];
ll a[N],b[N];
ll c1[N],c2[N];
ll A1[N],A2[N],A3[N];

inline ll ksm(ll aa,ll bb,ll p) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

```

```

inline ll gsc(ll aa,ll bb,ll p) {
    aa %= p; bb %= p;
    return (( aa * bb - (ll)( (ll)((db)aa / p * bb + 1e-3) * p)) %p + p) %p;
}

void NTT(ll *A,ll len,ll cl,ll p) {
    ll gi = ksm(3,p-2,p);
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<=1), p);
        for(ll j=0;j<len;j+=(k<=1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++,gk=gk*g1%p) {
                ll x = A[i+j], y = gk*A[i+j+k] %p;
                A[i+j] = (x+y) %p; A[i+j+k] = (x-y+p) %p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2, p);
    for(int i=0;i<len;i++) A[i] = A[i] * inv %p;
}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    ll len = init(n+m+1);
    ll *A = c1, *B = c2;

    for(int i=0;i<len;i++) A[i] = (i<=n) ? F[i] : 0, B[i] = (i<=m) ? G[i] : 0;
    NTT(A,len,1,p1); NTT(B,len,1,p1);
    for(ll i=0;i<len;i++) A1[i] = A[i] * B[i] %p1;
    NTT(A1,len,-1,p1);

    for(int i=0;i<len;i++) A[i] = (i<=n) ? F[i] : 0, B[i] = (i<=m) ? G[i] : 0;
    NTT(A,len,1,p2); NTT(B,len,1,p2);
    for(ll i=0;i<len;i++) A2[i] = A[i] * B[i] %p2;
    NTT(A2,len,-1,p2);

    for(int i=0;i<len;i++) A[i] = (i<=n) ? F[i] : 0, B[i] = (i<=m) ? G[i] : 0;
    NTT(A,len,1,p3); NTT(B,len,1,p3);
    for(ll i=0;i<len;i++) A3[i] = A[i] * B[i] %p3;
    NTT(A3,len,-1,p3);

    for(int i=0;i<=n+m;i++) {
        ll wo = (gsc(A1[i]*p2%M, ksm(p2%p1, p1-2, p1), M) +
                gsc(A2[i]*p1%M, ksm(p1%p2, p2-2, p2), M)) %M;
        ll K = ((A3[i]-wo) % p3 + p3) %p3 * ksm(M%p3, p3-2, p3) %p3;
        F[i] = ((K%p) * (M%p) %P + wo%p) %P;
    }
}

```

```

int main() {
    n = read(); m = read(); P = read();
    for(ll i=0;i<=n;i++) a[i] = read();
    for(ll i=0;i<=m;i++) b[i] = read();
    PMUL(a, b, n, m);
    for(ll i=0;i<=n+m;i++) cout<<a[i]<<" ";
    return 0;
}

```

6.11. 多项式除法

```

const ll N=501010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=998244353, g=3, gi=332748118, inv2=499122177;

ll n,m;
ll r[N];
ll a[N],b[N];
ll c1[N],c2[N],c3[N];
ll Q[N],R[N];
ll Fr[N],Gri[N],Gr[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=<1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<=<1));
        for(ll j=0;j<len;j+=(k<=<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++, (gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

void PI(ll *F,ll *G,ll n) { // F*G = 1 (mod x^n)
    G[0] = ksm(F[0],p-2);
    ll *A=c1, *B=c2, k=1;
    for(ll len,L=1;k<(n+n);k<=<1,L++) {
        len = k<=<1;

```

```

        for(ll i=0;i<k;i++) A[i] = F[i], B[i] = G[i];
        for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
        NTT(A, len, 1);
        NTT(B, len, 1);
        for(ll i=0;i<len;i++) G[i] = (2ll - A[i] * B[i] %p + p) %p * B[i] %p;
        NTT(G, len, -1);
        for(ll i=k;i<len;i++) G[i] = 0;
    }
    for(ll i=0;i<k;i++) A[i] = B[i] = 0;
    for(ll i=n;i<k;i++) G[i] = 0;
}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<<=1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    ll len = init(n+m+1);
    NTT(F,len,1);
    NTT(G,len,1);
    for(ll i=0;i<len;i++) F[i] = F[i] * G[i] %p;
    NTT(F,len,-1);
}

void PDIV(ll *F,ll *G,ll *Q,ll *R,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    for(ll i=0;i<=n;i++) Fr[n-i] = F[i];
    for(ll i=0;i<=m;i++) Gr[m-i] = G[i];
    for(ll i=n-m+2;i<=m;i++) Gr[i] = 0;
    PI(Gr,Gri,n-m+1);
    PMUL(Fr,Gri,n,n-m);
    for(ll i=0;i<=n-m;i++) Q[n-m-i] = Fr[i];
    ll *A = c3;
    for(ll i=0;i<=n-m;i++) A[i] = Q[i];
    PMUL(G,A,m,n-m);
    for(ll i=0;i<m;i++) R[i] = (F[i]-G[i]+p)%p;
}

int main() {
    n = read(); m = read();
    for(ll i=0;i<=n;i++) a[i] = read();
    for(ll i=0;i<=m;i++) b[i] = read();
    PDIV(a,b,Q,R,n,m);
    for(ll i=0;i<=n-m;i++) cout<<Q[i]<<" ";
    cout<<endl;
    for(ll i=0;i<m;i++) cout<<R[i]<<" ";
    return 0;
}

/*
5 1
1 9 2 6 0 8
1 7

```

```
237340659 335104102 649004347 448191342 855638018
760903695
```

```
*/
```

6.12. 第二类斯特林数（行）

```
const ll N=801010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=167772161, g=3, gi=55924054;

ll n,m;
ll r[N];
ll a[N],b[N];
ll f[N],ni[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void qiu() {
    f[0] = ni[0] = 1;
    for(int i=1;i<=N-10;i++) f[i] = f[i-1] * i %p;
    ni[N-10] = ksm(f[N-10],p-2);
    for(int i=N-11;i>=1;i--) ni[i] = ni[i+1] * (i+1) %p;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=<1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<=<1));
        for(ll j=0;j<len;j+=(k<=<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++,gk=gk*g1%p) {
                ll x = A[i+j], y = gk*A[i+j+k] %p;
                A[i+j] = (x+y) %p; A[i+j+k] = (x-y+p) %p;
            }
        }
    }
    if(cl==1) return ;
    ll inv = ksm(len,p-2);
    for(int i=0;i<len;i++) A[i] = A[i] * inv %p;
}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=<1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
```

```

    ll len = init(n+m+1);
    NTT(F,len,1);
    NTT(G,len,1);
    for(ll i=0;i<len;i++) F[i] = F[i] * G[i] %p;
    NTT(F,len,-1);
}

```

```

int main() {
    qiu();
    n = read();
    for(int i=0;i<=n;i++) {
        a[i] = (p+((i&1)?-1:1) * ni[i]) %p;
        b[i] = ksm(i,n) * ni[i] %p;
    }
    PMUL(a,b,n,n);
    for(int i=0;i<=n;i++) cout<<a[i]<<" ";
    return 0;
}

```

/* 通项公式:

```

for(int i=0;i<=k;i++) S(n,k) += ((-1)^(k-i) * i^n) / (i! * (k-i)!)

```

*/

6.13. FFT

```

const ll N=3001010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
typedef long double db;
const db pi = acos(-1.0);

```

```

int n,m;
int r[N];
complex <db> F[N],G[N],ans[N];

```

```

void FFT(complex <db> *A,int len,int cl) {
    for(int i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(int k=1;k<len;k<=1) {
        complex <db> w1( cos(pi/k), cl*sin(pi/k) );
        for(int j=0;j<len;j+=(k<=1)) {
            complex <db> wk(1,0);
            for(int i=0;i<k;i++,wk*=w1) {
                complex <db> x = A[i+j], y = wk*A[i+j+k];
                A[i+j] = x+y; A[i+j+k] = x-y;
            }
        }
    }
    if(cl==-1) for(int i=0;i<len;i++) A[i].real(A[i].real()/len);
}

```

```

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=1, L++;
}

```

```

    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

int main() {
    n = read(); m = read();
    int len = init(n+m+1);
    for(int i=0;i<=n;i++) F[i].real(read());
    for(int i=0;i<=m;i++) G[i].real(read());
    FFT(F,len,1);
    FFT(G,len,1);
    for(int i=0;i<len;i++) ans[i] = F[i] * G[i];
    FFT(ans,len,-1);
    for(int i=0;i<=n+m;i++) cout<<(int)round(ans[i].real())<<" ";
    return 0;
}

```

6.14. 第二类斯特林数 (列)

```

const ll N=801010;
const ll qwq=303030;
const ll inf=0x3f3f3f3f;
const ll p=167772161, g=3, gi=55924054;

ll n,m,K;
ll r[N];
ll a[N],b[N],c[N];
ll c1[N],c2[N],c3[N],c4[N],c5[N],c6[N],c7[N];
ll f[N],ni[N];

inline ll ksm(ll aa,ll bb) {
    ll sum = 1;
    while(bb) {
        if(bb&1) sum = sum * aa %p;
        bb >>= 1; aa = aa * aa %p;
    }
    return sum;
}

void qiu() {
    f[0] = ni[0] = 1;
    for(int i=1;i<=N-10;i++) f[i] = f[i-1] * i %p;
    ni[N-10] = ksm(f[N-10],p-2);
    for(int i=N-11;i>=1;i--) ni[i] = ni[i+1] * (i+1) %p;
}

void NTT(ll *A,ll len,ll cl) {
    for(ll i=0;i<len;i++) if(i<r[i]) swap(A[i],A[r[i]]);
    for(ll k=1;k<len;k<=<1) {
        ll g1 = ksm((cl==1)?g:gi, (p-1)/(k<<1));
        for(ll j=0;j<len;j+=(k<<1)) {
            ll gk = 1;
            for(ll i=0;i<k;i++,(gk*=g1)%=p) {
                ll x = A[i+j], y = gk*A[i+j+k]%p;
                A[i+j] = (x+y)%p; A[i+j+k] = (x-y+p)%p;
            }
        }
    }
}

```

```

    }
}
if(cl==1) return ;
ll inv = ksm(len,p-2);
for(ll i=0;i<len;i++) A[i] = A[i] * inv %p;
}

inline int init(int wo) {
    ll len = 1, L = 0; while(len<wo) len<=1, L++;
    for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
    return len;
}

void PMUL(ll *F,ll *G,ll n,ll m) { // F -> 1+...+x^n    G -> 1+...+x^m
    ll len = init(n+m+1);
    NTT(F,len,1);
    NTT(G,len,1);
    for(ll i=0;i<len;i++) F[i] = F[i] * G[i] %p;
    NTT(F,len,-1);
}

void PI(ll *F,ll *G,ll n) { // F*G = 1 (mod x^n)
    G[0] = ksm(F[0],p-2);
    ll *A=c1, *B=c2, k=1;
    for(ll len,L=1;k<(n+n);k<=1,L++) {
        len = k<<1;
        for(ll i=0;i<k;i++) A[i] = F[i], B[i] = G[i];
        for(ll i=0;i<len;i++) r[i] = (r[i>>1]>>1) | ((i&1)<<(L-1));
        NTT(A, len, 1);
        NTT(B, len, 1);
        for(ll i=0;i<len;i++) G[i] = (2ll - A[i] * B[i] %p + p) %p * B[i] %p;
        NTT(G, len, -1);
        for(ll i=k;i<len;i++) G[i] = 0;
    }
    for(ll i=0;i<k;i++) A[i] = B[i] = 0;
    for(ll i=n;i<k;i++) G[i] = 0;
}

void Pdao(ll *F,ll *G,ll n) {
    for(ll i=1;i<n;i++) G[i-1] = i*F[i] %p; G[n-1] = 0;
}

void Pji(ll *F,ll *G,ll n) {
    for(ll i=1;i<n;i++) G[i] = ksm(i,p-2)*F[i-1] %p; G[0] = 0;
}

void Pln(ll *F,ll *G,ll n) { // G = ln(F) (mod x^n)
    ll *A=c3, *B=c4;
    memset(c3,0,sizeof(c3));
    memset(c4,0,sizeof(c4));
    Pdao(F, A, n);
    PI(F, B, n);
    PMUL(A, B, n, n);
    Pji(A, G, n);
}

```

```

void Pexp(ll *F,ll *G,ll n) { // G = exp(F) (mod x^n)
    if(n==1) { G[0] = 1; return ; }
    Pexp(F, G, (n+1)>>1);
    ll *A=c5, *B=c6;
    for(ll i=0;i<=(n<<1);i++) A[i] = B[i] = 0;
    Pln(G, A, n);
    ll len = init(n+n);
    for(ll i=0;i<n;i++) B[i] = F[i];
    NTT(G, len, 1);
    NTT(A, len, 1);
    NTT(B, len, 1);
    for(ll i=0;i<len;i++) G[i] = (1ll-A[i]+B[i]+p) %p *G[i] %p;
    NTT(G, len, -1);
    for(ll i=n;i<len;i++) G[i] = 0;
}

void Pksm(ll *F,ll *G,ll K,ll n) { // G = F^K (mod x^n)
    ll *A=c7;
    Pln(F, A, n);
    for(int i=0;i<n;i++) A[i] = A[i] * K %p;
    Pexp(A, G, n);
}

void stling() {
    for(int i=0;i<=n;i++) a[i] = ni[i+1];
    Pksm(a, c, K, n+1);
    for(int i=n;i>=0;i--) {
        if(i>=K) c[i] = c[i-K];
        else c[i] = 0;
    }
    for(int i=0;i<=n;i++) c[i] = c[i] * f[i] %p * ni[K] %p;
}

int main() {
    qiu();
    n = read(); K = read();
    stling();
    for(int i=0;i<=n;i++) cout<<c[i]<<" ";
    return 0;
}

```

7. 数据结构

7.1. 李超树

```

#include <bits/stdc++.h>
#define ll long long
#define db double
#define ls now<<1
#define rs now<<1|1
using namespace std;
const int p1=39989;
const int p2=1000000000;
const double eps = 1e-9;

const int N=101010;
const int RR=40404;

```

```

int T,ans,cnt;
struct E{
    db K,B; int id;
}d[N],t[RR<<2];

inline void add(int x0,int y0,int x1,int y1) {
    ++cnt;
    if(x0==x1) d[cnt] = {0,(db)max(y0,y1),cnt};
    else d[cnt].K = (db)(y1-y0)/(x1-x0), d[cnt].B = y0-d[cnt].K*x0, d[cnt].id = cnt;
}

inline db w(E L,int x) { return L.K * x + L.B; }
bool cmp(E L1,E L2,int x) { if(fabs(w(L1,x)-w(L2,x))<eps) return L1.id<L2.id; return
w(L1,x)>w(L2,x); } // L1 > L2

void insert(int now,int l,int r,E g) {
    if( cmp(g,t[now],l) && cmp(g,t[now],r)) { t[now] = g; return; }
    if(!cmp(g,t[now],l) && !cmp(g,t[now],r)) return ;
    int mid = l+r >> 1;
    if(t[now].K < g.K) {
        if(cmp(g,t[now],mid)) insert(ls, l, mid, t[now]), t[now] = g;
        else insert(rs, mid+1, r, g);
    }
    else {
        if(cmp(g,t[now],mid)) insert(rs, mid+1, r, t[now]), t[now] = g;
        else insert(ls, l, mid, g);
    }
}

void update(int now,int l,int r,int x,int y,E g) {
    if(x<=l && r<=y) { insert(now, l, r, g); return ; }
    int mid = l+r >> 1;
    if(x<=mid) update(ls, l, mid, x, y, g);
    if(mid<y) update(rs, mid+1, r, x, y, g);
}

E query(int now,int l,int r,int x) {
    if(l==r) return t[now];
    int mid = l+r >> 1;
    E res = t[now];
    if(x<=mid) { E la = query(ls, l, mid, x); if(cmp(la,res,x)) res=la; }
    else { E ra = query(rs, mid+1, r, x); if(cmp(ra,res,x)) res=ra; }
    return res;
}

int main() {
    T = read();
    while(T--) {
        int cz;
        cz = read();
        if(cz==1) {
            int x0, y0, x1, y1;
            x0 = read(); y0 = read(); x1 = read(); y1 = read();
            if(x0>x1) swap(x0,x1), swap(y0,y1);
            add(x0,y0,x1,y1);
        }
    }
}

```

```

        update(1, 1, RR, x0, x1, d[cnt]);
    }
    else {
        int x = read();
        cout<<w(query(1, 1, RR, x),x)<<endl;
    }
}
return 0;
}

```

7.2. 点分树

```

/*
    求树上邻域权值和，单点修改
*/

#include<stdio.h>
#include<vector>
using namespace std;

inline int read(){
    int x=0,flag=1; char c=getchar();
    while(c<'0' || c>'9'){if(c=='-')flag=-1;c=getchar();}
    while(c>='0'&&c<='9'){x=(x<<1)+(x<<3)+c-48;c=getchar();}
    return flag*x;
}

const int N=1e5+7;

int fa[N][20],sz[N],dep[N],Fa[N][20],Dis[N][20],Dep[N],v[N];
vector<int> g[N],C[N][2];
bool vis[N];

inline void swap(int &x,int &y){x^=y^=x^=y;}
int Lca(int x,int y){
    if(dep[x]<dep[y]) swap(x,y);
    for(int i=16;~i;i--)
        if(dep[fa[x][i]]>=dep[y]) x=fa[x][i];
    if(x==y) return y;
    for(int i=16;~i;i--)
        if(fa[x][i]!=fa[y][i])
            x=fa[x][i],y=fa[y][i];
    return fa[x][0];
}

inline int lowbit(const int &x){return -x&x;}
inline void add(vector<int> &V,int x,int val){
    while(x<(int)V.size())V[x]+=val,x+=lowbit(x);
}
inline int query(vector<int> &V,int x){
    x=min((int)V.size()-1,x);
    int tmp=0;while(x)tmp+=V[x],x-=lowbit(x);return tmp;
}
int dis(int x,int y){return dep[x]+dep[y]-2*dep[Lca(x,y)];}

void dfs(int u){
    dep[u]=dep[fa[u][0]]+1;

```

```

    for(int i=1;i<17;i++)
        fa[u][i]=fa[fa[u][i-1]][i-1];
    for(int v:g[u])
        if(v!=fa[u][0]) fa[v][0]=u,dfs(v);
}

void count(int &s,int u,int f){
    s++;
    for(int v:g[u])
        if(v!=f&&!vis[v]) count(s,v,u);
}

void find_rt(int &rt,int &Min,int u,int f,const int &s){
    sz[u]=1; int tmp=0;
    for(int v:g[u])
        if(v!=f&&!vis[v]){
            find_rt(rt,Min,v,u,s);
            sz[u]+=sz[v],tmp=max(sz[v],tmp);
        }
    tmp=max(s-sz[u],tmp);
    if(tmp<Min) Min=tmp,rt=u;
}

void Dfs(int u,int f){
    int s=0,Min,rt;
    count(s,u,u),Min=s,rt=u;
    find_rt(rt,Min,u,u,s),vis[rt]=1;
    C[rt][0].resize(sz[u]+2);
    C[rt][1].resize(sz[u]+2);
    for(int i=0;i<17;i++){
        Fa[rt][i]=i? Fa[f][i-1]:f;
        if(!Fa[rt][i]) break;
        Dis[rt][i]=dis(rt,Fa[rt][i]);
        add(C[Fa[rt][i]][0],Dis[rt][i],v[rt]);
        add(C[i? Fa[rt][i-1]:rt][1],Dis[rt][i],v[rt]);
    }
    for(int v:g[rt])
        if(!vis[v]&&v!=f) Dfs(v,rt);
}

int main(){
    // freopen("P6329_1.in","r",stdin);
    // freopen("mine.txt","w",stdout);
    int n=read(),m=read(),ans=0;
    for(int i=1;i<=n;i++) v[i]=read();
    for(int i=1;i<=n;i++){
        int u=read(),v=read();
        g[u].push_back(v);
        g[v].push_back(u);
    }
    dfs(1);
    Dfs(1,0);
    // for(int i=1;i<=n;i++)
    //     printf("Fa[%d]=%d\n",i,Fa[i][0]);
    while(m--){
        int op=read();

```

```

    int x=read()^ans,y=read()^ans;
//    int x=read(),y=read();
    if(!op){
        int p=0;
        ans=query(C[x][0],y)+v[x];
        while(Fa[x][p]){
            if(y>Dis[x][p])
                ans+=query(C[Fa[x][p]][0],y-Dis[x][p])
                    -query(C[p? Fa[x][p-1]:x][1],y-Dis[x][p]);
            if(y>=Dis[x][p]) ans+=v[Fa[x][p]];
            p++;
        }
        printf("%d\n",ans);
    }else{
        int p=0;
        while(Fa[x][p]){
            add(C[Fa[x][p]][0],Dis[x][p],y-v[x]);
            add(C[p? Fa[x][p-1]:x][1],Dis[x][p],y-v[x]);
            p++;
        }
        v[x]=y;
    }
}
}

```

7.3. LCT

```

#include <bits/stdc++.h>
#define ll long long
#define QWQ cout<<"QwQ"<<endl;
#define FOR() int le=e[u].size();for(int i=0;i<le;i++)
#define ls son[x][0]
#define rs son[x][1]

using namespace std;
const int N=501010;
const int inf=0x3f3f3f3f;

int n,m,tot;
int fa[N],son[N][2],val[N],sum[N],rev[N];
int st[N],cnt;

inline void pushup(int x) { sum[x] = sum[ls] ^ sum[rs] ^ val[x]; }
inline void rever(int x) { rev[x] ^= 1; swap(ls,rs); }
inline void pushdown(int x) { if(rev[x]) { rev[x] = 0; if(ls) rever(ls); if(rs) rever(rs); } }
inline bool touhou(int x) { return son[fa[x]][1]==x; }
inline bool flandre(int x) { return son[fa[x]][0]==x || son[fa[x]][1]==x; }
inline void rotate(int x) {
    int y = fa[x], z = fa[y], k = touhou(x), w = son[x][k^1];
    if(flandre(y)) son[z][touhou(y)] = x; son[x][k^1] = y; son[y][k] = w;
    if(w) fa[w] = y; fa[y] = x; fa[x] = z; pushup(y); pushup(x);
}
inline void splay(int x) {
    int y = x; st[++cnt] = y;
    while(flandre(y)) st[++cnt] = (y=fa[y]);
    while(cnt) pushdown(st[cnt--]);
}

```

```

    while(flandre(x)) {
        y = fa[x];
        if(flandre(y)) {
            if(touhou(y)==touhou(x)) rotate(y);
            else rotate(x);
        }
        rotate(x);
    }
}

inline void access(int x) { for(int y=0;x;x=fa[y=x]) splay(x), son[x][1] = y,
pushup(x); }
inline void makert(int x) { access(x); splay(x); rever(x); }
inline void split(int x,int y) { makert(x); access(y); splay(y); }

int find(int x) { access(x); splay(x); while(ls) pushdown(ls), x = ls; splay(x);
return x; }
inline void link(int x,int y) { makert(x); if(find(y)!=x) fa[x] = y; }
inline void cut(int x,int y) { makert(x); if(find(y)==x && fa[y]==x && !son[y][0])
fa[y] = son[x][1] = 0, pushup(x); }

inline bool judge(int x,int y) { while(fa[x]) x = fa[x]; while(fa[y]) y = fa[y];
return x==y; } // 是否连通

int main() {
    int cz,x,y;
    n = read(); m = read();
    for(int i=1;i<=n;i++) val[i] = read();
    while(m--) {
        cz = read(); x = read(); y = read();
        if(cz==0) { split(x,y); cout<<sum[y]<<"\n"; }
        if(cz==1) { link(x,y); }
        if(cz==2) { cut(x,y); }
        if(cz==3) { splay(x); sum[x] ^= y ^ val[x]; val[x] = y; }
    }
    return 0;
}

```

7.4. 线段树合并 (雨天的尾巴)

```

#include <bits/stdc++.h>
#define FOR() int le=e[u].size();for(int i=0;i<le;i++)
#define ls L[now]
#define rs R[now]

using namespace std;
const int N=101010;
const int big = 100000;

int n,m;
vector <int> e[N];
int t[N*20],d[N*20],rt[N],cnt,L[N*20],R[N*20];
int dep[N],f[N][22];
int ans[N];

void DFS(int u,int fa) {
    f[u][0] = fa;

```

```

    dep[u] = dep[fa] + 1;
    FOR() {
        int v = e[u][i];
        if(v==fa) continue;
        DFS(v,u);
    }
}

inline int LCA(int x,int y) {
    if(dep[x]<dep[y]) swap(x,y);
    for(int k=20;k>=0;k--)
        if(dep[ f[x][k] ] >= dep[y]) x = f[x][k];
    if(x==y) return x;
    for(int k=20;k>=0;k--)
        if(f[x][k] != f[y][k])
            x = f[x][k], y = f[y][k];
    return f[x][0];
}

inline void pushup(int now) {    // 合并次数最多的颜色，相同则选择左侧
    if(d[ls]>=d[rs]) d[now] = d[ls], t[now] = t[ls];
    else            d[now] = d[rs], t[now] = t[rs];
}

void insert(int &now,int l,int r,int x,int g) {
    if(!now) now = ++cnt;
    if(l==r) { d[now] += g; t[now] = l; return ; }
    int mid = l+r >> 1;
    if(x<=mid) insert(ls, l, mid, x, g);
    else      insert(rs, mid+1, r, x, g);
    pushup(now);
}

int merge(int r1,int r2,int l,int r) {
    if(!r1 || !r2) return r1 + r2;
    if(l==r) {
        d[r1] += d[r2];
        return r1;
    }
    int mid = l+r >> 1;
    L[r1] = merge(L[r1], L[r2], l, mid);
    R[r1] = merge(R[r1], R[r2], mid+1, r);
    pushup(r1);
    return r1;
}

void hebing(int u) {
    FOR() {
        int v = e[u][i];
        if(v==f[u][0]) continue;
        hebing(v);
        rt[u] = merge(rt[u], rt[v], 1, big);
    }
    if(d[ rt[u] ]) ans[u] = t[ rt[u] ];
}

```

```

int main() {
    int x,y,z;
    n = read(); m = read();
    for(int i=1;i<n;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    DFS(1,0);
    for(int k=1;k<=20;k++)
        for(int i=1;i<=n;i++)
            f[i][k] = f[ f[i][k-1] ][k-1];
    while(m--) {
        x = read(); y = read(); z = read();
        int lca = LCA(x,y);
        insert(rt[x], 1, big, z, 1);
        insert(rt[y], 1, big, z, 1);
        insert(rt[lca], 1, big, z, -1);
        if(f[lca][0]) insert(rt[f[lca][0]], 1, big, z, -1);
    }
    hebing(1);
    for(int i=1;i<=n;i++) cout<<ans[i]<<"\n";
    return 0;
}

```

7.5. 换根树剖

```

#define ls now<<1
#define rs now<<1|1

using namespace std;
const int N=101010;
const int qwq=N<<2;
const int inf=0x3f3f3f3f;

int T;
int n,m,root = 1;
int a[N];
vector<int> e[N];
int son[N],fa[N],dep[N],siz[N];
int tp[N],id[N],w[N],cnt;
ll t[qwq],tag[qwq];

void DFS(int u,int f) {
    dep[u] = dep[f] + 1; siz[u] = 1; fa[u] = f;
    FOR() {
        int v = e[u][i];
        if(v==f) continue;
        DFS(v,u);
        siz[u] += siz[v];
        if(siz[v] > siz[son[u]]) son[u] = v;
    }
}

void DFS2(int u,int zuzu) {
    tp[u] = zuzu; id[u] = ++cnt; w[cnt] = a[u];
}

```

```

    if(son[u]) DFS2(son[u],zuzu);
    FOR() {
        int v = e[u][i];
        if(v==fa[u] || v==son[u]) continue;
        DFS2(v,v);
    }
}

inline void pushdown(int now,int l,int r) {
    if(!tag[now]) return ;
    int mid = l+r >> 1;
    tag[ls] += tag[now]; tag[rs] += tag[now];
    t[ls] += tag[now] * (ll)(mid-l+1); t[rs] += tag[now] * (ll)(r-mid);
    tag[now] = 0;
}

void built(int now,int l,int r) {
    if(l==r) { t[now] = w[l]; return ; }
    int mid = l+r >> 1;
    built(ls, l, mid);
    built(rs, mid+1, r);
    t[now] = t[ls] + t[rs];
}

inline void insert(int now,int l,int r,int x,int y,ll g) {
    if(x<=l && r<=y) { t[now] += (r-l+1) * g; tag[now] += g; return ; }
    int mid = l+r >> 1;
    pushdown(now, l, r);
    if(x<=mid) insert(ls, l, mid, x, y, g);
    if(y>mid) insert(rs, mid+1, r, x, y, g);
    t[now] = t[ls] + t[rs];
}

inline ll query(int now,int l,int r,int x,int y) {
    if(x<=l && r<=y) return t[now];
    ll res = 0, mid = l+r >> 1;
    pushdown(now, l, r);
    if(x<=mid) res += query(ls, l, mid, x, y);
    if(y>mid) res += query(rs, mid+1, r, x, y);
    return res;
}

inline int lca(int x,int y) {
    while(tp[x] != tp[y]) {
        if(dep[tp[x]] < dep[tp[y]]) swap(x,y);
        x = fa[tp[x]];
    }
    if(dep[x] > dep[y]) swap(x,y);
    return x;
}

inline int LCA(int x,int y) {
    if(dep[x]>dep[y]) swap(x,y);
    int rx = lca(x,root), ry = lca(y,root), xy = lca(x,y);
    if(xy==x) {
        if(rx==x) {

```

```

        if(ry==y) return y;
        return ry;
    }
    return x;
}
if(rx==x) return x;
if(ry==y) return y;
if((rx==root&&xy==ry) || (ry==root&&xy==rx)) return root;
if(rx==ry) return xy;
if(xy!=rx) return rx; return ry;
}

inline int find(int x,int y) { // find the son of x
    while(tp[x] != tp[y]) {
        if(dep[tp[x]] < dep[tp[y]]) swap(x,y);
        if(fa[tp[x]]==y) return tp[x];
        x = fa[tp[x]];
    }
    if(dep[x] > dep[y]) swap(x,y);
    return son[x];
}

inline void add(int x,ll g) {
    if(x==root) { t[1] += n * g; tag[1] += g; return ; }
    if(id[root]>id[x] && id[root]<=id[x]+siz[x]-1) { // root in x
        int y = find(x, root);
        t[1] += n * g; tag[1] += g;
        insert(1, 1, n, id[y], id[y]+siz[y]-1, -g);
    }
    else insert(1, 1, n, id[x], id[x]+siz[x]-1, g);
}

inline ll ask(int x) {
    if(x==root) return t[1];
    if(id[root]>id[x] && id[root]<=id[x]+siz[x]-1) {
        int y = find(x, root);
        return t[1] - query(1, 1, n, id[y], id[y]+siz[y]-1);
    }
    return query(1, 1, n, id[x], id[x]+siz[x]-1);
}

int main() {
    int cz,x,y,z;
    n = read(); m = read();
    for(int i=1;i<=n;i++) a[i] = read();
    for(int i=1;i<n;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    DFS(1,0);
    DFS2(1,1);
    built(1,1,n);
    while(m--) {
        cz = read();
        if(cz==1) root = read();
    }
}

```

```

        if(cz==2) {
            x = read(); y = read(); z = read();
            add(LCA(x,y), z);
        }
        if(cz==3) {
            x = read();
            printf("%lld\n",ask(x));
        }
    }
    return 0;
}

```

7.6. 可持久化 01tire

```

#define ls L[now]
#define rs R[now]

using namespace std;
const ll N=501010;
const ll qwq=N*35;
const ll inf=0x3f3f3f3f;

ll n,K;
ll ans;
ll a[N];
ll cnt[N];
ll rt[N],tot;
ll val[qwq];
ll L[qwq],R[qwq],siz[qwq];
struct E{
    ll zhi,id;
};
inline bool operator < (E A,E B) { return A.zhi < B.zhi; }
priority_queue <E> q;

void insert(ll &now,ll pre,ll h,ll v) {
    now = ++tot; siz[now] = siz[pre] + 1;
    ls = L[pre]; rs = R[pre];
    if(h== -1) { val[now] = v; return ; }
    if((v>>h)&1) insert(rs, R[pre], h-1, v);
    else insert(ls, L[pre], h-1, v);
}

ll query(ll now,ll k,ll h,ll v) {
    if(h== -1) { return v ^ val[now]; }
    if((v>>h)&1) {
        if(siz[ls] >= k) return query(ls, k, h-1, v);
        else return query(rs, k-siz[ls], h-1, v);
    }
    else {
        if(siz[rs] >= k) return query(rs, k, h-1, v);
        else return query(ls, k-siz[rs], h-1, v);
    }
}

int main() {

```

```

ll x;
n = read(); K = read();
for(ll i=1;i<=n;i++) {
    x = read(); a[i] = a[i-1] ^ x;
    insert(rt[i], rt[i-1], 31, a[i-1]);
    q.push( E{query(rt[i], ++cnt[i], 31, a[i]), i} );
}
ll ci = 0;
while(!q.empty()) {
    E now = q.top(); q.pop();
    ans += now.zhi;
    ci++;
    if(ci==K) break;
    ll u = now.id;
    if(cnt[u] < u)
        q.push( E{query(rt[u], ++cnt[u], 31, a[u]), u} );
}
cout<<ans;
return 0;
}

```

7.7. 树剖

```

int n,m,rt,p;
int a[N];
vector<int> e[N];
int siz[N],fa[N],son[N],dep[N];
int cnt,tp[N],id[N],w[N];
ll tree[qwq],tag[qwq];

void DFS1(int u,int f) {
    dep[u] = dep[f] + 1; siz[u] = 1; fa[u] = f;
    FOR() {
        int v = e[u][i];
        if(v==f) continue;
        DFS1(v,u);
        siz[u] += siz[v];
        if(siz[v] > siz[son[u]]) son[u] = v;
    }
}

void DFS2(int u,int zuzu) {
    tp[u] = zuzu; id[u] = ++cnt; w[cnt] = a[u];
    if(son[u]) DFS2(son[u],zuzu);
    FOR() {
        int v = e[u][i];
        if(v==fa[u] || v==son[u]) continue;
        DFS2(v,v);
    }
}

inline void add(int x,int y,int g) {
    while(tp[x] != tp[y]) {
        if(dep[tp[x]] < dep[tp[y]]) swap(x,y);
        insert(1, 1, n, id[tp[x]], id[x], g);
        x = fa[ tp[x] ];
    }
}

```

```

    if(dep[x] > dep[y]) swap(x,y);
    insert(1, 1, n, id[x], id[y], g);
}

inline int ask(int x,int y) {
    ll res = 0;
    while(tp[x] != tp[y]) {
        if(dep[tp[x]] < dep[tp[y]]) swap(x,y);
        (res += query(1, 1, n, id[tp[x]], id[x])) %= p;
        x = fa[ tp[x] ];
    }
    if(dep[x] > dep[y]) swap(x,y);
    (res += query(1, 1, n, id[x], id[y])) %= p;
    return res;
}

int main() {
    int x,y,z,cz;
    n = read(); m = read(); rt = read(); p = read();
    for(int i=1;i<=n;i++) a[i] = read();
    for(int i=1;i<n;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    DFS1(rt,rt);
    DFS2(rt,rt);
    built(1, 1, n);
    while(m--) {
        cz = read(); x = read();
        if(cz==1) { y = read(); z = read(); add(x,y,z); }
        if(cz==2) { y = read(); printf("%d\n",ask(x,y)); }
        if(cz==3) { z = read(); insert(1, 1, n, id[x], id[x]+siz[x]-1, z); }
        if(cz==4) { printf("%d\n",query(1, 1, n, id[x], id[x]+siz[x]-1)); }
    }
    return 0;
}

```

7.8. LCA (ST 版)

```

const int N=1101010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int T;
int n,m;
int ans,a[N];
vector<int> e[N];
int dfn[N],tot;
int dep[N];
int f[N][26],rec[N][26];
int ob[N];

void DFS(int u,int fa) {
    dep[u] = dep[fa] + 1;
    f[++tot][0] = dep[u];
    rec[tot][0] = u;
}

```

```

    dfn[u] = tot;
    FOR() {
        int v = e[u][i];
        if(v==fa) continue;
        DFS(v,u);
        f[++tot][0] = dep[u];
        rec[tot][0] = u;
    }
}

inline int LCA(int x,int y) {
    if(dfn[x]>dfn[y]) swap(x,y);
    int l = dfn[x], r = dfn[y], k = ob[r-l+1];
    if(f[l][k] < f[r-(1<<k)+1][k]) return rec[l][k];
    else return rec[r-(1<<k)+1][k];
}

int main() {
    ob[0] = -1; for(int i=1;i<=N-10;i++) ob[i] = ob[i>>1] + 1;
    int x,y,rt;
    n = read(); m = read(); rt = read();
    for(int i=1;i<n;i++) {
        x = read(); y = read();
        e[x].push_back(y);
        e[y].push_back(x);
    }
    DFS(rt,rt);
    for(int i=1;i<=tot;i++) cout<<rec[i][0]<<" ";
    for(int k=1;k<=22;k++) {
        for(int i=1;i+(1<<k-1)<=tot;i++) {
            if(f[i][k-1] < f[i+(1<<k-1)][k-1])
                f[i][k] = f[i][k-1], rec[i][k] = rec[i][k-1];
            else
                f[i][k] = f[i+(1<<k-1)][k-1], rec[i][k] = rec[i+(1<<k-1)][k-1];
        }
    }
    while(m--) {
        x = read(); y = read();
        cout<<LCA(x,y)<<"\n";
    }
    return 0;
}

```

7.9. 左偏树

```

#include <bits/stdc++.h>
#define ll long long
using namespace std;
const int N=101010;
const int inf=0x3f3f3f3f;

int n,m;
int a[N],fa[N],son[N][2];
int dep[N];

int merge(int x,int y) {
    if(x==0 || y==0) return x|y;

```

```

    if(a[x]>a[y] || (a[x]==a[y] && x>y) ) swap(x,y);
    son[x][1] = merge(son[x][1],y);
    if(dep[ son[x][0] ] < dep[ son[x][1] ]) swap(son[x][0],son[x][1]);
    dep[x] = dep[ son[x][1] ] + 1;
    return x;
}

int find(int x) { return fa[x]==x ? x : fa[x]=find(fa[x]); }

void pop(int x) {
    a[x] = -1;
    fa[x] = merge(son[x][0],son[x][1]);
    fa[ fa[x] ] = fa[x];
}

int main() {
    int cz,x,y;
    n = read(); m = read();
    dep[0] = -1;
    for(int i=1;i<=n;i++) a[i] = read(), fa[i] = i;
    for(int i=1;i<=m;i++) {
        cz = read();
        if(cz==1) {
            x = read(); y = read();
            if(a[x]==-1 || a[y]==-1) continue;
            int xx = find(x), yy = find(y);
            if(xx==yy) continue;
            fa[xx] = fa[yy] = merge(xx,yy);    // merge x tree and y tree
        }
        else {
            x = read();
            if(a[x]==-1) printf("-1\n");    // x has been deleted
            else {
                int rt = find(x);
                printf("%d\n",a[rt]);    // rt : the min val in x tree
                pop(rt);
            }
        }
    }

    return 0;
}

```

7.10. 李超树 ($kx+b$)

```

#include <bits/stdc++.h>
#define ll long long
#define ls L[now]
#define rs R[now]

typedef double db;
using namespace std;
const ll N=101010;
const ll inf=0x3f3f3f3f3f3f3f3f;

ll n;
ll Q;

```

```

struct E{
    ll k,b; // kx + b
    ll id;
}d[N],t[N*64],ling;
ll L[N*64],R[N*64],tot,rt;
ll max_X = 1e9;

ll calc(E wo,ll x) { return wo.k*x + wo.b; }

void outing(E wo) {
    cout<<" k="<<wo.k<<" b="<<wo.b<<" id="<<wo.id<<"\n";
}

void insert(ll &now,ll l,ll r,E v) {
    if(!now) { now = ++tot; t[now] = ling; }
    ll mid = l+r >> 1, p1 = calc(t[now],mid), p2 = calc(v,mid);
    // cout<<"t : "; outing(t[now]);
    // cout<<"v : "; outing(v);
    // cout<<endl;
    if(p1<p2 || (p1==p2 && t[now].id>v.id)) swap(t[now], v);
    if(l==r) return ;
    if(t[now].k > v.k) insert(ls, l, mid, v);
    else insert(rs, mid+1, r, v);
}

E query(ll &now,ll l,ll r,ll x) {
    if(!now) return ling;
    if(l==r) return t[now];
    ll mid = l+r >> 1;
    E res = t[now];
    if(x<=mid) res = query(ls, l, mid, x);
    else res = query(rs, mid+1, r, x);
    ll ct = calc(t[now],x), cr = calc(res,x);
    if(ct>cr || (ct==cr && t[now].id<res.id)) return t[now];
    else return res;
}

int main() {
    ll x;
    ling.b = -inf;
    n = read();
    for(ll i=1;i<=n;i++) {
        d[i].k = read(); d[i].b = read(); d[i].id = i;
        insert(rt, -max_X, max_X, d[i]);
    }
    Q = read();
    while(Q--) {
        x = read();
        E res = query(rt, -max_X, max_X, x);
        cout<<res.id<<" "<<calc(res,x)<<"\n";
    }
    return 0;
}

```

7.11. FHQ_Treap_Segment

```
struct Node
{
    ModInt Sum, val, tag;
    int ls, rs, key;
    int l, r, L, R;
} t[N];

int NewNode(int l, int r, ModInt val)
{
    static int cnt = 0;
    t[++cnt] = (Node){0, val, 0, 0, 0, rand(), l, r, l, r};
    t[cnt].Sum = ModInt(r - l + 1) * t[cnt].val;
    return cnt;
}

void update(int id)
{
    t[id].L = t[id].ls ? t[t[id].ls].L : t[id].l;
    t[id].R = t[id].rs ? t[t[id].rs].R : t[id].r;
    t[id].Sum = t[t[id].ls].Sum + t[t[id].rs].Sum + t[id].val * ModInt(t[id].r -
t[id].l + 1);
}

void add(int id, ModInt val)
{
    t[id].val += val;
    t[id].Sum += val * ModInt(t[id].R - t[id].L + 1);
    t[id].tag += val;
}

void pd(int id)
{
    if (!t[id].tag)
        return;
    if (t[id].ls)
        add(t[id].ls, t[id].tag);
    if (t[id].rs)
        add(t[id].rs, t[id].tag);
    t[id].tag = 0;
}

int merge(int x, int y)
{
    if (!x || !y)
        return x | y;
    if (t[x].key <= t[y].key)
    {
        pd(x);
        t[x].rs = merge(t[x].rs, y);
        return update(x), x;
    }
    else
    {
        pd(y);
        t[y].ls = merge(x, t[y].ls);
    }
}
```

```

        return update(y), y;
    }
}

void split(int id, int k, bool op, int &x, int &y)
{
    if (!id)
        x = y = 0;
    else
    {
        pd(id);
        if ((!op && t[id].l <= k) || (op && t[id].r <= k))
            x = id, split(t[x].rs, k, op, t[x].rs, y);
        else
            y = id, split(t[y].ls, k, op, x, t[y].ls);
        update(id);
    }
}

int n, m, rt;

void Split(int l, int r, int &x, int &y, int &z)
{
    static int sta[N], tp = 0;
    x = y = z = tp = 0;
    split(rt, l - 1, 0, x, y);
    split(y, r, 1, y, z);
    int now = x;
    while (t[now].rs)
        sta[tp++] = now, now = t[now].rs;
    if (now && t[now].r > r)
    {
        y = NewNode(l, r, t[now].val);
        z = merge(NewNode(r + 1, t[now].r, t[now].val), z);
        t[now].r = l - 1;
        do
        {
            update(now);
        } while (tp && (now = sta[--tp]));
    }
    else
    {
        if (now && t[now].r >= l)
        {
            y = merge(NewNode(l, t[now].r, t[now].val), y);
            t[now].r = l - 1;
            do
            {
                update(now);
            } while (tp && (now = sta[--tp]));
        }
        now = z, tp = 0;
        while (t[now].ls)
            sta[tp++] = now, now = t[now].ls;
        if (now && t[now].l <= r)
        {

```

```

        y = merge(y, NewNode(t[now].l, r, t[now].val));
        t[now].l = r + 1;
        do
        {
            update(now);
        } while (tp && (now = sta[--tp]));
    }
}
}

```

7.12. Splay(下标顺序)

```

#include <bits/stdc++.h>
#define QWQ cout<<"QwQ"<<endl;
#define ll long long
#define ls son[x][0]
#define rs son[x][1]
using namespace std;
const int N=501010;

int n,a[N];

int m,rt,tot;
int fa[N],son[N][2],siz[N],sum[N],val[N],rev[N];

inline void pushup(int x) {
    siz[x] = siz[ls] + siz[rs] + 1;
    sum[x] = sum[ls] + sum[rs] + val[x];
}

inline void pushdown(int x) {
    if(rev[x]) {
        rev[x] = 0;
        rev[ls] ^= 1; rev[rs] ^= 1;
        swap(son[ls][0],son[ls][1]);
        swap(son[rs][0],son[rs][1]);
    }
}

inline bool touhou(int x) { return son[fa[x]][1]==x; }
inline void rotate(int x) {
    int y = fa[x], z = fa[y], k = touhou(x), w = son[x][k^1];
    son[z][touhou(y)] = x; son[x][k^1] = y; son[y][k] = w;
    fa[x] = z; fa[y] = x; fa[w] = y; pushup(y); pushup(x);
}

inline void splay(int x,int goal) {
    while(fa[x]!=goal) {
        int y = fa[x], z = fa[y];
        if(z!=goal) {
            if(touhou(x)==touhou(y)) rotate(y);
            else rotate(x);
        }
        rotate(x);
    }
    if(!goal) rt = x;
}

inline int rnk(int k) {

```

```

    int x = rt;
    while(1) {
        pushdown(x);
        if(siz[ls]>=k) x = ls;
        else if(siz[ls]+1<k) k -= siz[ls]+1, x = rs;
        else return x;
    }
}

inline int Push(int v) {
    int x = ++tot;
    siz[x] = 1; val[x] = sum[x] = v; return x;
}

int built(int l,int r) {
    if(l>r) return 0;
    int mid = l+r >> 1, x = Push(a[mid]);
    if(ls=built(l,mid-1)) fa[ls] = x;
    if(rs=built(mid+1,r)) fa[rs] = x;
    pushup(x); return x;
}

inline void insert(int wei,int v) { //insert one num after one position
    int y = rnk(wei+1), z = rnk(wei+2);
    splay(y,0); splay(z,y); int x = Push(v);
    son[z][0] = x; fa[x] = z;
    pushup(z); pushup(y);
}

inline void del(int wei) {
    int y = rnk(wei), z = rnk(wei+2);
    splay(y,0); splay(z,y); int x = son[z][0];
    fa[x] = son[z][0] = 0;
    pushup(z); pushup(y);
}

inline void zhuan(int L,int R) {
    int y = rnk(L), z = rnk(R+2);
    splay(y,0); splay(z,y); int x = son[z][0];
    rev[x] ^= 1; swap(ls,rs);
}

inline int qiu(int L,int R) {
    int y = rnk(L), z = rnk(R+2);
    splay(y,0); splay(z,y);
    return sum[son[z][0]];
}

int main() {
    built(0,n+1); rt = 1;
    return 0;
}

```

7.13. Splay(大小顺序)

```

#include <bits/stdc++.h>
#define QWQ cout<<"QwQ"<<endl;
#define ll long long
#define ls son[x][0]
#define rs son[x][1]
using namespace std;

```

```

const int N=501010;
const int inf=0x3f3f3f3f;

int m,rt,tot;
int fa[N],son[N][2],siz[N],num[N],val[N];

/*
inline void access(ll x) {
    cnt = 0;
    st[++cnt] = x;
    while(fa[x]) st[++cnt] = (x=fa[x]);
    while(cnt) pushdown(st[cnt--]);
}
*/

inline int touhou(int x) { return son[fa[x]][1]==x; }
inline void pushup(int x) { siz[x] = siz[ls] + siz[rs] + num[x]; }
inline void rotate(int x) {
    int y = fa[x], z = fa[y], k = touhou(x), w = son[x][k^1];
    son[z][touhou(y)] = x; son[x][k^1] = y; son[y][k] = w;
    fa[w] = y; fa[y] = x; fa[x] = z; pushup(y); pushup(x);
}
inline void splay(int x,int goal) {
    // access(x);s
    while(fa[x]!=goal) {
        int y = fa[x], z = fa[y];
        if(z!=goal) {
            if(touhou(y)==touhou(x)) rotate(y);
            else rotate(x);
        }
        rotate(x);
    }
    if(!goal) rt = x;
}

inline void insert(int v) {
    int x = rt, f = 0;
    while(val[x]!=v && x)
        f = x, x = son[x][val[x]<v];
    if(x) num[x]++, pushup(x);
    else {
        x = ++tot;
        if(f) son[f][val[f]<v] = x;
        fa[x] = f; siz[x] = num[x] = 1;
        val[x] = v;
    }
    splay(x,0);
}

inline int rnk(int k) {
    int x = rt;
    while(1) {
        if(siz[ls]>=k) x = ls;
        else if(siz[ls]+num[x]<k) k -= siz[ls]+num[x], x = rs;
        else return x;
    }
}

```

```

}

inline void find(int v) {
    int x = rt;
    while(val[x]!=v && son[x][ val[x]<v ])
        x = son[x][ val[x]<v ];
    splay(x,0);
}

inline int qian(int v) {
    find(v);
    if(val[rt]<v) return rt;
    int x = son[rt][0];
    while(rs) x = rs; return x;
}

inline int hou(int v) {
    find(v);
    if(val[rt]>v) return rt;
    int x = son[rt][1];
    while(ls) x = ls; return x;
}

void del(int v) {
    int y = qian(v), x = hou(v);
    splay(y,0); splay(x,y);
    if(num[ls]>1) num[ls]--, siz[ls]--;
    else ls = 0;
    pushup(x); pushup(y);
}

inline void outing() {
    for(int i=0;i<=tot;i++) {
        cout<<i<<" : v = "<<val[i]<<" fa = "<<fa[i]<<" ls = "<<son[i][0]
        <<" rs = "<<son[i][1]<<" num = "<<num[i]<<" siz = "<<siz[i]<<endl;
    }
}

int main() {
    int cz,x;
    insert(1e9);
    insert(-1e9);
    m = read();
    while(m--) {
        cz = read(); x = read();
        if(cz==1) { insert(x); }
        if(cz==2) { del(x); }
        if(cz==3) { find(x); cout<<siz[son[rt][0]]+(val[rt]<x?num[rt]:0)<<"\n"; }
        if(cz==4) { cout<<val[rnk(x+1)]<<"\n"; }
        if(cz==5) { cout<<val[qian(x)]<<"\n"; }
        if(cz==6) { cout<<val[hou(x)]<<"\n"; }
    }

    return 0;
}

```

7.14. 线段树分治+可撤销并查集

```
#define ls now<<1
#define rs now<<1|1

using namespace std;
const int N=501010;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int T;
int n,m;
int da;
int L[N],R[N];
vector < pair<int,int> > t[N<<2];
struct CZ{
    int x,y,depy,tim;
}st[N];
int fa[N],tag[N],dep[N];
int cnt;

int find(int x) { return x==fa[x] ? x : find(fa[x]); }

void insert(int now,int l,int r,int x,int y,pair<int,int>pa) {
    if(x<=l && r<=y) { t[now].push_back(pa); return ; }
    int mid = l+r >> 1;
    if(x<=mid) insert(ls, l, mid, x, y, pa);
    if(y>mid) insert(rs, mid+1, r, x, y, pa);
}

void merge(int x,int y,int tim) {
    int xx = find(x), yy = find(y);
    if(xx==yy) return ;
    if(dep[xx] > dep[yy]) swap(xx,yy); // xx -> yy
    st[++cnt] = {xx, yy, dep[yy], tim};
    fa[xx] = yy;
    if(dep[xx]==dep[yy]) dep[yy]++;
    tag[xx] -= tag[yy];
}

void DFS(int now,int l,int r) {
    for(auto v : t[now]) {
        merge(v.first, v.second, now);
    }
    if(l==r) tag[find(l)]++;
    else {
        int mid = l+r >> 1;
        DFS(ls, l, mid);
        DFS(rs, mid+1, r);
    }
    while(st[cnt].tim==now) {
        CZ cz = st[cnt--];
        tag[cz.x] += tag[cz.y];
        fa[cz.x] = cz.x;
        dep[cz.y] = cz.depy;
    }
}
```

```

}

int main() {
    int x,y;
    n = read(); m = read();
    for(int i=1;i<=n;i++) L[i] = read(), R[i] = read(), da = max(da,R[i]);
    for(int i=1;i<=m;i++) {
        x = read(); y = read();
        int le = max(L[x], L[y]);
        int re = min(R[x], R[y]);
        if(le<=re) insert(1, 1, da, le, re, pair<int,int>{x,y});
    }
    for(int i=1;i<=n;i++) fa[i] = i;
    DFS(1, 1, da);
    for(int i=1;i<=n;i++) if(tag[i]) cout<<i<<" ";
    return 0;
}

```

7.15. MST

```

struct MST
{
    const int N = 5e4 + 7;
    int fa[N], rk[N];
    MST(int n)
    {
        for (int i = 1; i <= n; i++)
            fa[i] = i, rk[i] = 1;
    }
    int find(int x)
    {
        if (fa[x] == x)
            return x;
        return fa[x] = find(fa[x]);
    }
    void merge(int x, int y)
    {
        x = find(x), y = find(y);
        if (x == y)
            return;
        if (rk[x] < rk[y])
            swap(x, y);
        rk[x] = max(rk[x], rk[y] + 1);
        fa[y] = x;
    }
} T;

```

7.16. 主席树

```

#include <bits/stdc++.h>
#define ll long long
#define ls L[now]
#define rs R[now]
const int N=1010101;

int n,m;
int a[N],rt[N],tot;
int t[N*33], L[N*33], R[N*33];

```

```

inline void pushup(int now) { t[now] = t[ls] + t[rs]; }
void built(int &now,int l,int r) {
    now = ++tot;
    if(l==r) { t[now] = a[l]; return ; }
    int mid = l+r >> 1;
    built(ls, l, mid);
    built(rs, mid+1, r);
    pushup(now);
}

void insert(int &now,int pre,int l,int r,int we,int g) {
    now = ++tot;
    ls = L[pre]; rs = R[pre];
    if(l==r) { t[now] = g; return ; }
    int mid = l+r >> 1;
    if(we<=mid) insert(ls, L[pre], l, mid, we, g);
    else insert(rs, R[pre], mid+1, r, we, g);
    pushup(now); return ;
}

int query(int now,int l,int r,int x,int y) {
    if(x<=l && r<=y) return t[now];
    int mid = l+r >> 1, res = 0;
    if(x<=mid) res += query(ls, l, mid, x, y);
    if(y>mid) res += query(rs, mid+1, r, x, y);
    return res;
}

int main() {
    int x,y,z,w;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    built(rt[0],1,n);
    for(int i=1;i<=m;i++)
    {
        scanf("%d%d%d",&x,&y,&z);
        if(y==1)
        {
            scanf("%d",&w);
            insert(rt[i], rt[x], 1, n, z, w);
        }
        else
        {
            printf("%d\n",query(rt[x], 1, n, z, z));
            rt[i] = rt[x];
        }
    }
    return 0;
}

```

7.17. 李超树可持久化

```

#include <bits/stdc++.h>
#define ll long long
#define ls L[now]

```

```

#define rs R[now]

typedef double db;
using namespace std;
const ll N=101010;
const ll inf=0x3f3f3f3f3f3f3f;

ll n;
ll Q;
struct E{
    ll k,b; // kx + b
    ll id;
}d[N],t[N*64],ling;
ll L[N*64],R[N*64],tot,rt[N];
ll max_X = 1e9;

ll calc(E wo,ll x) { return wo.k*x + wo.b; }

void outing(E wo) {
    cout<<" k="<<wo.k<<" b="<<wo.b<<" id="<<wo.id<<"\n";
}

void insert(ll &now,ll pre,ll l,ll r,E v) {
    now = ++tot;
    if(!pre) {t[now] = ling; }
    else {
        t[now] = t[pre];
        L[now] = L[pre];
        R[now] = R[pre];
    }
    ll mid = l+r >> 1, p1 = calc(t[now],mid), p2 = calc(v,mid);
    // cout<<"t : "; outing(t[now]);
    // cout<<"v : "; outing(v);
    // cout<<endl;
    if(p1<p2 || (p1==p2 && t[now].id>v.id)) swap(t[now], v);
    if(l==r) return ;
    if(t[now].k > v.k) insert(ls, L[pre], l, mid, v);
    else insert(rs, R[pre], mid+1, r, v);
}

E query(ll &now,ll l,ll r,ll x) {
    if(!now) return ling;
    if(l==r) return t[now];
    ll mid = l+r >> 1;
    E res = t[now];
    if(x<=mid) res = query(ls, l, mid, x);
    else res = query(rs, mid+1, r, x);
    ll ct = calc(t[now],x), cr = calc(res,x);
    if(ct>cr || (ct==cr && t[now].id<res.id)) return t[now];
    else return res;
}

int main() {
    ll r,x;
    ling.b = -inf;

```

```

n = read();
for(ll i=1;i<=n;i++) {
    d[i].k = read(); d[i].b = read(); d[i].id = i;
    insert(rt[i], rt[i-1], -max_X, max_X, d[i]);
}
Q = read();
while(Q--) {
    r = read(); x = read();
    E res = query(rt[r], -max_X, max_X, x);
    cout<<res.id<<" "<<calc(res,x)<<"\n";
}
return 0;
}

```

7.18. ST 表

```

const int N=2020200;
const int qwq=303030;
const int inf=0x3f3f3f3f;

int n,Q;
int a[N];
int ob[N],ma[N][22];

int ask(int l,int r) {
    int k = ob[r-l+1];
    return max(ma[l][k], ma[r-(1<<k)+1][k]);
}

void built() {
    ob[0] = -1; for (int i=1;i<=N-10;i++) ob[i] = ob[i>>1] + 1;
    for (int k=1;k<=21;k++)
        for (int i=1;i+(1<<(k-1))<=n;i++)
            ma[i][k] = max(ma[i][k-1], ma[i+(1<<(k-1))][k-1]);
}

int main() {
    int x,y;
    n = read(); Q = read();
    for(int i=1;i<=n;i++) a[i] = read(), ma[i][0] = a[i];
    built();
    while(Q--) {
        x = read(); y = read();
        cout<<ask(x,y)<<"\n";
    }
    return 0;
}

```

7.19. DSU

```

void JI(int u) {
    nowans += f[u];
    FOR() JI(e[u][i]);
}

void JIA(int u,int cl) {
    f[u] += cl;
}

```

```

    FOR() JIA(e[u][i],cl);
}

void DSU(int u,bool shan) {
    FOR() {
        int v = e[u][i];
        if(v==son[u]) continue;
        DSU(v,1);
    }

    if(son[u]) DSU(son[u],0);

    for(int i=0;i<le;i++) {
        int v = e[u][i];
        if(v==son[u]) continue;
        JI(v); JIA(v,1);
    }

    if(shan) JIA(u,-1);
}

```

7.20. FHQ_Treap

```

#include <iostream>
#include <cstdlib>
#include <algorithm>
#include <vector>
#include <stack>

using namespace std;

const int N = 5e5 + 7;

struct FHQ_TREAP
{
    struct Node
    {
        int ls, rs, key, sz, val;
        Node(int l_ = 0, int r_ = 0, int key_ = 0, int sz_ = 0, int val_ = 0)
            : ls(l_), rs(r_), key(key_), sz(sz_), val(val_)
        {
            if (!key)
                key = rand() + 1;
        }
    };

    vector<Node> t;
    stack<int> pool;
    int nd_cnt, rt, x, y, z;

    FHQ_TREAP()
    {
        srand(114514);
        t.push_back(Node());
        rt = nd_cnt = 0;
    }
}

```

```

int node(int x)
{
    Node tmp(0, 0, 0, 1, x);
    if (!pool.empty())
    {
        int id = pool.top();
        pool.pop();
        t[id] = tmp;
        return id;
    }
    else
    {
        t.push_back(tmp);
        return ++nd_cnt;
    }
}

void update(int id)
{
    t[id].sz = 1 + t[t[id].ls].sz + t[t[id].rs].sz;
}

int merge(int x, int y)
{
    if (!x || !y)
        return x + y;
    if (t[x].key < t[y].key)
    {
        t[x].rs = merge(t[x].rs, y);
        update(x);
        return x;
    }
    else
    {
        t[y].ls = merge(x, t[y].ls);
        update(y);
        return y;
    }
}

void split(int id, int k, int &x, int &y)
{
    if (!id)
        x = y = 0;
    else
    {
        if (t[id].val <= k)
            x = id, split(t[id].rs, k, t[id].rs, y);
        else
            y = id, split(t[id].ls, k, x, t[id].ls);
        update(id);
    }
}

void Insert(int val)
{

```

```

        z = node(val);
        split(rt, val - 1, x, y);
        rt = merge(x, merge(z, y));
    }

void Delete(int val)
{
    split(rt, val - 1, x, y);
    split(y, val, y, z);
    pool.push(y);
    y = merge(t[y].ls, t[y].rs);
    rt = merge(merge(x, y), z);
}

int get_rank(int val)
{
    split(rt, val - 1, x, y);
    int tmp = t[x].sz + 1;
    rt = merge(x, y);
    return tmp;
}

int get_kth(int rk)
{
    int id = rt;
    while (1)
    {
        if (rk <= t[t[id].ls].sz)
            id = t[id].ls;
        else if (rk == t[t[id].ls].sz + 1)
            return t[id].val;
        else
        {
            rk -= (t[t[id].ls].sz + 1);
            id = t[id].rs;
        }
    }
}

int get_pre(int val)
{
    split(rt, val - 1, x, y);
    int id = x;
    while (t[id].rs)
        id = t[id].rs;
    rt = merge(x, y);
    return t[id].val;
}

int get_suc(int val)
{
    split(rt, val, x, y);
    int id = y;
    while (t[id].ls)
        id = t[id].ls;
    rt = merge(x, y);
}

```

```
        return t[id].val;
    }
} T;
```